

Verification Study — CV32E40P (RV32IM)

Repository ARETRAIL — ORPLAN-driven study of the Databook, the RTL, the verification guidelines, the verification plan and the testbench architecture.

Verification Study - CV32E40P (RV32IM), repository **ARETRAIL**

Purpose of this document

This is a teaching document. It is written for a student who knows basic programming but is new to digital design, RTL, CPUs, RISC-V, SystemVerilog and UVM. Every technical word is explained the first time it is used.

It follows the study order that was requested:

1. Databook -> what the DUT (Design Under Test) claims to do.
2. RTL -> what the DUT actually does.
3. Guidelines-> what we are required to verify.
4. ORPLAN -> the verification plan ([ourplan/risc_v_verification_plan \(1\).xlsx](#)).
5. Existing testbench architecture ([architecture/uvm_architecture_pro_updated.drawio](#)).
6. Comparison of ORPLAN against that architecture.
7. Minimal, ORPLAN-driven modifications to the draw.io diagram.
8. Explanation of every modification.

Evidence labels used throughout

Label	Meaning
FACT	Directly supported by a file in this repository (Databook text, RTL code, guidelines Q&A, ORPLAN cell).
INFERENCE	Derived by combining two or more pieces of repository evidence. The reasoning is shown.
UNKNOWN	Cannot be determined from the repository. Listed again in section 13.

Repository map (what everything is)

Path	What it is
DATABOOK/	The Databook: the official CV32E40P User Manual (OpenHW Group), exported as reStructuredText sources (<code>_sources/*.rst.txt</code>) plus images. This is the design specification.
rtl/	The RTL source code of the DUT (28 SystemVerilog files + 3 packages).
guidelines/Questions QRE Final project status 2026/	The verification guidelines: questions asked by the teams and the answers given by the supervisor. This document fixes the verification scope.
ourplan/risc_v_verification_plan (1).xlsx	ORPLAN - the verification plan (5 sheets: System, Generation_, Checking_, Coverage_, test list).
plantemplate/	The empty plan template (the same 5 sheets, filled with an APB/AHB example). ORPLAN follows this template exactly.
PLANIDEAS/	Earlier drafts / ideas: DRs (Design Requirements), VRs (Verification Requirements), scenarios, and two Markdown plan drafts. Useful as background; ORPLAN supersedes them.
architecture/uvm_architecture_pro_updated.drawio (+ .jpg)	The existing testbench architecture diagram (draw.io XML). This is what we must update.
standard/riscv-spec-20191213.pdf	The official RISC-V User-Level ISA specification (the ISA reference for opcode/encoding/corner-case rules).
parameters/parameters.png	A screenshot of core parameters (see section 13: I could not inspect image content).

1. Project Overview

1.1 What is the project?

We are building a **verification environment** (a "testbench") for a RISC-V processor core called **CV32E40P**, and we are only verifying two parts of it:

- **RV32I** - the base integer instruction set (add, subtract, compare, shift, branch, jump, load, store ...).
- **RV32M** - the integer multiply/divide extension (MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU).

Everything else (floating point, compressed instructions, CSRs, interrupts, debug, sleep, performance counters, custom PULP instructions ...) is **out of scope** - but some of it must still be driven to a harmless value so that the core works normally.

1.2 What is "verification"?

- **Design** = writing the RTL (the hardware description).
- **Verification** = proving, by simulation, that the RTL behaves like its specification.

We do **not** re-write the design. We build a second "world" around it (the testbench) that (a) stimulates it, (b) watches it, and (c) automatically decides PASS or FAIL.

1.3 The three documents that define our job

Document	Role	Analogy
Databook (DATABOOK/)	Describes the design	The "product manual"
Guidelines (guidelines/)	Describes the scope of our work	The "contract / statement of work"
ORPLAN (ourplan/...xlsx)	Describes what we will check and how	The "test plan"

A verification item is only legitimate if it can be traced back through all three:

Databook says X -> Guideline says "verify X" -> ORPLAN row describes how -> Testbench component implements it

2. Fundamentals you need before anything else

Read this section once. Everything later assumes it.

2.1 Digital design fundamentals

- **Signal** - a wire that carries one bit (0 or 1). A group of wires is a **bus**, e.g. **data_addr_o[31:0]** is 32 bits.
- **Clock (clk_i)** - a signal that toggles 0->1->0 forever. Flip-flops (memory elements) copy their input to their output on a clock edge. One clock period = one **cycle**.
- **Reset (rst_ni)** - a signal that forces every flip-flop to a known value. Here it is **active-low** (the **n** in **rst_n**): when **rst_ni == 0** the core is held in reset. It is **asynchronous**, meaning it works immediately, without waiting for a clock edge. [FACT] DATABOOK/ sources/integration.rst.txt: "**rst_ni** ... Active-low asynchronous reset".
- **Combinational logic** - output depends only on the current inputs (no memory), e.g. an adder.
- **Sequential logic** - output depends on the past (flip-flops), e.g. a counter.
- **RTL (Register Transfer Level)** - code (here SystemVerilog) that describes hardware as registers + combinational logic between them.
- **Module / instance** - a module is a reusable hardware block (**cv32e40p_load_store_unit**); an instance is one copy of it inside a bigger module.
- **X propagation** - "X" means unknown in simulation. If an input is left unconnected (floating), X can spread through the design and make results meaningless. That is why ORPLAN row **risc_sys_01** demands that **every** port be connected or tied, with "no X propagation".

2.2 CPU fundamentals

- **CPU (processor)** - a machine that repeatedly: fetch an instruction from memory -> decode what it means -> execute it -> write back the result.
- **Instruction** - a 32-bit binary word that encodes one operation, e.g. **ADD x1, x2, x3**.
- **ISA (Instruction Set Architecture)** - the contract between hardware and software: the list of instructions, their encodings and their meaning. Here: **RISC-V RV32I + RV32M**.
- **PC (Program Counter)** - a register holding the address of the next instruction to fetch.
- **Register file** - 32 fast storage locations inside the CPU, called **x0...x31**, each 32 bits wide. **x0** is special: it is **hardwired to zero** - reading it always gives 0 and writing to it is discarded. [FACT] DATABOOK/ sources/register_file.rst.txt; RTL **cv32e40p_register_file_ff.sv** (**mem[0] <= 32'b0**; in both reset and normal operation).
- **ALU (Arithmetic Logic Unit)** - the combinational block that does add/sub/and/or/xor/compare/shift.
- **Load / Store** - load = read from memory into a register (**LW, LH, LB, LHU** ...); store = write a register to memory (**SW, SH, SB**).
- **Byte / halfword / word** - 8 / 16 / 32 bits. A **word** is 4 bytes.
- **Alignment** - an address is "naturally aligned" for a size if it is a multiple of that size: a word access should be a multiple of 4 (**addr[1:0] == 00**), a halfword a multiple of 2 (**addr[0] == 0**). If not, the access is **misaligned**.

(Section 8.4.4 explains this in the depth the task requires.)

- **Sign extension** - when a small value (e.g. 8 bits) is placed in a 32-bit register, the missing upper bits are filled with copies of the sign bit (**LB, LH**) or with zeros (**LBU, LHU**).
- **Trap / exception** - an event that forces the CPU to stop the normal instruction stream and jump to a handler. An **exception** is caused by the instruction itself (e.g. illegal instruction); an **interrupt** is caused by the outside world (**irq_i**). The address jumped to is the **trap vector base (mtvec)**.
- **CSR (Control and Status Register)** - special registers that configure/record the CPU state (**mstatus, mie, mepc, mtvec, ...**). They are accessed with the **Zicsr** instructions, which are **out of scope** for us.

2.3 Pipeline fundamentals (this core has 4 stages)

A **pipeline** is like an assembly line: while instruction n is being executed, instruction $n+1$ is being decoded and $n+2$ is being fetched. The CV32E40P has 4 stages [FACT] ([DATABOOK/sources/pipeline.rst.txt](#)):

Stage	Name	What happens there
IF	Instruction Fetch	Ask memory for instruction words through a prefetch buffer (a small FIFO that can hold 2 words); pre-decode compressed instructions.
ID	Instruction Decode	Decode the instruction, read the register file, compute jump targets. Jumps are resolved here. ALU / multiplier / divider run; branches are resolved here; the load/store address is computed here. Multi-cycle instructions stall this stage.
EX	Execute	ALU/MUL/DIV results are written back to the register file from EX.
WB	Write-Back	Load data returning from memory is written back to the register file.

Three words you must know:

- **Stall** - freeze some stages for one or more cycles because data is not ready yet (e.g. waiting for memory, or a slow divider).
- **Flush** - throw away instructions that were fetched but must not execute (because a branch/jump/trap changed the PC). Those discarded slots are often called **bubbles**.
- **Hazard** - a situation where the pipeline would wrongly execute something:
- **RAW (Read-After-Write)** - an instruction needs a value that a previous instruction has not written yet. Solved by **forwarding** (also called **bypassing**): sending the result directly from EX back to ID so the consumer does not have to wait. [FACT] pipeline chapter: "There is a forwarding path between ALU, Multiplier and Divider result in EX stage and ID stage flip-flops to avoid the need of a write-through register file... allows to have 0-cycle penalty".
- **Load-use hazard** - the consumer sits immediately after a load; the data only arrives at the end of WB, so forwarding is too late -> **1 stall cycle**. [FACT] pipeline chapter: "CV32E40P experiences a 1-cycle penalty on the following hazards: Load data hazard ... Jump register (jalr) data hazard".
- **Control hazard** - after a branch/jump we fetched the wrong instructions -> they must be flushed.
- **WAW (Write-After-Write)** - two writes to the same register in the same cycle. Here this can happen because there are **two write ports** (one from EX, one from WB); the register file resolves it by priority (see section 8.4.10).

2.4 Bus / protocol fundamentals (OBI)

The core talks to the outside world through two **OBI (Open Bus Interface)** ports [FACT] ([DATABOOK/sources/intro.rst.txt](#), "Bus Interfaces").

OBI has two phases per transaction:

1. **Address (request) phase** - the master (the core) puts an address on the bus and raises **req**. The slave (our testbench memory) answers with **gnt** (grant) when it accepts.
2. **Response phase** - later, the slave raises **rvalid** for exactly one cycle and puts the data on **rdata**. For a write, **rvalid** still marks the end of the transaction (but **rdata** is meaningless).

The OBI rules that matter for us [FACT] ([intro.rst.txt](#), "Memory-Protocol"; [instruction_fetch.rst.txt](#); [load_store_unit.rst.txt](#)):

- A request must **not be retracted**: once **req** is raised it stays high until **gnt** is seen, and the address must stay stable meanwhile.
- **req** must **not combinatorially depend on rvalid** (no **rvalid** -> **req** combinational path).
- **gnt** may be asserted **even when there is no request**; that must cause no side effect.
- Responses must come back **in the order the requests were issued** (the core does not use transaction IDs).
- The interfaces have **no err signal** (no error response) - the environment is expected never to fail a transaction.
- Instruction side: up to **2 outstanding** transactions (prefetch FIFO depth = 2). Data side: up to **2 outstanding** transactions.

2.5 Verification fundamentals (UVM)

UVM (Universal Verification Methodology) is a standard way (a library + conventions, written in SystemVerilog) to structure a testbench. The vocabulary you will see in the architecture diagram:

Term	Plain-English meaning
DUT	Design Under Test - here cv32e40p_top. The thing we verify.
Testbench (TB)	Everything that is not the DUT: stimulators, watchers, comparators, memories.
Test	One simulation run with a specific purpose (e.g. risc_div_corner_test). It chooses which stimulus to run.
Interface	A bundle of the DUT's physical pins with a name, so components can talk to them without knowing individual signals. A virtual interface is a handle (pointer) to such an interface, stored in the configuration database.
Transaction	One meaningful "packet" of activity described at a high level, e.g. "read word at address 0x1000" or "MUL x5, x6, x7". Components create, send, observe and compare transactions instead of wiggling individual wires.
Sequence	A small program that generates a stream of transactions (e.g. "2000 random ALU instructions").
Sequencer	A mailbox/arbiter that hands transactions from sequences to a driver.
Driver	Converts transactions into pin-level activity on the interface (it drives the wires).
Monitor	Watches the interface and converts pin activity back into transactions. It never drives anything.
Agent	A container that groups the pieces belonging to one interface: sequencer + driver + monitor. An active agent drives and watches; a passive agent only watches.
Analysis port / export / imp	UVM's broadcast plumbing: a monitor has an analysis port (ap) and publishes transactions; subscribers (scoreboard, coverage, reference model) expose an export/imp and receive them. In the diagram: green diamond = analysis port, red circle = export/imp.
Reference model (golden model / predictor)	An independent, simple, obviously-correct model of what the DUT should do (here: an RV32IM instruction simulator). It produces the expected result.
Scoreboard	The judge: it compares expected (from the reference model) with actual (from the monitors) and reports PASS/FAIL.
Checker / assertion	A rule that is checked continuously (e.g. "address must be stable while waiting for grant"). SVA = SystemVerilog Assertions.
Coverage	A measure of what has been exercised, not whether it was right. A covergroup defines bins (interesting values/cases); we then check that all bins were hit. Functional coverage = coverage of features; code coverage = coverage of RTL lines (out of our scope here).
Environment (env)	The container that holds all agents, the reference model, the scoreboard and the coverage, for one DUT.
Configuration object (env_cfg)	A small object holding the knobs of the environment (e.g. "zero-wait memory", "long stall bursts").
Virtual sequencer / virtual sequence	A sequencer (resp. sequence) that coordinates several agents at once - e.g. "start a program and inject random bus delays and re-assert reset after 500 cycles".
Memory model	A testbench model of a memory that behaves like a real OBI slave (it holds the program or the data and answers reads/writes).

The classic UVM data flow (memorize this; everything later is an instance of it):

```

Test
  v chooses and starts
Sequence (creates transactions = stimulus)
  v
Sequencer (arbitrates/delivers)
  v
Driver (transactions -> pin wiggles)
  v
Interface (virtual pins)
  v
DUT (executes / processes)

```

```

      v
Interface
      v
Monitor  (pin wiggles -> observed transactions)
      v
Reference model      (observed)
      v expected
Scoreboard <- compares expected vs observed -> PASS / FAIL
      v
Coverage  (records what was seen)

```

3. DUT Overview

3.1 What the DUT is

FACT - DATABOOK/_sources/intro.rst.txt:

"CV32E40P is a 4-stage in-order 32-bit RISC-V processor core."

Key properties:

- **32-bit:** registers are 32 bits, addresses are 32 bits.
- **In-order:** instructions complete in program order (no out-of-order tricks), which makes verification much easier: we can predict the result instruction by instruction.
- **4 stages:** IF -> ID -> EX -> WB (section 2.3).
- **Single hart:** one hardware thread, one program counter. Therefore memory accesses are naturally sequential.
- **Machine mode only** (M-mode): the privileged spec's User mode is not implemented [**FACT**] **intro.rst.txt:** "RV32A Extensions, Security and Memory Protection ... CV32E40P core does not support the RV32A (atomic) extensions, the U-mode, and the PMP anymore."

3.2 Where the DUT boundary is

FACT - DATABOOK/_sources/integration.rst.txt: "The main module is named **cv32e40p_top** and can be found in **cv32e40p_top.sv**." And ORPLAN **risc_sys_00**: "DUT is **cv32e40p_top** with default parameters and configurations. **top_tb** instantiates **cv32e40p_top.sv** and verifies the core as a whole at top level."

So our DUT is the **whole core**, not an internal block. This matters: it fixes the verification style (see the guidelines in section 6 - top-level / black-box with scoreboards on the datapath, not one scoreboard per module).

cv32e40p_top (FACT **rtl/cv32e40p_top.sv**) instantiates **cv32e40p_core** and, **only if FPU == 1**, an FPU wrapper (**cv32e40p_fp_wrapper**) behind an APU interface. With the default parameters (**FPU = 0**) the wrapper is not instantiated and the APU signals are driven to 0 (**no_fpu_gen** branch). **Note:** the file **cv32e40p_fp_wrapper.sv** is not present in this repository's **rtl/** folder; with **FPU = 0** it is never needed. [**FACT**]

3.3 Parameters (the "configuration knobs")

FACT - rtl/cv32e40p_top.sv header and **DATABOOK/_sources/integration.rst.txt:**

Parameter	Default	Meaning
COREV_PULP	0	Enable custom PULP ISA extensions + custom CSRs (post-increment loads/stores, hardware loops ...)
COREV_CLUSTER	0	Enable PULP Cluster support (cv.elw)
FPU	0	Enable the floating-point unit
FPU_ADDMUL_LAT	0	Pipeline registers for FP add/mul
FPU_OTHERS_LAT	0	Pipeline registers for other FP ops
ZFINX	0	FP instructions use the integer register file
NUM_MHPMCOUNTERS	1	Number of performance counters

ORPLAN **risc_sys_00** says: "**default parameters and configurations**". That means: **COREV_PULP = 0**, **COREV_CLUSTER = 0**, **FPU = 0**, **ZFINX = 0**, **NUM_MHPMCOUNTERS = 1**. With those values the core is a **vanilla RV32IM + Zicsr + Zicntr + Zifencei + C** machine [**FACT**] (**intro.rst.txt:** C, M, Zicntr, Zicsr, Zifencei are "always enabled").

[!] Important consequence: **C (compressed instructions) is always enabled in hardware**, even though we do not generate compressed programs. This is why ORPLAN **risc_if_09** can state that an instruction-address-misaligned exception can never happen (section 8.4.2).

3.4 The port list (the DUT's complete interface to the world)

FACT - rtl/cv32e40p_top.sv, DATABOOK/_sources/integration.rst.txt.

Port	Width	Dir	What it does	In our scope?
clk_i	1	in	Clock	[OK] driven

Port	Width	Dir	What it does	In our scope?
rst_ni	1	in	Active-low asynchronous reset	[OK] driven
scan_cg_en_i	1	in	DFT: force clock gates on. Must be 0 in normal operation	[OK] tied to 0
pulp_clock_en_i	1	in	PULP cluster clock enable (unused when COREV_CLUSTER = 0)	[OK] tied to 0
core_sleep_o	1	out	"core is sleeping" (WFI / cv.elw)	[obs] observed, not checked (sleep out of scope)
fetch_enable_i	1	in	Gates the very first instruction fetch after reset	[OK] driven
boot_addr_i	32	in	PC after reset (must be half-word aligned)	[OK] driven
mtvec_addr_i	32	in	Initial mtvec (trap vector) base	[OK] driven
dm_halt_addr_i	32	in	Address jumped to on debug entry	[OK] tied (debug out of scope)
dm_exception_addr_i	32	in	Address jumped to on exception during debug	[OK] tied
hart_id_i	32	in	Hart ID (readable through mhartid)	[OK] driven/tied, stable
instr_req_o	1	out	Instruction fetch request	[OK] observed + responded
instr_addr_o	32	out	Fetch address (always word-aligned, see 5.3)	[OK] observed
instr_gnt_i	1	in	Fetch grant	[OK] driven
instr_rvalid_i	1	in	Fetch response valid (1 cycle per request)	[OK] driven
instr_rdata_i	32	in	Fetch instruction word	[OK] driven
data_req_o	1	out	Data request	[OK] observed + responded
data_addr_o	32	out	Data address (may be misaligned, see 5.4)	[OK] observed
data_we_o	1	out	1 = store, 0 = load	[OK] observed
data_be_o	4	out	Byte enables (which of the 4 bytes are touched)	[OK] observed
data_wdata_o	32	out	Store data	[OK] observed
data_gnt_i	1	in	Data grant	[OK] driven
data_rvalid_i	1	in	Data response valid (ends reads and writes)	[OK] driven
data_rdata_i	32	in	Load data	[OK] driven
irq_i	32	in	Interrupt inputs	[OK] tied to 0 (interrupts out of scope)
irq_ack_o, irq_id_o	1 / 5	out	Interrupt acknowledge / id	[obs] ignored
debug_req_i	1	in	Debug request	[OK] tied to 0
debug_havereset_o, debug_running_o, debug_halted_o	1	out	Debug status	[obs] ignored (but useful for sanity)

This table is the implementation of ORPLAN **risc_sys_01**: "Port list of **cv32e40p_top** must be connected and observed while ignoring or tying out of scope ports and no 'X' propagation".

3.5 Internal blocks (what is inside)

FACT - rtl/ file list + rtl/cv32e40p_core.sv instantiations:

Module	Role
cv32e40p_top	Top level: core (+ optional FPU). This is our DUT.

Module	Role
cv32e40p_core	The CPU: wires all stages together
cv32e40p_if_stage	Next-PC mux, IF/ID pipeline register, exception PC mux
cv32e40p_prefetch_buffer	Fetch FIFO (depth 2) + OBI adapter
cv32e40p_prefetch_controller	Decides when to issue a fetch transaction; handles branch flush
cv32e40p_aligner	Extracts a 16- or 32-bit instruction from the fetched words (needed for compressed code)
cv32e40p_compressed_decoder	Expands a 16-bit compressed instruction into its 32-bit equivalent
cv32e40p_id_stage	Decoder + register file + forwarding muxes + ID/EX pipeline registers + jump target mux
cv32e40p_decoder	Turns the 32-bit instruction word into control signals; raises illegal_insn_o
cv32e40p_register_file_ff	31x32-bit registers, 3 read ports, 2 write ports
cv32e40p_controller	The big FSM: stalls, flushes, traps, PC selection, forwarding control
cv32e40p_ex_stage	ALU + multiplier (+ APU dispatcher) and the EX write-back mux
cv32e40p_alu	Integer ALU; also hosts the divider (cv32e40p_alu_div)
cv32e40p_mult	Single-cycle MUL; multi-cycle MULH/MULHSU/MULHU FSM
cv32e40p_alu_div	Iterative serial divider (DIV/DIVU/REM/REMU)
cv32e40p_load_store_unit	Byte-enable generation, misaligned splitting, load data alignment/sign extension, outstanding-transaction counter (DEPTH = 2)
cv32e40p_obi_interface	Generic OBI adapter (handles address-phase stability)
cv32e40p_cs_registers	All CSRs (out of scope, but mtvec/mepc matter for traps)
cv32e40p_int_controller	Interrupt controller (out of scope)
cv32e40p_sleep_unit	Clock gating + fetch_enable_i stickiness
cv32e40p_hwloop_regs,	cv32e40p_apu_disp,
cv32e40p_popcnt,	cv32e40p_ff_one,
cv32e40p_sim_clock_gate,	cv32e40p_fifo,
cv32e40p_register_file_latch	clock gate). The clock-gate module is a simulation-only model [FACT] integration.rst.txt.

4. Databook Understanding

The Databook is the design specification. Below, each chapter is summarised **and** linked to the ORPLAN items that exist because of it.

4.1 intro.rst.txt - what the core is, and its history

- 4-stage in-order 32-bit RISC-V core; instruction and data buses are **OBI**.
- Standard compliance: RV32I v2.1 base; extensions C, M, Zicntr, Zicsr, Zifencei (always enabled); F / Zfinx (optional, off by default); custom Xcv / Xcvelw (off by default).
- Privileged: M-mode only, Machine ISA v1.11, trap handling in direct or vectored mode.
- **History section is gold for verification** because it explains why the OBI rules exist:
- "the grant signal can now be kept high by the bus even without the core raising a request" -> ORPLAN **risc_if_01**.
- "the request signal does not depend anymore on the rvalid signal (no combinatorial dependency)" -> ORPLAN **risc_if_01**.
- "the protocol forces the address stability. Thus, the core can not retract memory requests once issued, nor can it change the issued address" -> ORPLAN **risc_if_00**, **risc_lsu_00**.
- [note] **Databook** -> **ORPLAN: risc_if_00/01, risc_lsu_00** exist because of this paragraph.

4.2 integration.rst.txt - instantiation, parameters, ports

- Instantiation template (the exact port list of section 3.4).
- Parameter table (section 3.3).
- Static-configuration rule: **boot_addr_i, mtvec_addr_i, dm_halt_addr_i, dm_exception_addr_i** "Do not change after enabling core via **fetch_enable_i**" -> ORPLAN **risc_sys_03**.
- **fetch_enable_i**: "The first instruction fetch after reset de-assertion will not happen as long as this signal is 0. **fetch_enable_i** needs to be set to 1 for at least one cycle while not in reset to enable fetching. **Once fetching has been enabled the value fetch_enable_i is ignored.**" -> ORPLAN **risc_sys_03** (this is the sticky behaviour we verify).
- **scan_cg_en_i** "should be 0 during normal / functional operation" -> tie-off list of **risc_sys_01**.
- Clock gating cell: **cv32e40p_sim_clock_gate.sv** provides **cv32e40p_clock_gate** (simulation only).
- [note] **Databook** -> **ORPLAN: risc_sys_00, risc_sys_01, risc_sys_03**.

4.3 pipeline.rst.txt - the cycle-count table (the most quoted chapter)

FACT - the "Cycle counts per instruction type" table (zero-wait assumed):

Instruction type	Cycles	Notes
Integer computational	1	
mul	1	single-cycle 32x32 multiplier, 32-bit result
mulh, mulhsu, mulhu	5	"take 5 cycles to compute" "depends on the divider operand value (operand b), i.e. in the number of leading bits at 0. The minimum number of cycles is 3 when the divider has zero leading bits at 0 (e.g. 0x8000000). The maximum number of cycles is 35 when the divider is 0."
div, divu, rem, remu	3..35	1 bus transaction, EX + WB one cycle each
Load/Store	1	2 bus transactions, EX + WB two cycles each
Load/Store (non-word-aligned word, or halfword crossing a word boundary)	2	needs COREV_CLUSTER = 1 -> out of scope
cv.elw	4	Jumps in ID; IF (incl. prefetch buffer) flushed; the new PC request appears on the bus in the same cycle the jump is in ID
Jump	2	no stall
Branch not taken	1	branch decided in EX; IF and ID flushed
Branch taken	3	out of scope
CSR access	4 or 1	out of scope
FENCE.I	2	out of scope
FP	1..N	out of scope

Also stated: * "There is a forwarding path between ALU, Multiplier and Divider result in EX stage and ID stage flip-flops ... allows to have 0-cycle penalty" -> **risc_haz_00**. * "1-cycle penalty on ... Load data hazard ... Jump register (jalr) data hazard" -> **risc_haz_01, risc_haz_02**. * "Multi-cycle instructions will stall this stage until they are complete." -> **risc_m_01, risc_m_02**. * Writes back: "The ALU, Multiplier and Divider instructions write back their result to the register file **from the EX stage**"; **Writeback (WB)**: Writes the result of Load instructions back" -> **risc_haz_03** (two write ports!).

[note] **Databook** -> **ORPLAN**: **risc_alu_00** (1 cycle), **risc_m_00/01/02** (1 / 5 / 3..35), **risc_lsu_02** (1 or 2 bus transactions), **risc_if_05** (jump = 2 cycles), **risc_if_06** (branch = 3 / 1), **risc_haz_00/01/02/03**.

[!] **Documented discrepancy inside the Databook itself. register_file.rst.txt** says "Register file reads are performed in the ID stage. Register file **writes are performed in the WB stage**." That contradicts **pipeline.rst.txt** (ALU/MUL/DIV write from EX) and the RTL (two write ports). The **RTL is authoritative for the implementation**; the register-file chapter is a simplification. ORPLAN follows the pipeline chapter + RTL (**risc_haz_03**). See section 13.

4.4 instruction_fetch.rst.txt - the IF interface

- Signals: **instr_addr_o** (word aligned), **instr_req_o** (stays high until **instr_gnt_i** is high for one cycle), **instr_gnt_i** (address may change the next cycle), **instr_rvalid_i** (high exactly one cycle per request), **instr_rdata_i**.
- Misaligned instruction fetches**: "Externally, the IF interface performs word-aligned instruction fetches only. Misaligned instruction fetches are handled by performing two separate word-aligned instruction fetches. **Internally, the core can deal with both word- and half-word-aligned instruction addresses to support compressed instructions. The LSB of the instruction address is ignored internally.**" -> This is the Databook basis of ORPLAN **risc_if_09** (no instruction-address-misaligned exception can exist).
- Prefetch depth**: "stores the fetched words in a FIFO with a number of entries depending of a local parameter. It is called **DEPTH** and can be found in **cv32e40p_prefetch_buffer.sv** (default value of 2). As a result of this (speculative) prefetch, CV32E40P can fetch up to **DEPTH** words outside of the code region and care should therefore be taken that **no unwanted read side effects occur** for such prefetches outside of the actual code region." -> ORPLAN **risc_if_02** (up to 2 outstanding) and **risc_if_04** ("the instruction memory model must serve any address without side effects and must tolerate reads past the program end").
- Protocol: the IF interface does not implement **we, be, wdata, auser, wuser, aid, rready, err, ruser, rid**; they are "tied off as specified in the OBI specification" -> **risc_lsu_09**-style reasoning for the instruction side too, and **risc_if_01**.
- "OBI specification states that links are always in-order from master point of view" -> **risc_if_02**.

4.5 load_store_unit.rst.txt - the data interface

- Signals: **data_addr_o, data_req_o** (stays high until **data_gnt_i** high one cycle), **data_gnt_i, data_we_o, data_be_o, data_wdata_o** (all "sent together with **data_req_o**"), **data_rvalid_i** ("high for exactly one cycle to signal the end of the response phase **for both read and write** transactions"), **data_rdata_i**.

- Outstanding: "The CV32E40P data interface can cause up to **2 outstanding transactions** and there is no FIFO to allow more outstanding requests." -> **risc_lsu_01**.
- **Misaligned accesses (the single most important paragraph for the LSU)**: "The LSU **never raises address-misaligned exceptions**. For loads and stores where the effective address is not naturally aligned to the referenced datatype ... the load/store is performed as **two bus transactions** in case that the data item crosses a word boundary. A single load/store instruction is therefore performed as two bus transactions for the following scenarios:
 - Load/store of a word for a non-word-aligned address
 - Load/store of a halfword crossing a word address boundary In both cases **the transfer corresponding to the lowest address is performed first**. All other scenarios can be handled with a single bus transaction." -> ORPLAN **risc_lsu_04**. Note the subtlety: a halfword at offset **01** or **10** does **not** cross a word boundary, so it is a **single** transaction; only offset **11** (e.g. **0x...3**) crosses. The ORPLAN captures exactly this (**risc_lsu_04**, **risc_lsu_03**, and the Generation row "lh/sh at offset 3 (crossing) and offsets 1,2 (non-crossing)").
- Protocol: no **auuser**, **wuser**, **aid**, **rready**, **err**, **ruser**, **rid** -> no error path -> **risc_lsu_09**.
- Post-increment load/store only exists when **COREV_PULP = 1** -> out of scope (default parameters).
- PMP: not supported -> **risc_lsu_09** ("the load_err/store_err paths in the RTL are tied inactive in this configuration (no PMP)").

4.6 register_file.rst.txt

- 31 registers **x1..x31**; **x0** is "statically bound to 0 and can only be read, it does not contain any sequential logic".
- "The register file has **three read ports and two write ports**."
- FP register file: only when **FPU = 1** (off).
- [note] **Databook** -> ORPLAN: **risc_rf_00** (x0), **risc_rf_01** (31 registers, 3R/2W).

4.7 exceptions_interrupts.rst.txt

- On entering a handler: **mepc** <- current PC; **mstatus.MIE** saved to **mstatus.MPIE**; all **exceptions** jump to the **base** address in **mtvec** (interrupts may be vectored, but interrupts are out of scope).
- "The core starts fetching at the address defined by **boot_addr_i**."
- Exception causes that exist: **2 = illegal instruction**, **3 = breakpoint (EBREAK)**, **11 = ECALL from M-mode**. (Instruction/load/store access faults exist only with PMP -> not here.)
- "The core raises an illegal instruction exception for any instruction ... explicitly defined as being illegal according to the ISA implemented by the core, **as well as for any instruction that is left undefined** in these specifications unless the instruction encoding is configured as a custom CV32E40P instruction for specific parameter settings". -> With **FPU = 0**, **COREV_PULP = 0**, **COREV_CLUSTER = 0**, all FP / PULP / atomics encodings are illegal, and "The illegal instruction exception ... cannot be disabled and is always active."
- [note] **Databook** -> ORPLAN: **risc_dec_02** (illegal instruction -> trap to the **mtvec** base). It also tells us that **ECALL/EBREAK** exist (cause 3 and 11) - but the guidelines put them out of scope (section 6).

4.8 sleep.rst.txt - clock gating and fetch_enable_i

- The Sleep Unit contains the only clock gate; **all other modules use the gated clock**.
- "Startup behavior: **clk_i** is internally gated off (while signaling **core_sleep_o = 0**) during CV32E40P startup:
- **clk_i** is internally gated off during **rst_ni** assertion
- **clk_i** is internally gated off from **rst_ni** deassertion until **fetch_enable_i = 1** After initial assertion of **fetch_enable_i**, the **fetch_enable_i** signal is **ignored until after a next reset assertion**." -> **This one paragraph is the entire justification** for ORPLAN **risc_sys_02** (no activity while in reset / not enabled) and **risc_sys_03** (sticky fetch enable).
- WFI: only relevant with **COREV_CLUSTER = 0** and no debug -> sleep is out of scope, but **core_sleep_o** is a real output that we tie/observe.
- **cv.elw**: needs **COREV_CLUSTER = 1** -> out of scope (**risc_lsu_02** explicitly says "cv.elw is out of scope").
- **pulp_clock_en_i**: "not used. Tie to 0" when **COREV_CLUSTER = 0** -> tie-off list.

4.9 debug.rst.txt - out of scope, but must be driven inert

Debug is a full featured debug unit (**debug_req_i**, halt addresses, single step, triggers). We must drive **debug_req_i = 0** and tie the **dm * addr_i** inputs so the core never enters debug mode. That is the "minimal work that does not hinder the main task" the guidelines ask for (section 6).

4.10 verification.rst.txt - how OpenHW verified this core

Not a requirement for us, but very informative: * v1.0.0 used **step & compare** against an ISS (instruction set simulator). * v2.0.0 replaced it with a **true reference model** ("ImperasDV") plus a standardised tracer interface called **RVFI / RVVI** (RISC-V Verification Interface). -> **This is why ORPLAN and our architecture talk about an rvfi monitor and an "RVFI Internal Interface"**: RVFI is the industry-standard way to observe retired instructions. Our plan re-uses that idea (a retire-stream monitor), but we must build it ourselves (guidelines: "Build everything yourself"). * It also lists how the real bugs were found (directed tests, step&compare, constrained-random, formal, lint) - a reminder that **both** directed and random stimulus are needed.

4.11 control_status_registers.rst.txt (103 KB) and instruction_set_extensions.rst.txt (212 KB)

These are the reference for every CSR and every custom instruction. For our scope we only need two facts out of them: * **mtvec** holds the trap vector base (driveable through **mtvec_addr_i**), and it must be 256-byte aligned. * Everything else (counters, hardware loops, PULP extensions, FP) is **out of scope** and must not be generated.

5. RTL Understanding

The Databook is the promise; the RTL is the reality. In this section I trace the important features through the actual code, and I flag every place where the two disagree.

5.1 The two OBI adapters - how "request persistence" is actually implemented

FACT - `rtl/cv32e40p_obi_interface.sv` has a parameter **TRANS_STABLE**:

- **TRANS_STABLE = 1 (gen_trans_stable)**: the incoming transaction is already stable -> signals go straight to the pins (`obi_req_o = trans_valid_i, obi_addr_o = trans_addr_i, ...`).
- **TRANS_STABLE = 0 (gen_no_trans_stable)**: an internal FSM (**TRANSPARENT** -> **REGISTERED** -> **TRANSPARENT**) registers the address phase when a request is not immediately granted:
- In **REGISTERED**: "`obi_req_o = 1'b1; // Never retract request`" and the address/control come from the saved registers.
- **trans_ready_o = (state_q == TRANSPARENT)** - i.e. no new transaction is accepted while one is pending.

Who uses which?

Interface	Instantiation	TRANS_STABLE	Consequence
Instruction	<code>cv32e40p_prefetch_buffer.sv</code> : <code>instruction_obi_i</code>	0	Address stability is guaranteed by hardware (the FSM) -> <code>risc_if_00</code> is checkable as an SVA. Stability must come from upstream (the LSU/ex_stage must hold <code>trans_*</code>), because the pins follow <code>trans_*</code> combinatorially.
Data	<code>cv32e40p_load_store_unit.sv</code> : <code>data_obi_i</code>	1	

INFERENCE (why the data side is still OBI-legal): with **TRANS_STABLE = 1**, `data_addr_o = data_addr_int = operand_a_ex_i + operand_b_ex_i` (or `operand_a_ex_i`), i.e. it is a combinational function of the EX-stage operand registers. Those registers only change when the pipeline advances, and the pipeline only advances when the LSU accepts the transaction (`lsu_ready_ex_o` -> `ctrl_update/id_ready_o`). So while `data_req_o = 1` && `data_gnt_i = 0` the EX operands are frozen and the address phase is stable. Evidence: `cv32e40p_load_store_unit.sv` assign `lsu_ready_ex_o = (data_req_ex_i == 1'b0) ? 1'b1 : (cnt_q == 2'b00) ? (trans_valid && trans_ready) : ...` and `cv32e40p_id_stage.sv` assign `id_ready_o = ((~misaligned_stall) & (~jr_stall) & (~load_stall) & ... & ex_ready_i);`

5.2 The instruction side: request generation, DEPTH, and no **rvalid** -> **req** path

FACT - `rtl/cv32e40p_prefetch_controller.sv`:

```
generate
  if (PULP_OBI == 0) begin : gen_no_pulp_obi
    // OBI compatible (avoids combinatorial path from instr_rvalid_i to instr_req_o).
    assign trans_valid_o = req_i && (fifo_cnt_masked + cnt_q < DEPTH);
  end else begin : gen_pulp_obi
    // Legacy PULP OBI behavior, i.e. only issue subsequent transaction if preceding transfer
    // is about to finish (re-introducing timing critical path from instr_rvalid_i to instr_req_o)
    assign trans_valid_o = ... && resp_valid_i;
  end
endgenerate
```

- **PULP_OBI** is a **localparam = 0** in `cv32e40p_core.sv` (line 115), so the OBI-compatible branch is used -> **instr_req_o does not depend on instr_rvalid_i** -> **risc_if_01**. **[FACT]**
- **DEPTH** is passed as the FIFO depth. **FACT** - `cv32e40p_prefetch_buffer.sv`: **localparam FIFO_DEPTH = 2;** -> at most 2 outstanding fetch transactions and at most 2 speculative words beyond the executed code -> **risc_if_02** and **risc_if_04**.
- Back-to-back fetches: **trans_ready_o = (state_q == TRANSPARENT)** in the OBI adapter, so after a grant the FSM returns to **TRANSPARENT** and a new request can be issued **in the very next cycle** -> **risc_if_03**.
- Branch/jump handling: **IDLE** state outputs **aligned_branch_addr = {branch_addr_i[31:2], 2'b00}** when **branch_i**, else the incremented address **{trans_addr_q[31:2], 2'b00} + 4**. While a branch target has not yet been accepted, the FSM waits in **BRANCH_WAIT** and **replays** the address ("Never retract request"). Responses belonging to the wrong path are dropped with the **flush_cnt_q** counter -> **risc_if_05**, **risc_if_06**, **risc_lsu_08**.

FACT - `cv32e40p_prefetch_buffer.sv` assertions (compiled only when **CV32E40P_ASSERT_ON**): * **p_instr_addr_word_aligned**: `instr_addr_o[1:0] == 2'b00` always. * **p_branch_halfword_aligned**: a branch target must have `branch_addr_i[0] == 0`. * **p_no_error**: `instr_err_i` and `instr_err_pmp_i` must be 0 (no bus errors supported).

The first assertion is the hardware proof behind ORPLAN **risc_if_09**: **if every fetch address is word-aligned, an instruction-address-misaligned exception is impossible.**

5.3 How a jump/branch actually redirects the PC - trace

1. **Jump (JAL/JALR)** - decoded in ID. **FACT** `cv32e40p_id_stage.sv`: `jump_target_mux: *JT_JAL: jump_target = pc_id_i + imm_uj_type; *JT_JALR: jump_target = regfile_data_ra_id + imm_i_type`; Note: **no & ~1 here**. The clearing of bit 0 happens later.
2. **FACT** `cv32e40p_if_stage.sv`: the prefetch buffer is instantiated with `.branch_addr_i({branch_addr_n[31:1], 1'b0})` -> **bit 0 of every branch/jump target is forced to 0**. This is the hardware implementation of the RISC-V rule `target = (rs1 + imm) & ~1` -> **risc_haz_02**.
3. **FACT** `cv32e40p_controller.sv`, state **DECODE**: `*jump_in_dec (BRANCH_JALR || BRANCH_JAL) -> pc_mux_o = PC_JUMP`; and `pc_set_o = 1` (unless `jr_stall_o`), i.e. **the jump is resolved in ID**. `*if (branch_taken_ex_i) -> pc_mux_o = PC_BRANCH; pc_set_o = 1; is_decoding_o = 0`; i.e. **the branch is resolved in EX** (`branch_decision_o = alu_cmp_result` in `cv32e40p_ex_stage.sv`).
4. `pc_set_o` makes `cv32e40p_if_stage` raise `branch_req`, which flushes the prefetch FIFO (`fifo_flush_o = branch_i`) and starts fetching at the new address -> **risc_if_05, risc_if_06, risc_haz_04**.

5.4 The data side: byte enables, misaligned splitting, load alignment

All in `rtl/cv32e40p_load_store_unit.sv`. Data type encoding: **data_type** `2'b00 = word`, `2'b01 = halfword`, `2'b10/2'b11 = byte`.

(a) **Byte enables (data_be_o)** - **FACT**, the **always_comb** "BE generation" block:

Type	addr[1:0]	beat 1 (misaligned_st = 0)	beat 2 (misaligned_st = 1)
Word	00	1111	(0000, unused)
Word	01	1110	0001
Word	10	1100	0011
Word	11	1000	0111
Half	00	0011	-
Half	01	0110	-
Half	10	1100	-
Half	11	1000	0001
Byte	00/01/10/11	0001/0010/0100/1000	-

This table is **ORPLAN risc_lsu_03**, copied out of the RTL. The plan's rule "Only these patterns may ever appear on **data_be_o**" is therefore directly checkable.

(b) **Misaligned detection** - **FACT**:

```
if ((data_req_ex_i == 1'b1) && (data_misaligned_ex_i == 1'b0)) begin
  case (data_type_ex_i)
    2'b00: if (data_addr_int[1:0] != 2'b00) data_misaligned_o = 1'b1; // word
    2'b01: if (data_addr_int[1:0] == 2'b11) data_misaligned_o = 1'b1; // halfword crossing
  endcase
end
```

-> exactly the Databook rule: **word at any non-zero offset, and halfword at offset 3**. Note **data_misaligned_o** is 0 during the second beat (because **data_misaligned_ex_i** is then 1) - the flag is a "needs a second beat" pulse that the controller turns into a stall.

(c) **The two beats** - **FACT + INFERENCE**:

```
// For last phase of misaligned transfer the address needs to be word aligned
assign trans_addr = data_misaligned_ex_i ? {data_addr_int[31:2], 2'b00} : data_addr_int;
```

and, in `cv32e40p_id_stage.sv` (ID/EX pipeline update, **else if (data_misaligned_i)** branch):

```
alu_operand_b_ex_o <= 32'h4; // add 4 to the address
prepost_useincr_ex_o <= 1'b1; // use a+b for the address
regfile_alu_we_ex_o <= 1'b0; // no extra write-back on the second beat
data_misaligned_ex_o <= 1'b1; // remember: we are now in the second beat
```

INFERENCE - put the two together. Example: **SW** to address **0x1001** (offset 1): *** beat 1: data_addr_int = 0x1001, data_misaligned_ex_i = 0 -> trans_addr = 0x1001, be = 1110** (bytes 1,2,3 of the word at **0x1000**). *** The controller stalls (misaligned_stall_o = data_misaligned_i)** and the controller's forwarding mux forces **operand a fw_mux_sel_o = SEL_FW_EX** so the EX address is kept. *** beat 2: operand_b becomes 4 -> data_addr_int = 0x1005 -> trans_addr = {0x1005[31:2], 2'b00} = 0x1004, be = 0001** (byte 0 of the word at **0x1004**).

So the sequence on the bus is: **lowest address first (0x1001, be 1110)**, then **the next word (0x1004, be 0001)**, which is exactly **ORPLAN risc_lsu_04** ("the transfer for the lowest address is issued first, followed by (aligned base+4) with the complementary byte-enables"). Note the subtlety this exposes: **data_addr_o is NOT word-aligned on the first beat** (only **instr_addr_o** is always word-aligned). The ORPLAN's **+4** wording means word-aligned base + 4.

(d) **Store data rotation** - **FACT**:

```
assign wdata_offset = data_addr_int[1:0] - data_reg_offset_ex_i[1:0];
case (wdata_offset)
```

```

2'b00: data_wdata = data_wdata_ex_i[31:0];
2'b01: data_wdata = {data_wdata_ex_i[23:0], data_wdata_ex_i[31:24]};
2'b10: data_wdata = {data_wdata_ex_i[15:0], data_wdata_ex_i[31:16]};
2'b11: data_wdata = {data_wdata_ex_i[7:0], data_wdata_ex_i[31:8]};
endcase

```

-> ORPLAN **risc_lsu_05** (the memory only applies the **data_be_o** lanes). With the default (non-PULP) configuration **data_reg_offset_ex_i** is 0 (no post-increment), so **wdata_offset == addr[1:0]**.

(e) Load data assembly and sign extension - FACT: * **rdata_q** keeps the **first** response when a misaligned access is in flight, and the final value is assembled as, e.g. for offset 1: **rdata_w_ext = {resp_rdata[7:0], rdata_q[31:8]}** (the low byte of the second response goes to the top; the upper 3 bytes of the first response go to the bottom). * **rdata_h_ext / rdata_b_ext** implement the sign/zero extension: **data_sign_ext_q == 2'b00** -> zero-extend (**16'h0000 / 24'h000000**), **2'b10** -> a strange **16'hffff/24'hffffff** case (that is a PULP vector-mode encoding), otherwise -> replicate the sign bit. The decoder sets **data_sign_extension_o = {1'b0, ~instr_rdata_i[14]}** - bit 14 of the instruction is 0 for **LB/LH/LW** and 1 for **LBU/LHU** -> sign-extend vs zero-extend. **[FACT]** (**cv32e40p_decoder.sv, OPCODE_LOAD**). * **data_rdata_ex_o = (resp_valid == 1'b1) ? data_rdata_ext : rdata_q;**

(f) Outstanding counter and back-pressure - FACT: **localparam DEPTH = 2;**, **cnt_q** counts up on **trans_valid** && **trans_ready** and down on **resp_valid**; **trans_valid = data_req_ex_i** && (**cnt_q < DEPTH**) (OBI branch). So the core can have **2 outstanding data transactions**, and when the memory is slow the LSU de-asserts **lsu_ready_ex_o**, which stalls EX/WB instead of issuing more requests -> **risc_lsu_01**.

(g) Errors - FACT: **cv32e40p_core.sv** instantiates the LSU with **.data_err_i(1'b0)** and **.data_err_pmp_i(data_err_pmp)** (with **USE_PMP = 0** localparam), and the LSU has an assertion **a_no_error** requiring **data_err_i == 0** && **data_err_pmp_i == 0**. -> **risc_lsu_09**.

5.5 The register file: x0, ports, and WAW priority

FACT - rtl/cv32e40p_register_file_ff.sv:

```

// R0 is nil
always_ff @(posedge clk or negedge rst_n) begin
    if (~rst_n) mem[0] <= 32'b0; else mem[0] <= 32'b0;
end
for (i = 1; i < NUM_WORDS; i++) begin
    always_ff @(posedge clk, negedge rst_n) begin
        if (rst_n == 1'b0) mem[i] <= 32'b0;
        else begin
            if (we_b_dec[i] == 1'b1) mem[i] <= wdata_b_i; // port B checked FIRST
            else if (we_a_dec[i] == 1'b1) mem[i] <= wdata_a_i;
        end
    end
end
end

```

- **mem[0]** is permanently 0 -> **risc_rf_00**.
- All registers reset to 0 -> **risc_rf_01** (the ORPLAN flags this as an implementation property, not an ISA requirement - correct: RISC-V does not require GPRs to reset to 0).
- **Port B wins** on a same-cycle collision. **FACT - cv32e40p_id_stage.sv** connects
- port **A** (**we_a_i**) <- **regfile_we_wb_power_i / regfile_waddr_wb_i / regfile_wdata_wb_i** -> the **WB** write (load data),
- port **B** (**we_b_i**) <- **regfile_alu_we_fw_power_i / regfile_alu_waddr_fw_i / regfile_alu_wdata_fw_i** -> the **EX** write (ALU/MUL/DIV/CSR). -> the architecturally **younger** instruction (the one in EX) wins. This is exactly ORPLAN **risc_haz_03**.

5.6 Hazards: forwarding, load-use, JALR

FACT - cv32e40p_controller.sv:

```

// Forwarding control unit
if (regfile_we_wb_i == 1'b1) begin // WB -> ID forwarding
    if (reg_d_wb_is_reg_a_i) operand_a_fw_mux_sel_o = SEL_FW_WB; ...
end
if (regfile_alu_we_fw_i == 1'b1) begin // EX -> ID forwarding
    if (reg_d_alu_is_reg_a_i) operand_a_fw_mux_sel_o = SEL_FW_EX; ...
end

```

- The RAW detection signals come from **cv32e40p_id_stage.sv**: **reg_d_ex_is_reg_a_id = (regfile_waddr_ex_o == regfile_addr_ra_id) && reg_a_used_dec && (regfile_addr_ra_id != '0)**; Note the **!= '0** term: **x0 is never forwarded** (it is always 0) -> supports **risc_rf_00** and **risc_dec_03**.
- Selection: **SEL_FW_EX** uses **regfile_alu_wdata_fw_i** (the EX result) -> **0-cycle penalty** -> **risc_haz_00**.
- **Load-use stall:** **systemverilog if ((data_req_ex_i && regfile_we_ex_i) || (!wb_ready_i && regfile_we_wb_i)) && (reg_d_ex_is_reg_a_i || reg_d_ex_is_reg_b_i || reg_d_ex_is_reg_c_i || ...)) begin deassert_we_o = 1'b1; load_stall_o = 1'b1; end** **load_stall_o** feeds **id_ready_o**, so ID (and therefore IF) is frozen for exactly one cycle; **deassert_we_o** prevents a spurious write during that cycle -> **risc_haz_01**.
- **JALR stall:** **systemverilog if ((ctrl_transfer_insn_in_dec_i == BRANCH_JALR) && ((regfile_we_wb_i && reg_d_wb_is_reg_a_i) || ((regfile_we_ex_i && reg_d_ex_is_reg_a_i) || ...))) begin jr_stall_o = 1'b1; deassert_we_o = 1'b1; end** -> 1-cycle stall when JALR's **rs1** is produced by the immediately preceding instruction -> **risc_haz_02**. (The target itself is still **(rs1+imm) & ~1** because of the IF-stage bit-0 clearing, section 5.3.)

5.7 The multiplier and the divider

MUL - FACT: single-cycle; **cv32e40p_mult.sv** computes **int_result = op c + op b msu + op a msu * op b** (a 32x32->32 multiply). The decoder sends **MUL_MAC32** with **regc_mux = REGC_ZERO -> risc_m_00** (1 cycle).

MULH / MULHSU / MULHU - FACT: **cv32e40p_mult.sv** contains the FSM **IDLE MULT -> STEP0 -> STEP1 -> STEP2 -> FINISH -> IDLE MULT**. **multicycle_0** (which stalls EX) is 1 in STEP0/STEP1/STEP2; **mulh_ready** is 1 in **FINISH**. Sign handling per instruction **[FACT]** (**cv32e40p_decoder.sv**): **mulh -> mult_signed_mode = 2'b11** (signed x signed), **mulhsu -> 2'b01**, **mulhu -> 2'b00**. **INFERENCE:** 1 cycle to enter + STEP0/1/2 + FINISH = **5 cycles**, matching the Databook's "5 (mulh, mulhsu, mulhu)" -> **risc_m_01**.

DIV / DIVU / REM / REMU - FACT: * The decoder swaps the operands for the divide instructions: **alu_op_a_mux_sel o = OP_A REGB OR FWD; alu_op_b_mux_sel o = OP_B REGA OR FWD;** -> in the ALU, **operand_a = rs2** (the divisor) and **operand_b = rs1** (the dividend). * **cv32e40p_alu.sv** then instantiates the divider with a further swap - the code even says // **inputs A and B are swapped: .OpA_DI(operand_b i)** (= dividend) and **.OpB_DI(shift left result)** (= the divisor, pre-shifted). * The number of iterations: **Cnt_DN = (LoadEn) ? OpBShift_DI : ...** with **OpBShift_DI = div_shift**, and **div_shift = div_shift_int + (div_op_a_signed ? 0 : 1)** where **div_shift_int = ff_no_one ? 6'd31 : clb_result** and **clb_result = ff1_result - 5'd1** computed in 6 bits from the 5-bit **ff1_result**. The leading-bit count is taken on the divisor: for **DIVU/REMU** on **operand_a_rev**, for **DIV/REM** on **operand_a_neg_rev** when the divisor is negative (**cv32e40p_ff_one** finds the lowest set bit). * **cv32e40p_alu_div.sv** FSM: **IDLE** (load) -> **DIVIDE** (one cycle per count value) -> **FINISH** (result valid).

INFERENCE (cycle count, matches the Databook exactly): * divisor **0x8000_0000** (no leading zeros): **ff1_result = 0 -> clb_result = 0-1 = 63** (6-bit) -> **div_shift = 63+1 = 64 -> 0** in 6 bits -> **Cnt = 0 -> 1 DIVIDE cycle -> total 1 (IDLE) + 1 (DIVIDE) + 1 (FINISH) = 3 cycles**. [ok] Databook: "minimum is 3 when the divider has zero leading bits at 0 (e.g., 0x8000000)". * divisor = 0: **ff_no_one = 1 -> div_shift_int = 31 -> div_shift = 32 -> 33 DIVIDE cycles -> total 35 cycles**. [ok] Databook: "maximum is 35 when the divider is 0". * general case: **latency = 3 + leading_zeros(divisor)**.

[!] Because this depends on 6-bit modular arithmetic, it is a classic place where a bug could hide; the ORPLAN therefore asks for **both** a golden-result check and a **latency range check (3..35) -> risc_m_02, div_latency_cg**.

5.8 Illegal instruction detection

FACT - cv32e40p_decoder.sv sets **illegal_insn_o = 1'b1** in (among many others):

Category	Example in RTL	ORPLAN category
Undefined major opcode	the default: of the big unique case (a) (instr_rdata_i[6:0]) OPCODE_BRANCH:	
Legal opcode, undefined funct3	3'b000/001/100/101/110/111 are the 6 branches, default: (b) illegal_insn_o = 1'b1; OPCODE_OP, prefix 00/01: unique case ({instr[30:25], instr[14:12]})	
Legal opcode, undefined funct7	with an explicit list (b) (ADD/SUB/SLT/SLTU/XOR/OR/AND/ SLL/SRL/SRA + the 8 M ops) and default: illegal_insn_o = 1'b1; OPCODE_OPIMM: 3'b001 (SLLI) requires instr[31:25] == 7'b0;	
Reserved shift encodings	3'b101 requires 7'b0 (SRLI) or (c) 7'b010_0000 (SRAI); otherwise illegal	
Disabled extensions	OPCODE_AMO -> illegal (A_EXTENSION = 0); OPCODE_OP_FP/OPCODE_OP_FP_M ADD/... -> illegal when FPU == 0; OPCODE_SYSTEM - (CSR/ECALL/EBREAK/MRET) is decoded but ECALL/EBREAK are the separate exceptions we do not test	

What happens to an illegal instruction - **FACT** (**cv32e40p_controller.sv**): 1. In **DECODE**, if (**illegal_insn_i**): **halt if o = 1; ctrl_fsm_ns = id_ready_i ? FLUSH_EX : DECODE; illegal_insn_n = 1;** and (in the combinational default block) if (**illegal_insn_i**) deassert we **o = 1'b1;** -> no register write. 2. In **FLUSH_EX** (when **ex_valid_i**): **csr_save_id_o = 1; csr_cause_o = {1'b0, EXC_CAUSE_ILLEGAL_INSN};** 3. In **FLUSH_WB**: **pc_mux_o = PC_EXCEPTION;** **pc_set_o = 1; exc_pc_mux_o = EXC_PC_EXCEPTION;** 4. In **cv32e40p_if_stage.sv**: **EXC_PC_EXCEPTION: exc_pc = {trap_base_addr, 8'h00};** where **trap_base_addr** comes from **m_trap_base_addr_i** = the **mtvec** CSR (initialised from the **mtvec_addr_i** port). 5. **cv32e40p_cs_registers.sv**: **mtvec_n = csr_mtvec_init_i ? mtvec_addr_i[31:8] : mtvec_q;** and **csr_mtvec_init_o = (pc_mux_i == PC_BOOT) & pc_set_i;** -> **mtvec** is loaded from the **mtvec_addr_i** port at boot.

-> So the observable effect of an illegal instruction is: **the next fetch address becomes {mtvec_addr_i[31:8], 8'h00}, mepc** receives the illegal instruction's PC, and nothing else changes. That is precisely what **ORPLAN risc_dec_02** checks (**sb_illegal_trap, exc_trap_base_prop**) - and note it is all observable **without reading any CSR**, which is why it fits the "no CSR" scope.

5.9 Reset, **fetch_enable_i** and clock gating

FACT - cv32e40p_sleep_unit.sv (with **COREV_CLUSTER = 0**, block **g_no_pulp_sleep**):

```
assign fetch_enable_d = fetch_enable_i ? 1'b1 : fetch_enable_q; // sticky
assign core_busy_d   = if_busy_i || ctrl_busy_i || lsu_busy_i || apu_busy_i;
assign clock_en      = fetch_enable_q && (wake_from_sleep_i || core_busy_q);
assign core_sleep_o   = fetch_enable_q && !clock_en;
```

- while **rst_ni == 0** -> all three FFs reset to 0 -> **clock_en = 0** -> **the internal clock is gated** -> **risc_sys_02**.
- after reset, until **fetch_enable_i** is sampled 1 -> **fetch_enable_q = 0** -> **clock_en = 0** -> **still gated** -> **risc_sys_03**.
- once 1, **fetch_enable_q** stays 1 until the next reset (sticky) -> "the value of **fetch_enable_i** is ignored afterwards".

FACT - cv32e40p_controller.sv, state **RESET: instr_req_o = 1'b0; is_decoding_o = 1'b0; if (fetch_enable_i == 1'b1) ctrl_fsm_ns = BOOT_SET**; -> **no fetch request can be asserted before fetch_enable_i (sys_no_fetch_before_fe_prop)**. Then **BOOT_SET** sets **pc_mux_o = PC_BOOT** and **pc_set_o = 1** -> the first fetch address is **boot_addr_i** (**FACT cv32e40p_if_stage.sv: PC_BOOT: branch_addr_n = {boot_addr_i[31:2], 2'b00}**);).

5.10 What RTL tells us that the Databook does not

Topic	Databook	RTL (extra detail)	Why it matters for verification
Address stability	"the protocol forces the address stability"	Two different mechanisms: a hardware FSM on IF (TRANS_STABLE=0) and upstream freezing on LSU (TRANS_STABLE=1)	The LSU stability check depends on the stimulus actually delaying gnt - a check that passes trivially in zero-wait mode.
ILLEGAL -> trap	generic	exact path: deassert_we -> FLUSH_EX -> FLUSH_WB -> PC_EXCEPTION with exc_pc = {mtvec[23:0], 8'h00}	The TB can predict the trap PC from mtvec_addr_i alone, no CSR access needed.
Register writes	register-file chapter says "WB"	two write ports; EX (port B) and WB (port A); B wins beat 1 at the unaligned address; beat 2 at aligned(addr)+4 because alu_operand_b is forced to 4	risc_haz_03 WAW check.
Misaligned second beat	"lowest address first ... two bus transactions"	div_shift computed with 6-bit modular arithmetic from cv32e40p_ff_one	Needed to write the scoreboard's expected bus traffic.
Divider latency	"3..35 depending on leading zeros of operand b"	mtvec_q	Corner case (divisor = 0x80000000) deserves a directed test.
mtvec	"initial value for the address part of mtvec"	mtvec_addr_i[31:8] when csr_mtvec_init_i (at PC_BOOT)	The trap base is under TB control -> risc_dec_02 is checkable.
Debug / interrupts	full chapters	must be driven inert (debug_req_i = 0, irq_i = 0)	risc_sys_01 tie-offs.

6. Verification Guidelines - the scope of our work

The guidelines are the Q&A spreadsheets in **guidelines/Questions QRE Final project status 2026/Team 1...Team 6.html**. They are the only place where the supervisor defines what is in and out of scope, so **every scope decision in this document is traced to one of these answers**.

6.1 The rules, quoted verbatim

#	Rule (verbatim from the guidelines)	Source
G1	"Main task: the I & M extensions only for now. Other parts of the design (interfaces & extensions), You are required to do the minimal work that does not hinder the main task. If, for example, the databook says that the debug interface should be driven to specific values for the controller to work normally. You should implement the drivers that ensure the normal operation (Do not say that we are focusing on the I & M)"	Team 1, Q1
G2	(About writing the plan) "Both (1), (2) are acceptable ... A mix between the two cases where you mention the part from the spec followed by a one line explaining potentially ambiguous statements. Do not be verbose in any case"	Team 1, Q2
G3	"The goal is a verification of the core only (No memory). Anything other than the core RTL is considered part of the TB. Engineers are required to implement the whole TB (Including the memory if they see it as a requirement)"	Team 1, Q3
G4	"You are implementing the drivers. knowledge of how the OBI works is required to properly implement the drivers. You need to know enough to be able to drive and implement the required interface checkers"	Team 1, Q4
G5	"Focus is top level verification. With scoreboard in the datapath to help you debug (not one scoreboard per module, but one scoreboard per major group of blocks)"	Team 2, Q2 (and Q1: aligner & compressed decoder "are part of the compressed instructions (C extension). Which is not part of the verification goal. The major focus is the core as a whole with scoreboards that verify the operation of major blocks such as the registerfile and ALU.")
G6	"simple exceptions such as misaligned memory operations is good to verify. ecall and ebreak are not required" / "Ecalls ebreak and hints are not required"	Team 2, Q3; Team 4, Q1
G7	"Build everything yourself" (no open-source reference model)	Team 2, Q5
G8	"No need for coverpoints that are out of the scope"	Team 2, Q6
G9	"do we concern with the CSR? -> No" / "do we concern with the interrupts ...? -> No"	Team 3, Q1, Q2
G10	"Should we include these custom instructions in our verification scope ...? -> Only vanilla I & M"	Team 4, Q2; Team 5, Q1 ("Only Vanilla I & M")
G11	"Design specification is the databook. Only submit verification plan (excel sheet and powerpoint)"	Team 5, Q2
G12	"The aim for delaying the RTL is that most verification projects start before the RTL is finalised. We want to hone your abilities to work with abstract specifications without access to RTL"	Team 5, Q3

#	Rule (verbatim from the guidelines)	Source
G13	(about the data memory) "Please allow yourself to analyse and see what best fits your verification need. The aim of the project is to give some control of the decision making"	Team 5, Q4

Unanswered questions (rows with an empty "Answers" cell) - these are real ambiguities: * Team 2 Q9: "For a load or store with a misaligned effective address, does the CV32E40P's LSU raise an address-misaligned exception itself, or does it handle the misalignment some other way (e.g., splitting the access into multiple bus transactions)?" * Team 2 Q10: "For any instruction encoding that isn't implemented ... does the core raise an illegal-instruction exception? And does that same behavior apply to the reserved shift encodings ...?" * Team 4 Q4: what happens to a multi-cycle instruction (MULH/DIV) when a taken branch flushes the pipeline. * Team 4 Q5: driver-answers-each-fetch vs pre-generated program. * Team 4 Q6: register-file write-port priority on a WAW collision.

-> All five are answered in this document **from the Databook + RTL** (sections 5.4, 5.8, 5.7, 5.5) and are repeated in section 13 as open questions for the supervisor.

Two further ambiguities are raised **inside ORPLAN itself** (they are notes in the plan, not answers): * **risc_sys_01** carries the note: "**what about boot address and mtvec address?**" * **risc_env_00** requires deterministic memory pre-load - but the plan does not say how the data memory image is initialised relative to the golden model.

6.2 Derived scope classification

[OK] MUST VERIFY (explicitly required by a guideline)

Feature	Guideline
All RV32I base instructions executing correctly	G1, G10 ("vanilla I & M")
All RV32M instructions	G1, G10
(MUL/MULH/MULHSU/MULHU/DIV/DIVU/REM/REMU)	
The interfaces needed to make the core run: OBI instruction fetch + OBI data (drivers, monitors, protocol checkers)	G1 ("minimal work ... ensure the normal operation"), G4 ("knowledge of OBI is required ... required interface checkers")
Reset behaviour, fetch_enable_i behaviour, boot address	G1 (minimal work to make the core run)
Simple exceptions: misaligned data accesses (correct splitting, no exception)	G6 ("simple exceptions such as misaligned memory operations is good to verify")
Top-level (black-box) checking with a scoreboard on the datapath	G5

[~] VERIFY (clearly inside the required scope)

Feature	Justification
Illegal-instruction detection inside the RV32I/M opcode space	It is a decode behaviour of RV32I, not a system instruction. G6 only excludes ECALL/EBREAK/hints. The Databook makes illegal-instruction detection always active (section 4.7). (Flagged as NEEDS CLARIFICATION only because guideline Team 2 Q10 was left unanswered.)
Pipeline hazards (RAW forwarding, load-use, JALR, WAW, flush cleanliness)	They change the visible behaviour of vanilla I&M programs (results and timing). G5 explicitly mentions "scoreboard in the datapath".
Multi-cycle timing (MULH 5 cycles, DIV 3..35)	It is the documented behaviour of the M extension (G1).
Register file behaviour (x0, 31 registers)	G5 names "the registerfile" explicitly as a major block worth a scoreboard.
Memory consistency (store->load read-back)	Needed to make the golden comparison meaningful for the I&M loads/stores.
OBI timing stress (gnt/rvalid delays, outstanding depth, gnt-before-req)	G4 requires interface drivers/checkers; the Databook requires in-order responses and 2 outstanding.

[X] OUT OF SCOPE / NOT REQUIRED (explicitly excluded)

Feature	Guideline
CSRs and CSR instructions	G9 ("do we concern with the CSR? -> No")
Interrupts	G9 ("do we concern with the interrupts? -> No")
ECALL, EBREAK, hints, memory-ordering (FENCE)	G6 ("ecall and ebreak are not required", "hints are not required")
C extension (compressed instructions), the aligner and the compressed decoder as features	Team 2 Q1: "aligner and compress decoder are part of the compressed instructions (C extension). Which is not part of the verification goal"
FPU / Zfinx and FP CSRs	G1, G10
CORE-V custom (PULP/CLUSTER) instructions and CSRs, cv.elw	G10 ("Only vanilla I & M")
Debug (as a feature)	G1 - but the interface must be driven inert so the core works
Sleep / WFI (as a feature)	G1 - but fetch_enable_i startup gating is in scope because without it the core never starts
Atomics (A), PMP, U-mode, performance counters, hardware loops	G1, G10
Block-level (per-module) testbenches and one-scoreboard-per-module	G5
Using a third-party/open-source reference model	G7 ("Build everything yourself")
Coverage points for out-of-scope features	G8
BRV (verification of the memories themselves)	G3 - memory is part of the TB, not the DUT

[?] NEEDS CLARIFICATION

Topic	Why
Illegal-instruction testing (reserved shift encodings etc.)	Team 2 Q10 was never answered; ORPLAN includes it.
Whether the trap behaviour (mepc/mcause CSR values) should be checked	CSR = out of scope (G9); ORPLAN only checks the fetch redirect to the mtvec base, which is the right compromise.
The exact values for boot_addr_i / mtvec_addr_i	ORPLAN note: "what about boot address and mtvec address?"
Multi-cycle instruction + flush interaction	Team 4 Q4 unanswered; answered here from the RTL (section 5.7 / 8.4.9).
Register-file write-port priority	Team 4 Q6 unanswered; answered here from the RTL (section 5.5).
How retire information (RVFI) is obtained	The RTL ships no RVFI module; the ORPLAN's rvfi_monitor must be built by us (see 10.5).

7. Verification Architecture Fundamentals

This section explains **every component that appears in our architecture diagram**, in the order the data flows. If a word is unfamiliar, its first occurrence here is the definition.

7.1 top_tb - the static top of the simulation

What it is: the SystemVerilog module that is the root of the simulation. It contains: * the **DUT** instance (**cv32e40p_top**), * the **interfaces** (bundles of wires) connected to the DUT pins, * the clock and reset generators, * the call that starts UVM (**run_test()**), * the configuration of virtual interfaces into the UVM configuration database.

Why it exists: UVM objects cannot touch wires directly. **top_tb** is the bridge between the static hardware world (module instances and wires) and the dynamic object world (UVM classes).

ORPLAN link: **risc_sys_00** ("**top_tb** instantiates **cv32e40p_top.sv** and verifies the core as a whole at top level") and **risc_env_02** (teardown + watchdog).

7.2 Interface / virtual interface

Interface: a SystemVerilog construct that groups related signals together (e.g. all the OBI instruction signals) and can contain **clocking blocks** (which define when signals are sampled/driven relative to the clock) and **modports** (which define direction per user).

Virtual interface: a handle (like a pointer) to an interface instance. UVM components store virtual interfaces so they can read/drive the DUT pins without being modules.

In our project there are four (matching the four interfaces of the diagram): 1. **IF Virtual Interface - instr_req_o**,

instr_addr_o, **instr_gnt_i**, **instr_rvalid_i**, **instr_rdata_i**. 2. **LSU Virtual Interface** - **data_req_o**, **data_addr_o**, **data_we_o**, **data_be_o**, **data_wdata_o**, **data_gnt_i**, **data_rvalid_i**, **data_rdata_i**. 3. **System Control Interface** - **clk_i**, **rst_ni**, **fetch_enable_i**, **boot_addr_i**, **mtvec_addr_i**, **hart_id_i**, **dm_halt_addr_i**, **dm_exception_addr_i**, plus the **out-of-scope tie-offs** (**irq_i**, **debug_req_i**, **pulp_clock_en_i**, **scan_cg_en_i**) and the observed status outputs (**core_sleep_o**, **debug*_o**). 4. **RVFI Internal Interface** - the retirement view of the core (see 7.7).

ORPLAN link: **risc_sys_01** ("Virtual interfaces wrap the pins"), **risc_env_01**.

7.3 Test, environment, configuration

- **Test (uvm_test)**: the top-level object. It builds the environment, configures it, and starts the stimulus. One test = one simulation with a clear goal.
- **Environment (uvm_env)**, drawn as the purple container **env**: holds everything that is reusable across tests: the agents, the reference model, the scoreboard and the coverage.
- **env_cfg (Configuration object)**: a small object with the knobs - e.g. "IF memory is zero-wait", "LSU uses random gnt delays 0..7", "number of instructions", "seed". Tests set the knobs; components read them. This is how one environment serves 27 different tests.

ORPLAN link: **risc_env_01** (gnt/rvalid delay modes), **risc_env_00** (memory pre-load mode).

7.4 Sequences, sequencers, drivers - how stimulus reaches the DUT

- **Transaction (uvm_sequence_item)**: a class describing one unit of activity in high-level terms. In this project there are two very different kinds:
- a **bus-response transaction** (used by **obi_memory_sequence**): "answer this request after N cycles of **gnt** delay and M cycles of **rvalid** delay, with this data".
- an **instruction transaction** (used by **riscv*_sequence**): "here is a 32-bit instruction word to be placed in the program".
- **Sequence**: generates a stream of transactions (randomised within constraints, or directed).
- **Sequencer**: delivers transactions to the running sequence to the driver, handling arbitration.
- **Driver**: converts a transaction into **pin-level activity** on the virtual interface.

In this project the direction is unusual and very important to understand: the DUT is the **bus master** on both OBI ports (it issues ***req_o**), and **our testbench is the slave** (it answers with ***gnt_i** / ***rvalid_i** / ***rdata_i**). Therefore: * the IF/LSU **drivers do not create requests** - they respond to the requests the core makes, and the timing of those responses is what the sequences randomise; * the content of the responses (the instruction words, the loaded data) comes from the **memory models** (7.5); * the **program itself** is generated by the **program builder** (7.6) and pre-loaded into the instruction memory before the core is released from reset.

ORPLAN link: **risc_if_00..03**, **risc_lsu_00..01**, **risc_env_01**, and the whole **Generation_** sheet.

7.5 Memory models (instr_mem_model, data_mem_model)

What they are: testbench models of memories that speak OBI as a **slave**. * **instr_mem_model** holds the **program** (a sparse array of 32-bit words + a fill/illegal-word policy). * **data_mem_model** holds the **data image** (pre-loaded deterministically so that a load of a never-written location is reproducible).

Why the TB needs them (and not a "real" memory): guideline G3 - "The goal is a verification of the core only (No memory). Anything other than the core RTL is considered part of the TB. Engineers are required to implement the whole TB (**Including the memory if they see it as a requirement**)."

What they must guarantee (from the Databook + ORPLAN): * serve **any** address, including the up-to-2 speculative words fetched past the end of the program, **without side effects** (**risc_if_04**); * never return an error (the OBI ports have no **err** signal) (**risc_lsu_09**); * apply **only** the byte lanes selected by **data_be_o** (**risc_lsu_05**); * commit a store when its **rvalid** is returned, and **never reorder** (**risc_lsu_07**); * clear their outstanding queues on reset (**risc_rst_00/01**).

7.6 Program builder (riscv_program)

What it is: the generator that assembles the instruction stream: it picks instructions, registers and immediates under constraints, resolves branch/jump targets, guarantees that the program terminates at a known **end-of-program sentinel address**, and produces the 32-bit words that are loaded into **instr_mem_model**.

Why it is needed: guideline G5/G10 - we only generate **vanilla RV32I + RV32M** 32-bit instructions. The builder is where that constraint is enforced.

Constraints it must enforce (ORPLAN risc_if_08): * only 32-bit encodings (no RVC/compressed words); * word-aligned branch/jump targets, inside the program window; * bounded programs that end at the sentinel; * no self-modifying code (that would need **FENCE.I**, which is omitted).

7.7 Monitors - how we observe the DUT

Monitor: a passive component that watches an interface and converts pin activity back into transactions; it then **publishes** them through an **analysis port** (green diamond in the diagram).

Four monitors exist in our architecture:

Monitor	Watches	What it publishes	Used by
System Control Agent - Input Monitor	rst_ni, fetch_enable_i, boot_addr_i, mtvec_addr_i, hart_id_i, core_sleep_o	reset/enable events and their cycle context	Reference model (to know when to re-sync), Coverage (reset_context_cg, fetch_enable_delay_cg)
IF Agent - Input Monitor	the request side of the instruction bus (instr_req_o, instr_addr_o)	the fetch-address stream	Reference model (this is the instruction stream the model executes)
LSU Agent - Input Monitor	the request side of the data bus (data_req_o, data_addr_o, data_we_o, data_be_o, data_wdata_o)	the load/store request stream	Reference model (to update its own memory image)
IF Agent (Passive) Output Monitor	the response side of the instruction bus (instr_gnt_i, instr_rvalid_i, instr_rdata_i) plus protocol rules	fetch transactions (address + word + timing)	Scoreboard, Coverage, SVA (if_obi_slave_protocol_prop, if_obi_trans_match_prop)
LSU Agent (Passive) Output Monitor	the response side of the data bus	data transactions (address, we, be, wdata, rdata, timing)	Scoreboard, Coverage, SVA (lsu_be_legal_prop, lsu_misaligned_order_prop, lsu_max_outstanding_prop)
RVFI Monitor - Internal State Monitor	internal/core-level retirement information (the "RVFI Internal Interface")	the retired instruction stream: PC, instruction word, rd, rd value, memory access, order, timing	Scoreboard, Coverage

Important teaching point - why a retire (RVFI) monitor? Monitoring the buses tells you what was fetched and what data moved, but not what the core actually committed: an instruction fetched after a taken branch is thrown away, and an instruction can be fetched twice. The **retirement** view (in RISC-V verification usually called **RVFI**, "RISC-V Formal Interface", or **RVVI**) is the architectural truth: one entry per instruction that really completed. The Databook confirms this approach was used officially: "the step-and-compare and the ISS have been replaced by a true reference model (RM) ... the adoption of a standardized tracer interface to the DUT and RM, based on the open-source RISC-V Verification Interface (RVVI)" (**verification.rst.txt**). [!] The RTL delivered in this repository contains **no** RVFI module (checked: no **rvfi** string in **rtl/**), so our **rvfi_monitor** must be built by us - see section 10.5 and 13.

7.8 Reference model (ref_model)

What it is: an independent, simple implementation of "what a correct RV32IM core does" - effectively a tiny instruction-set simulator written in SystemVerilog (remember G7: build it yourself).

What it receives: the instruction stream (from the IF input monitor) and the data requests (from the LSU input monitor) plus reset/enable events. **What it produces:** the **expected** architectural state after every retired instruction: **PC**, **x1..x31**, the expected memory image, and the expected next PC.

Why this and not "expected values per instruction"? Because the expected value of an instruction depends on all previous instructions. A stateful model handles that automatically.

7.9 Scoreboard (uvm_scoreboard)

What it is: the judge. It keeps two views and compares them:

- **expected** <- from the reference model;
- **actual** <- from the monitors (retire stream, bus transactions, final register file and memory image).

It also holds a **shadow memory** (its own copy of the data memory, updated from the observed stores) so it can check store->load consistency without trusting the DUT.

ORPLAN link: almost every **Checking** row (**sb_rv32i_result_match**, **sb_load_result_match**, **sb_store_mem_match**, **sb_mem_consistency**, **sb_pc_flow_match**, **sb_no_unexpected_trap**, **sb_div_latency_range**, **sb_waw_younger_wins**, **sb_final_state_match**, ...).

7.10 Coverage (Coverage (Subscriber))

Coverage measures what we exercised, not whether it was right. A **covergroup** declares **coverpoints** (variables to sample) and **bins** (interesting values/ranges), and **crosses** (combinations of two or more coverpoints).

Rule from the guidelines (G8): "No need for coverpoints that are out of the scope." So: no FPU bins, no CSR bins, no compressed-instruction bins, no register-file port-C coverpoints.

7.11 Assertions (SVA)

SVA = SystemVerilog Assertions: small always-on properties compiled into the simulation, e.g.

```
// concept (not project code): request persistence on the instruction bus
property p_req_persistence;
  @(posedge clk) (instr_req_o && !instr_gnt_i) |=> instr_req_o;
endproperty
```

They are the natural home for **interface protocol rules** (G4: "implement the required interface checkers") and for the environment's own self-checks (e.g. "the TB slave must never return two **rvalids** for one grant").

Where ORPLAN puts them: inside the **agents' output monitors** and inside the **memory-model drivers** (**Checking** sheet, "comment" column) - because those components know the transaction context (e.g. which access type/offset produced a given **data_be_o**).

7.12 Watchdog

A timer that fails the test if **nothing retires for N cycles**. Without it, a hung core (e.g. waiting forever for a response we never send) would simply consume simulation time until the simulator's global limit. **ORPLAN link: risc_env_02** ("a global watchdog timeout catches any hung core").

8. ORPLAN - Detailed Explanation

8.1 How ORPLAN is organised

FACT - ourplan/risc_v_verification_plan (1).xlsx has five sheets, identical in structure to **plantemplate/** (which is the same template filled with an APB/AHB example):

Sheet	Column B		Column C	Column D		Rows
System	Design Description	Requirement	Implementation / note	component name		49 requirements (risc_sys_00 ... risc_scope_00)
Generation_	Design Description	Requirement	Generation stimulus)	(what component/Object name	property name (+ comment: where it lives)	38 rows
Checking_	Design Description	Requirement	check			41 rows
Coverage_	Design Description	Requirement	Coverage	covergroup		37 rows
test list	test name		sequence run	stimuls generated		27 tests

The **ID column is the key**: the same ID (**risc_lsu_04**, for example) appears in all four requirement sheets. Reading the four rows with the same ID gives you the complete specification of one verification item:

```
risc_lsu_04 (System)      -> WHAT the DUT must do
risc_lsu_04 (Generation) -> WHAT stimulus we create
risc_lsu_04 (Checking)   -> HOW we decide right/wrong (+ property name)
risc_lsu_04 (Coverage)   -> HOW we prove we exercised it (+ covergroup name)
```

Note on IDs: the plan skips **risc_if_07** and **risc_rf_00** appears only in **System+Checking** (not in **Generation/_Coverage_**). These are small inconsistencies in the plan, not errors in the DUT. Listed in section 13.

Below, every item is explained with the same 12-point template: **1 What is it? - 2 Why is it here? - 3 Required behaviour - 4 Stimulus - 5 DUT behaviour - 6 What to observe - 7 PASS - 8 FAIL - 9 RTL location - 10 Guideline - 11 Databook - 12 Flow.**

8.2 Family **risc_sys_*** - System, reset, start-up, ports, environment

8.2.1 **risc_sys_00** - DUT = **cv32e40p_top** at top level

1. What is it? The declaration of what we verify: the complete core, instantiated in **top_tb**, with the **default** parameters. **2. Why?** Guideline G5: "Focus is top level verification" (not block level). Databook **integration.rst.txt**: "The main module is named **cv32e40p_top**". **3. Required behaviour:** one instance of **cv32e40p_top**, parameters at their defaults (**FPU=0, COREV_PULP=0, COREV_CLUSTER=0, ZFINX=0, NUM_MHPMCOUNTERS=1**), driven as a black box through its ports. **4. Stimulus:** n/a (structural requirement). **5. DUT behaviour:** n/a. **6. Observe:** that the DUT instance is the RTL file **rtl/cv32e40p_top.sv** and that the parameters are not overridden. **7. PASS:** elaboration succeeds, the instance exists, and later checks (fetch from **boot_addr_i**, RV32IM behaviour) work. **8. FAIL:** a non-default parameter is applied (e.g. **FPU=1**, which would need files that are not in this repo), or the wrong module is instantiated (**cv32e40p_core** instead of **cv32e40p_top**). **9. RTL:** **rtl/cv32e40p_top.sv** (module header, lines 15-23). **10. Guideline:** G5 (top level), G1 (minimal extras). **11. Databook:** **integration.rst.txt**.

8.2.2 risc_sys_01 - every port connected/observed/tied; no X propagation

1. What is it? A hygiene rule: no floating inputs, no unconnected outputs, no Xs. **2. Why?** An X on an input can propagate into the design and destroy all later comparisons. The Databook also tells us which inputs must be tied: `irq_i[15:12], irq_i[10:8], irq_i[6:4], irq_i[2:0]` "shall be tied to 0 externally" (`exceptions_interrupts.rst.txt`); `pulp_clock_en_i` "Tie to 0" when `COREV_CLUSTER=0`; `scan_cg_en_i` "should be 0 during normal / functional operation". **3. Required behaviour:** * driven: `clk_i`, `rst_ni`, `fetch_enable_i`, `boot_addr_i`, `mtvec_addr_i`, `hart_id_i`, `instr_gnt_i`, `instr_rvalid_i`, `instr_rdata_i`, `data_gnt_i`, `data_rvalid_i`, `data_rdata_i`; * tied to 0: `irq_i`, `debug_req_i`, `pulp_clock_en_i`, `scan_cg_en_i`; * tied (any stable value): `dm_halt_addr_i`, `dm_exception_addr_i` (never used if debug never requests); * observed/ignored: `core_sleep_o`, `irq_ack_o`, `irq_id_o`, `debug_havereset_o`, `debug_running_o`, `debug_halted_o`. **4. Stimulus:** static configuration + an X-checker on every DUT input. **5. DUT behaviour:** runs normally, never enters debug or sleep. **6. Observe:** all DUT input pins (an SVA `$isunknown()` check), plus `irq_ack_o`/`debug_*_o` staying quiet. **7. PASS:** no X/Z on any DUT input after reset de-assertion, for the whole test; `debug_halted_o` never asserted; `irq_ack_o` never asserted. **8. FAIL:** any X/Z on an input; an unexpected `irq_ack_o`; entering debug mode. **9. RTL:** `rtl/cv32e40p_top.sv` port list; RTL assertion precedent: `cv32e40p_load_store_unit.sv` has a `address_phase_signals_defined` (!`$isunknown(data_addr_o) && ...`) which the plan copies in spirit. **10. Guideline:** G1 ("minimal work that does not hinder the main task" - tying `debug_req_i` is exactly that), G3. **11. Databook:** `integration.rst.txt` (interfaces table), `sleep.rst.txt`, `exceptions_interrupts.rst.txt`. **12. Flow:** `top_tb` (tie-offs) -> DUT inputs -> SVA X-check + status-output monitor -> PASS/FAIL.

[!] Open question recorded in ORPLAN itself: the plan's note column for `risc_sys_01` says "what about boot address and mtvec address?". See 8.2.4 and section 13.

8.2.3 risc_sys_02 - reset: outputs inactive and the internal clock is gated

1. What is it? While `rst_ni == 0` (and until `fetch_enable_i` is seen high) the core must be electrically quiet and internally clock-gated. **2. Why?** Databook `sleep.rst.txt`: "`clk_i` is internally gated off during `rst_ni` assertion" and "from `rst_ni` deassertion until `fetch_enable_i = 1`". The `integration.rst.txt` interfaces table adds that `rst_ni` is **asynchronous** - so the reaction must be immediate, not at the next clock edge. **3. Required behaviour (step by step):** 1. `rst_ni` falls (at an arbitrary time, not aligned to `clk_i`). 2. Immediately (asynchronously) `instr_req_o = 0` and `data_req_o = 0`. 3. The sleep unit's clock gate closes -> no internal toggling, no bus activity, `core_sleep_o = 0` during reset. 4. `rst_ni` rises: the core stays quiet until `fetch_enable_i` has been 1 for at least one cycle. **4. Stimulus:** `sys_ctrl_init_sequence` / `sys_ctrl_reset_inject_sequence`: random reset hold time, random de-assert phase relative to `clk_i`, plus mid-test re-assertion (this is what makes it interesting: reset while the pipeline is busy). **5. DUT behaviour:** `cv32e40p_sleep_unit.sv`: `fetch_enable_q <= 0` on reset -> `clock_en = 0`; `cv32e40p_controller.sv` state `RESET`: `instr_req_o = 1'b0`. **6. Observe:** `instr_req_o`, `data_req_o` at every clock edge while `rst_ni == 0`; optionally `core_sleep_o`. **7. PASS:** `sys_reset_no_req_prop` never fires: `instr_req_o == 0 && data_req_o == 0` in every cycle with `rst_ni == 0`. **8. FAIL:** any * `req_o` pulse during reset; any bus transaction started; the clock gate not closed (visible as continued internal activity, e.g. PC changing). **9. RTL:** `rtl/cv32e40p_sleep_unit.sv` (`clock_en`, `core_sleep_o`), `rtl/cv32e40p_controller.sv` (state `RESET`), `rtl/cv32e40p_prefetch_buffer.sv/cv32e40p_load_store_unit.sv` (request generation). **10. Guideline:** G1 (minimal work to make the core run). **11. Databook:** `sleep.rst.txt` "Startup behavior". **12. Flow:**

```
sys_ctrl_reset_inject_sequence
-> sys_ctrl_agent.sequencer -> sys_ctrl_agent.driver
-> System Control Interface (rst_ni)
-> DUT (sleep_unit clock gate + controller RESET state)
-> if_passive_agent.output_monitor / lsu_passive_agent.output_monitor observe *_req_o
-> SVA sys_reset_no_req_prop
-> PASS / FAIL
```

8.2.4 risc_sys_03 - fetch_enable_i gating and stable static configuration

1. What is it? The "start button" of the core, plus the rule that the static configuration inputs must not change after start. **2. Why?** Databook `integration.rst.txt`: "The first instruction fetch after reset de-assertion will not happen as long as this signal is 0. `fetch_enable_i` needs to be set to 1 for at least one cycle while not in reset ... Once fetching has been enabled the value `fetch_enable_i` is ignored." and, for `boot_addr_i`/`mtvec_addr_i`/`dm_halt_addr_i`/`dm_exception_addr_i`: "Do not change after enabling core via `fetch_enable_i`". **3. Required behaviour:** 1. No `instr_req_o` pulse occurs between reset de-assertion and the first cycle with `fetch_enable_i == 1`. 2. After `fetch_enable_i` has been 1 for ≥ 1 cycle, fetching proceeds and later changes of `fetch_enable_i` (e.g. 1->0) are ignored until the next reset. 3. `boot_addr_i`, `mtvec_addr_i`, `dm_*_addr_i`, `hart_id_i` are held constant once enabled. **4. Stimulus:** `sys_ctrl_init_sequence` with the field `fe_delay` randomised over {0, 1, 2..8, >8} cycles (exactly the bins of `fetch_enable_delay_cg`); `boot_addr_i`/`mtvec_addr_i` programmed at corner values. **5. DUT behaviour:** `cv32e40p_sleep_unit.sv`: assign `fetch_enable_d = fetch_enable_i ? 1'b1 : fetch_enable_q`; (sticky FF, cleared only by reset) -> `clock_en -> cv32e40p_controller.sv RESET -> BOOT SET -> first fetch at {boot_addr_i[31:2], 2'b00}`. **6. Observe:** `instr_req_o` (no pulse before enable), the first `instr_addr_o` (= boot address), and that later toggling of `fetch_enable_i` does not stop execution. **7. PASS:** `sys_no_fetch_before_fe_prop` holds; the first fetch address equals the programmed boot address; execution continues after `fetch_enable_i` is lowered; all static config inputs unchanged for the whole test. **8. FAIL:** a fetch before enable; the core stopping when `fetch_enable_i` is lowered; a first fetch at an address other than `boot_addr_i`; any static input changing after enable. **9. RTL:** `cv32e40p_sleep_unit.sv` (`fetch_enable_q`), `cv32e40p_controller.sv` (`RESET/BOOT SET`), `cv32e40p_if_stage.sv` (`PC BOOT: branch_addr_n = {boot_addr_i[31:2], 2'b00}`). **10. Guideline:** G1. **11. Databook:** `integration.rst.txt` (`fetch_enable_i`, `boot_addr_i`, `mtvec_addr_i` descriptions). **12. Flow:** `sys_ctrl_init_sequence(fe_delay)` -> driver -> `fetch_enable_i` -> sleep unit -> controller -> first `instr_req_o/instr_addr_o` -> if_passive_agent.output_monitor -> SVA `sys_no_fetch_before_fe_prop` + scoreboard "first PC == boot_addr_i" -> PASS/FAIL.

[!] Discrepancy to be aware of (plan vs Databook): the Databook says `boot_addr_i` "Must be half-word aligned", while ORPLAN `risc_if_08` only guarantees word-aligned programs/targets. The RTL masks bit 1 anyway (`{boot_addr_i[31:2], 2'b00}`), so a half-word-aligned boot address would silently drop bit 1. Recommendation: generate word-aligned boot addresses (ORPLAN's own `risc_if_08` rule) and treat half-word alignment as an untested corner. Section 13.

8.2.5 risc_env_00 - TB-owned memories, deterministic pre-load

1. What is it? The TB owns an instruction memory and a data memory (OBI slaves); the instruction memory is pre-loaded with the program and the data memory with a deterministic data pool. **2. Why?** Guideline G3 explicitly allows/requires it: "Including the memory if they see it as a requirement". Deterministic initialisation is what makes the golden comparison possible: if a load of an unwritten location returned **X**, the scoreboard could not compare it. **3. Required behaviour:** both memories answer OBI transactions; the data image is fully initialised (no **X**); stores commit at **rvalid**; no reordering. **4. Stimulus:** **riscv_program** output -> **instr mem model**; a fixed data pool -> **data mem model**. **5. DUT behaviour:** unaffected (this is TB behaviour). **6. Observe:** every load value, and the final data memory image. **7. PASS:** all load results match the golden model, including loads of never-written addresses; final **dmem == golden (sb final state match)**. **8. FAIL:** any mismatch; any **X** returned. **9. RTL:** n/a (TB). **10. Guideline:** G3, G7. **11. Databook:** **load_store_unit.rst.txt** (memory behaviour), **verification.rst.txt** (TB approach).

8.2.6 risc_env_01 - OBI response features (gnt/rvalid delays, modes)

1. What is it? The bus-timing knobs of the TB slaves: per-transaction **gnt** delay (0..K), **rvalid** delay (1..L), and the modes zero-wait, random, long-stall bursts, **gnt**-always-high, independently per bus. **2. Why?** Guideline G4 requires real OBI knowledge and "required interface checkers"; the Databook's cycle table assumes **zero wait states**, so wait states are the main way to stress the pipeline's stall logic. **3. Required behaviour:** the slave may delay **gnt** and **rvalid** freely (within the in-order rule), may hold **gnt** high with no request, and must give exactly one **rvalid** per granted request. **4. Stimulus:** **obi memory sequence** (modes + delay distributions; see **risc_obi waitstate test**, **risc_obi stall stress test**, **risc_gnt always high test**). **5. DUT behaviour:** the core must hold its request (address stable), must not retract it, must not create more than 2 outstanding transactions, and must apply back-pressure (pipeline stall) instead of issuing more. **6. Observe:** *** req o/* gnt i/* rvalid i** timing, outstanding counters, and that results are still architecturally correct. **7. PASS:** all protocol properties hold (if **obi slave protocol prop**, **lsu max outstanding prop**, **env slave protocol prop**) and every architectural result still matches the golden model. **8. FAIL:** retracted/changed request, >2 outstanding, two **rvalids** for one grant, a result mismatch under wait states, or a lost/duplicated transaction. **9. RTL:** **cv32e40p_obi interface.sv** (FSM), **cv32e40p_prefetch_controller.sv** (cnt q, trans valid o), **cv32e40p_load_store_unit.sv** (cnt q, trans valid, lsu ready ex o). **10. Guideline:** G4. **11. Databook:** **instruction_fetch.rst.txt**, **load_store_unit.rst.txt** (Protocol sections), **pipeline.rst.txt** ("assume zero stall" caveat).

8.2.7 risc_env_02 - teardown: drain, complete retire stream, final state compare, watchdog

1. What is it? The end-of-test procedure. **2. Why?** Without a defined ending, tests would be killed mid-flight and results would be incomplete. **3. Required behaviour:** 1. Deliver all outstanding responses so both buses are quiet (**outstanding == 0**). 2. Confirm the retire stream is complete (the program reached the end-of-program sentinel). 3. Compare the final register file and the final data memory against the golden model. 4. A **global watchdog** fails the test if nothing retires for N cycles. **4. Stimulus:** end-of-program sentinel generated by **riscv_program**; the watchdog runs continuously. **5. DUT behaviour:** executes until the sentinel. **6. Observe:** **sb final state match** (GPRs + dmem), **env teardown check** (outstanding == 0, no timeout). **7. PASS:** all four conditions above. **8. FAIL:** outstanding transactions left, a missing/extra retire entry, a final-state mismatch, or a watchdog timeout. **9. RTL:** n/a (TB), except that the retirement evidence comes from the core. **10. Guideline:** G5. **11. Databook:** **verification.rst.txt** (tracer/retire approach).

8.2.8 risc_scope_00 - the explicit out-of-scope list

1. What is it? A single row listing everything we keep inactive or never generate: C-extension internals (aligner, compressed decoder), CSR instructions and CSR state, interrupts, debug, sleep/WFI/**cv.elw**, FPU and FP CSRs, CORE-V custom instructions and CSRs, atomics/PMP/user mode, performance counters, FENCE/ECALL/EBREAK/MRET. **2. Why?** It converts the scattered guideline answers (G1, G6, G9, G10, Team 2 Q1) into one checkable statement: "No checkers or coverage are built for these." **3. Required behaviour:** these features are either tied inactive or simply never stimulated. **4-6. Stimulus/observe:** none (that is the point). **7. PASS:** the generated program contains none of these encodings; the tied inputs stay tied; no coverage group references them. **8. FAIL:** the program builder emits a compressed/FPU/CSR/custom encoding; a coverage bin is defined for an out-of-scope feature (violates G8). **9. RTL:** the corresponding modules are still in the design (**cv32e40p_aligner.sv**, **cv32e40p_compressed_decoder.sv**, **cv32e40p_cs_registers.sv**, **cv32e40p_int_controller.sv**, **cv32e40p_hwloop_regs.sv**, **cv32e40p_apu_disp.sv**) - they are exercised only as **pass-through** hardware. **10. Guideline:** G1, G6, G8, G9, G10, Team 2 Q1. **11. Databook:** **intro.rst.txt** (extension list).

Deep dive - what "C is always enabled" means for us: the compressed decoder sits between the fetch FIFO and the ID stage. Because we only feed 32-bit words, the decoder always reports "not compressed" and passes the word through. We do not verify it, but it is in the data path, so any bug there would show up as a wrong instruction. This is exactly the kind of "pass-through hardware" the plan calls out.

8.3 Family risc_if_* - Instruction fetch

8.3.1 risc_if_00 - request persistence and address stability

1. What is it? Once the core raises **instr_req_o**, it must keep it high (and keep **instr_addr_o** stable) until **instr_gnt_i** is sampled high for one cycle. The address may only change the cycle after the grant. **2. Why?** Databook **intro.rst.txt** (history): "the protocol forces the address stability. Thus, the core can not retract memory requests once issued, nor can it change the issued address". Guideline G4 requires us to implement this checker. **3. Required behaviour:** **req && !gnt** in cycle N => **req** still high in cycle N+1 and **addr** unchanged. **4. Stimulus:** **obi memory sequence** holding **instr_gnt_i** low for 0..K cycles per fetch, including long stalls. **5. DUT behaviour:** **cv32e40p_obi interface.sv** (**TRANS_STABLE = 0** branch) moves to **REGISTERED**, where **obi_req_o = 1'b1**; // Never retract request and the address comes from **obi_addr_q**. **6. Observe:** **instr_req_o, instr_addr_o** on every cycle. **7. PASS:** **SVA if obi_req_persistence_prop + if obi_addr_stable_prop** never fire. **8. FAIL:** **req** dropping before a grant, or the address changing while **req=1 && gnt=0**. **9. RTL:** **cv32e40p_obi interface.sv** (FSM **TRANSPARENT/REGISTERED**), instantiated in **cv32e40p_prefetch_buffer.sv** with **.TRANS_STABLE(0)**. **10. Guideline:** G4. **11. Databook:** **intro.rst.txt**, **instruction_fetch.rst.txt** ("Request valid, will stay high until **instr_gnt_i** is high for one cycle"). **12. Flow:** **obi memory sequence(gnt delay)** -> **if agent.driver** -> **instr_gnt_i** -> DUT OBI FSM -> **instr_req_o/instr_addr_o** -> **if passive_agent.output_monitor** -> **SVA** -> **PASS/FAIL**.

8.3.2 risc_if_01 - no rvalid -> req dependency; gnt before request is legal

1. What is it? Two rules: (a) **instr_req_o** must not change combinationaly because **instr_rvalid_i** changed; (b) the memory may hold **instr_gnt_i** high even when the core is not requesting, with no side effect. **2. Why?** Databook **intro.rst.txt**: "the request signal does not depend anymore on the rvalid signal (no combinatorial dependency)" and "the grant signal can now be kept high by the bus even without the core raising a request". **3. Required behaviour:** no combinatorial path from **rvalid** to **req**; a **gnt** with no **req** starts no transaction; over the whole test **!(req && gnt) == #(rvalid)**. **4. Stimulus:** **obi_memory_sequence** in **gnt_always_high** mode (**risc_gnt_always_high** test), including windows with no request pending. **5. DUT behaviour:** **PULP_OBI = 0** localparam in **cv32e40p_core.sv** selects **trans_valid_o = req_i && (fifo_cnt_masked + cnt_q < DEPTH)** - no **resp_valid_i** term. The OBI adapter accepts a grant only when it itself is requesting (**trans_ready_o** is irrelevant for grant acceptance in the **gen_trans_stable...** note: on the instruction side the FSM's **REGISTERED** state exits on **obi_gnt_i**, and **obi_req_o** is independent of **instr_rvalid_i**). **6. Observe:** counters of granted requests and of **rvalids**; **instr_req_o** edges versus **instr_rvalid_i** edges. **7. PASS:** if **obi_trans_match_prop**: the two counters match at end of test (no request swallowed, no phantom response accepted). **8. FAIL:** counts differ; a transaction started by a spurious grant; **req** reacting in the same cycle as **rvalid** (detectable as a combinational glitch / delta-cycle dependency - in practice the plan checks it by counter balance plus timing behaviour). **9. RTL:** **cv32e40p_core.sv** (localparam **PULP_OBI = 0**), **cv32e40p_prefetch_controller.sv** (**gen_no_pulp_obi**). **10. Guideline:** G4. **11. Databook:** **intro.rst.txt** (Memory-Protocol history).

8.3.3 risc_if_02 - response phase: one rvalid per grant, in order, <=2 outstanding

1. What is it? The response rules of the instruction bus. **2. Why?** Databook **instruction_fetch.rst.txt**: "**instr_rvalid_i** ... will be high for exactly one cycle per request"; "links are always in-order from master point of view"; "up to **DEPTH** - outstanding transactions" (**DEPTH = 2**). This is the environment's own guarantee, so the plan puts it as an **environment self-check** assertion. **3. Required behaviour:** exactly one **rvalid** per granted request; **rdata** stable during its **rvalid** cycle; responses in issue order; never more than 2 outstanding. **4. Stimulus:** random **rvalid** latency 1..L, including separating the responses of two outstanding requests. **5. DUT behaviour:** the core issues up to 2 requests before seeing any response. **6. Observe:** **instr_rvalid_i** vs outstanding count; **instr_rdata_i** stability. **7. PASS:** if **obi_slave_protocol_prop** (TB-side self-check) + if **obi_outstanding_cg** bins {1,2} hit + if **obi_rvalid_latency_cg** bins {1,2,3-7,>7} hit. **8. FAIL:** a response with no outstanding request; two responses for one grant; out-of-order responses; >2 outstanding.

8.3.4 risc_if_03 - back-to-back fetches

1. What is it? The core can issue a new request in the cycle immediately after a grant. **2. Why?** Databook **pipeline.rst.txt**: "capable of fetching 1 instruction per cycle if the instruction side memory system allows"; the figure **obi_instruction_basic.svg** shows back-to-back transactions. **3. Required behaviour:** with a zero-wait memory, **req** is asserted every cycle and the fetch stream is continuous (no duplicated or lost word). **4. Stimulus:** mixed zero-wait / random-wait windows (**obi_memory_sequence**). **5. DUT behaviour:** after a grant the OBI FSM returns to **TRANSPARENT** and the prefetch controller immediately requests the incremented address. **6. Observe:** the fetch-address stream seen by the monitor versus the retired PC stream. **7. PASS:** **sb_fetch_stream_consistent** - no duplicated/lost fetch word at retire time. **8. FAIL:** a fetch address skipped or repeated, or a retire whose PC was never fetched.

8.3.5 risc_if_04 - speculative prefetch beyond the program (up to 2 words)

1. What is it? The prefetcher may read up to **DEPTH = 2** words past the code being executed. **2. Why?** Databook **instruction_fetch.rst.txt**: "CV32E40P can fetch up to **DEPTH** words outside of the code region and care should therefore be taken that **no unwanted read side effects occur** for such prefetches outside of the actual code region." **3. Required behaviour:** the instruction memory model must serve **any** address (including past the end of the program, and even illegal/unmapped addresses) with a defined value and no side effects, and must tolerate reads beyond the program end. **4. Stimulus:** short programs (so the prefetcher runs past the end) + a memory model with a deterministic default word. **5. DUT behaviour:** issues up to 2 speculative fetch requests that will never be executed. **6. Observe:** the fetch addresses (some beyond the program) and that no crash/error/X occurs. **7. PASS:** all speculative fetches are answered; the retire stream contains only the executed instructions; no test error. **8. FAIL:** the memory model refuses an address, returns **X**, or has a side effect (e.g. asserts an error or modifies state on a read).

8.3.6 risc_if_05 - flush on a taken jump (JAL/JALR), jump costs 2 cycles

1. What is it? Jumps are resolved in **ID**; on a taken jump, the prefetch buffer and IF are flushed and the new PC request appears on the bus in the same cycle the jump is in **ID**; with zero wait states the jump costs **2 cycles**. **2. Why?** Databook **pipeline.rst.txt** table: "Jump | 2 | Jumps are performed in the ID stage. Upon a jump the IF stage (including prefetch buffer) is flushed. The new PC request will appear on the instruction-side memory interface the same cycle the jump instruction is in the ID stage." **3. Required behaviour:** 1. The instruction after the jump (in program order) is **not** executed (it is flushed). 2. The next retired PC is the jump target: **JAL -> PC + imm**; **JALR -> (rs1 + imm) & ~1**. 3. The link register gets **PC + 4** (unless **rd == x0**, where no link happens). 4. Zero-wait cost = 2 cycles. **4. Stimulus:** **riscv_branch_sequence** (jump mode): forward/backward jumps, small and large offsets, link registers **x0/x1/x5**, **JALR** with odd immediates. **5. DUT behaviour:** **cv32e40p_controller.sv** (jump in dec -> **pc_mux_o = PC_JUMP**, **pc_set_o**), **cv32e40p_id_stage.sv** (**JT_JAL: pc_id + imm_uj_type, JT_JALR: regfile_data_ra_id + imm_i_type**), **cv32e40p_if_stage.sv** (**branch_addr_i{branch_addr_n[31:1], 1'b0}**) -> bit 0 cleared), **cv32e40p_prefetch_controller.sv** (**fifo_flush_o == branch_i**). **6. Observe:** the retired PC stream; the **rd** value; the number of cycles from the jump to the target instruction. **7. PASS:** **sb_pc_flow_match + sb_jump_link_match**: fall-through PC never retires after a jump; target retires next; link == **PC+4**; measured latency == 2 at zero wait. **8. FAIL:** a fall-through instruction retiring after a jump (a "leaked" instruction), a wrong target, a wrong link value, or a different cycle count.

8.3.7 risc_if_06 - flush on a taken branch; taken = 3 cycles, not-taken = 1

1. What is it? Branch conditions are evaluated in **EX**; a taken branch flushes **IF** and **ID**; not-taken costs nothing. **2. Why?** Databook **pipeline.rst.txt**: "Branch (Taken) | 3 | The EX stage is used to compute the branch decision. Any branch where the condition is met will be taken from the EX stage and will cause a flush of the IF stage (including prefetch buffer) and ID stage." / "Branch (Not-Taken) | 1". **3. Required behaviour:** on a taken branch, the (up to two) instructions fetched behind it never retire and never touch memory or registers; execution resumes at **PC + imm**;

taken = 3 cycles, not-taken = 1 cycle at zero wait. **4. Stimulus:** `riscv_branch_sequence`: all 6 conditions x taken/not-taken x forward/backward x operand relations including `INT_MIN/INT_MAX`. **5. DUT behaviour:** `cv32e40p_ex_stage.sv assign branch_decision o = alu_cmp_result; cv32e40p_controller.sv if (branch_taken_ex_i) begin pc_mux_o = PC_BRANCH; pc_set_o = 1; is_decoding_o = 0; end`. **6. Observe:** retire stream, data-bus log, register-file state, cycle counts. **7. PASS:** `sb_pc_flow_match, sb_flush_clean, sb_lsu_trans_stream_match`. **8. FAIL:** a flushed-path instruction producing a register write or a memory request (this is the classic and most valuable bug class here), a wrong target, or a wrong cycle count.

8.3.8 risc_if_08 - stimulus constraint: 32-bit, word-aligned, bounded programs

1. What is it? A **generation** rule (it constrains the stimulus, not the DUT). **2. Why?** Guideline G10 ("Only vanilla I & M") + Team 2 Q1 (C extension not a goal). It also makes the whole environment simpler: no compressed decoding, no half-word-aligned targets, no **FENCE.I**. **3. Required behaviour:** only 32-bit RV32I/M encodings; all targets word-aligned and inside the program window; the program ends by reaching the end-of-program sentinel. **4. Stimulus:** enforced inside `riscv_program` (the program builder). **5. DUT behaviour:** n/a. **6. Observe:** the generated program (a build-time check). **7. PASS:** every generated word is a legal 32-bit RV32I/M encoding or a deliberately-injected illegal word; every target is word-aligned; the program terminates. **8. FAIL:** a compressed (16-bit) encoding, a non-word-aligned target, or a program that runs away.

8.3.9 risc_if_09 - no instruction-address-misaligned exception can exist (anti-check)

1. What is it? A **sanity / anti-check**: on a stream of legal instructions the core must never trap. **2. Why?** Databook `instruction_fetch.rst.txt`: "Externally, the IF interface performs word-aligned instruction fetches only. Misaligned instruction fetches are handled by performing two separate word-aligned instruction fetches. Internally, the core can deal with both word- and half-word-aligned instruction addresses ... **The LSB of the instruction address is ignored internally.**" **3. Required behaviour:** no trap redirect ever occurs on a legal instruction stream. **4. Stimulus:** every test (it is a continuous check), especially the random mix. **5. DUT behaviour:** `cv32e40p_prefetch_buffer.sv` assertion `p_instr_addr_word_aligned (instr_addr_o[1:0] == 2'b00)`. **6. Observe:** the PC stream; any jump to the `mtvec` base. **7. PASS:** `sb_no_unexpected_trap` - zero unexpected redirects for the whole regression. **8. FAIL:** any jump to `{mtvec_addr_i[31:8], 8'h00}` that was not caused by an injected illegal instruction. **9. RTL:** `cv32e40p_prefetch_buffer.sv` (assertion), `cv32e40p_prefetch_controller.sv` (`aligned_branch_addr = {branch_addr_i[31:2], 2'b00}`). **10. Guideline:** G1 (it is a cheap, high-value sanity check on the main task). **11. Databook:** `instruction_fetch.rst.txt` "Misaligned Accesses".

8.4 Family risc_lsu_* - Load/Store Unit

8.4.1 risc_lsu_00 - data request handshake and address/control stability

1. What is it? `data_req_o` stays high until `data_gnt_i` is high for one cycle, and `data_addr_o`, `data_we_o`, `data_be_o`, `data_wdata_o` - which are sent together with the request - must stay stable while `req == 1 && gnt == 0`. **2. Why?** Databook `load_store_unit.rst.txt`: "`data_req_o` Request valid, will stay high until `data_gnt_i` is high for one cycle"; "`data_we_o` ... Sent together with `data_req_o`"; same for `be` and `wdata`. Guideline G4 requires the checker. **3. Required behaviour:** identical in spirit to `risc_if_00`, but on the data bus. **4. Stimulus:** `obi_memory_sequence` (`lsu_channel`) with `gnt` delayed 0..K cycles, including multi-cycle stalls. **5. DUT behaviour:** the data OBI adapter is instantiated with `TRANS_STABLE = 1` (pins follow the internal `trans_*` signals), so stability comes from the LSU/pipeline holding `trans_*` - which happens because `lsu_ready_ex_o` de-asserts and freezes EX/ID (section 5.1). **6. Observe:** `data_req_o`, `data_addr_o`, `data_we_o`, `data_be_o`, `data_wdata_o` every cycle. **7. PASS:** `lsu_obi_req_persistence_prop + lsu_obi_stable_prop`. **8. FAIL:** a retracted request or a changed address/control while waiting for the grant. **9. RTL:** `cv32e40p_load_store_unit.sv` (`trans_addr, trans_we, trans_be, trans_wdata, trans_valid`), `cv32e40p_obi_interface.sv` (`gen_trans_stable`). **12. Flow:** `obi_memory_sequence(gnt delay) -> lsu_agent.driver -> data_gnt_i -> DUT LSU (cnt q, trans_valid) -> data_req_o/data_addr_o/... -> lsu_passive_agent.output_monitor -> SVA -> PASS/FAIL`.

8.4.2 risc_lsu_01 - response phase: one **rvalid** per grant, ends writes too, <=2 outstanding

1. What is it? `data_rvalid_i` is high for exactly one cycle and terminates **both** reads and writes; responses are in order; at most 2 outstanding; when responses lag the core stalls rather than issuing more requests. **2. Why?** Databook `load_store_unit.rst.txt`: "`data_rvalid_i` will be high for exactly one cycle to signal the end of the response phase for both read and write transactions"; "up to 2 outstanding transactions and there is no FIFO to allow more outstanding requests". **3. Required behaviour:** as stated; plus back-pressure instead of a third request. **4. Stimulus:** slow-response windows (`rvalid` delayed K cycles) with a store followed by a load -> saturates the 2-outstanding depth. **5. DUT behaviour:** `lsu_ready_ex_o/lsu_ready_wb_o` fall -> EX/WB stall. **6. Observe:** outstanding count (must never exceed 2), `rvalid` per grant, order. **7. PASS:** `lsu_obi_slave_protocol_prop + lsu_max_outstanding_prop; lsu_outstanding_cg` bins {1,2}; `lsu_obi_rvalid_latency_cg` bins {1,2,3-7,>7}; `lsu_stall_mid_pair_cg`. **8. FAIL:** >2 outstanding; a third request issued while 2 are pending; missing/extra `rvalid`. **9. RTL:** `cv32e40p_load_store_unit.sv` (`DEPTH = 2, cnt q, lsu_ready_ex_o = (data_req_ex_i==0) ? 1'b1 : (cnt_q==0) ? (trans_valid && trans_ready) : (cnt_q==1) ? (resp_valid && trans_valid && trans_ready) : resp_valid`).

8.4.3 risc_lsu_02 - cycle cost: 1 bus transaction for aligned, 2 for misaligned

1. What is it? Informational timing: at zero wait states a load/store uses EX and WB for 1 cycle each; a misaligned (split) access uses 2 bus transactions and 2 cycles in each stage. **2. Why?** Databook `pipeline.rst.txt` cycle table (section 4.3). **3. Required behaviour:** aligned load/store = 1 bus transaction; non-word-aligned word, or halfword crossing a word boundary = 2 bus transactions. `cv.elw` (4 cycles) is out of scope (needs `COREV_CLUSTER = 1`). **4. Stimulus:** zero-wait memory; count transactions per instruction. **5. DUT behaviour:** `data_misaligned_o` decides (section 5.4b). **6. Observe:** the number of `data_req_o/data_gnt_i` handshakes per retired load/store. **7. PASS:** the count matches 1 or 2 as classified by `risc_lsu_04`. **8. FAIL:** 3+ transactions for one instruction, or 1 transaction where 2 are required (data loss). **9. RTL:** `cv32e40p_load_store_unit.sv` (`data_misaligned_o, misaligned_st`), `cv32e40p_controller.sv` (`misaligned_stall_o = data_misaligned_i`).

8.4.4 risc_lsu_03 - byte-enable generation (full table)

1. What is it? `data_be_o[3:0]` tells the memory which of the 4 bytes of the addressed word are being touched. `data_be_o[0]` = byte at the lowest address of the word, `data_be_o[3]` = highest. **2. Why?** The CV32E40P data bus is 32 bits wide with byte lanes; sub-word and misaligned accesses are implemented purely by byte enables + data rotation, never by narrower bus widths. **3. Required behaviour (FACT, straight from cv32e40p_load_store_unit.sv):**

Instruction	addr[1:0]	beat 1	beat 2
SW	00	1111	-
SW	01	1110	0001
SW	10	1100	0011
SW	11	1000	0111
SH	00	0011	-
SH	01	0110	-
SH	10	1100	-
SH	11	1000	0001
SB	00/01/10/11	0001/0010/0100/1000	-

"Only these patterns may ever appear on data_be_o." **4. Stimulus: riscv_lsu_sequence (offset sweep):** lw/sw at all 4 offsets, lh/sh at all 4 offsets, lb/lbu/sb at all 4 offsets. **5. DUT behaviour:** the `always_comb` "BE generation" block (section 5.4a). **6. Observe:** `data_be_o` on every granted data transaction, together with the access type and phase known by the monitor. **7. PASS:** `lsu_be_legal_prop` - every observed `be` is in the legal set for that (type, offset, phase). **8. FAIL:** any other pattern (e.g. 1111 on a misaligned first beat, or `be == 0000`). **9. RTL:** `cv32e40p_load_store_unit.sv` (BE generation `always_comb`). **10. Guideline:** G4 + G1 (it is the data-path behaviour of the I extension). **11. Databook:** `load_store_unit.rst.txt` (signal table + misaligned paragraph).

8.4.5 risc_lsu_04 - MISALIGNED DATA ACCESS (the requested deep dive)

This is the item the task asks to be explained from first principles.

(i) What a memory access is. The core executes **load** instructions (copy bytes from memory into a register: **LW** word, **LH** halfword sign-extended, **LHU** halfword zero-extended, **LB** byte sign-extended, **LBU** byte zero-extended) and **store** instructions (copy a register into memory: **SW**, **SH**, **SB**).

(ii) What "aligned" means. Memory is byte-addressed but the core's data bus transfers **words = 4 bytes** at a time. A word "lives" at addresses **0x1000, 0x1004, 0x1008, ...** Four byte lanes inside that word are selected by `data_be_o`:

```
word at 0x1000 : [byte3 @0x1003][byte2 @0x1002][byte1 @0x1001][byte0 @0x1000]
data_be_o      :      be[3]      be[2]      be[1]      be[0]
```

- A **word** access is naturally aligned when `addr[1:0] == 00` (a multiple of 4).
- A **halfword** access is naturally aligned when `addr[0] == 0` (a multiple of 2). Offsets **01** and **10** are aligned for a halfword (the two bytes live inside one word: `be 0110 / 1100`); offset **11** is **not** - the halfword straddles two words (byte 3 of word N and byte 0 of word N+1).
- A **byte** access is always aligned.

(iii) What "misaligned" means here. An access is misaligned (split) when the data item **crosses a word boundary**:
* a word at `addr[1:0] != 00` (offsets 1, 2 or 3); * a halfword at `addr[1:0] == 11`.

(iv) Why alignment matters. One bus transaction can only touch the 4 byte lanes of **one** word. An item that spans two words therefore needs **two** transactions. Many CPUs instead raise an address-misaligned exception and let software fix it up. **The CV32E40P does not** - it performs the split in hardware. **FACT** (Databook `load_store_unit.rst.txt`): "The LSU **never** raises address-misaligned exceptions." **FACT** (RTL): `cv32e40p_load_store_unit.sv` computes `data_misaligned_o` and the controller turns it into a stall - there is no exception path for it.

(v) How the RTL detects misalignment.

```
if ((data_req_ex_i == 1'b1) && (data_misaligned_ex_i == 1'b0)) begin
    case (data_type_ex_i)
        2'b00: if (data_addr_int[1:0] != 2'b00) data_misaligned_o = 1'b1; // word
        2'b01: if (data_addr_int[1:0] == 2'b11) data_misaligned_o = 1'b1; // halfword crossing
    endcase
end
```

`data_misaligned_o` goes to the controller, which asserts `misaligned_stall_o` (freezing ID/IF) while the LSU performs the second beat.

(vi) What the two beats look like on the bus. Take **SW x5, 1(x6)** with `x6 = 0x1000` -> effective address **0x1001**, data **0xAABBCCDD**:

Beat	data_addr_o	data_be_o	data_wdata_o	bytes written
			{0x00BBCCDD} ...	
1 (first)	0x1001	1110	rotated by 1 -> {DD[23:0], DD[31:24]}	0x1001, 0x1002, 0x1003
2 (second)	0x1004	0001	same rotated word	0x1004

Mechanism: **wdata_offset = addr[1:0] - reg_offset[1:0]** rotates the word so that the byte that belongs to **0x1001** sits in lane 1 (**riscv_lsu_05**), and for beat 2 the ID stage forces **alu_operand_b_ex_o = 32'h4** and **prepost_useincr_ex_o = 1**, so **data_addr_int = 0x1005** and **trans_addr = {0x1005[31:2], 2'b00} = 0x1004** (section 5.4c).

For a load, e.g. **LW x7, 1(x6)** at **0x1001**: * beat 1 returns the word at **0x1000**; the LSU stores it in **rdata_q**; * beat 2 returns the word at **0x1004**; the LSU assembles **rdata_w_ext = {resp_rdata[7:0], rdata_q[31:8]}** -> the byte from **0x1004** becomes the **top** byte, and bytes 3..1 of the first response become the low 24 bits. [ok] Correct little-endian reassembly.

(vii) What the DUT is supposed to do (summary). 1. No exception, no trap, no **mepc** update. 2. Exactly two OBI transactions, lowest address first, second at aligned base + 4. 3. Byte enables exactly as in the table of 8.4.4(iii). 4. The instruction retires **once**, with one register write, when the second **rvalid** returns. 5. One stall of the ID/IF stages while the second beat runs.

(viii) What happens to the interface request: nothing is cancelled or retried - the second beat is a **new, complete OBI transaction** with its own **req/gnt/rvalid**.

(ix) What the testbench must do. * Stimulus (**riscv_lsu_misaligned_sequence**): **lw/sw** at offsets 1, 2, 3; **lh/sh** at offset 3 (crossing) and offsets 1, 2 (single beat, to prove the boundary of the rule); back-to-back misaligned pairs; misaligned access while the bus stalls (**lsu_stall_mid_pair_cg**). * Memory model: apply only the selected byte lanes per beat; never error; keep the two beats in order. * Checks: * **lsu_misaligned_order_prop** (SVA in **lsu_passive_agent.output_monitor**): beat 1 address = the original address with the phase-1 **be**; beat 2 address = aligned base + 4 with the phase-2 **be**. * **sb_no_exc_on_misaligned**: no trap redirect happened. * **sb_load_result_match / sb_store_mem_match**: the final loaded value / stored bytes equal the golden model (the golden model is a simple byte-accurate memory - it does not care about beats). * Coverage: **lsu_misaligned_cg** (word@1/@2/@3, half@3, half@1/@2 single-beat) x **misaligned_wait_cross_cg** (wait states on beat 1 / beat 2).

(x) PASS condition. For every misaligned access: exactly two transactions, in the right order, with the right addresses and byte enables, **no trap**, one retirement, and the golden value in the register / the golden bytes in memory. All **lsu_misaligned_cg** bins hit.

(xi) FAIL conditions. * An address-misaligned trap (jump to the **mtvec** base) -> contradicts the Databook. * Only one transaction (half of the data would be lost). * Wrong second address (e.g. the same word again instead of +4). * Illegal byte-enable pattern. * Two register writes, or a write with the wrong (partially assembled) value. * The value assembled in the wrong byte order.

(xii) Flow.

```
riscv_lsu_misaligned_sequence (lw/sw @1,2,3 ; lh/sh @3 ...)
  v
riscv_program (builds the program words)
  v
instr_mem_model (pre-load) -> if_agent.driver answers fetches
  v
DUT: ID decodes LW -> EX computes 0x1001 -> LSU sets data_misaligned_o
  v
controller asserts misaligned_stall
LSU beat 1: data_req_o=1, addr=0x1001, be=1110 -> gnt -> rvalid
LSU beat 2: ID forces operand_b=4 -> addr=0x1004, be=0001 -> gnt -> rvalid
  v
LSU assembles rdata -> WB writes rd
  v
lsu_passive_agent.output_monitor (2 transactions, addresses, be, order) -> SVA
lsu_misaligned_order_prop
rvfi_monitor (one retire: PC, rd, value) -> scoreboard
ref_model (golden: read 4 bytes from a byte-accurate memory) -> expected
  v
scoreboard: sb_load_result_match + sb_no_exc_on_misaligned + sb_lsu_trans_stream_match
  v
coverage: lsu_misaligned_cg, misaligned_wait_cross_cg
  v
PASS / FAIL
```

8.4.6 riscv_lsu_05 - store data rotation

1. What is it? The store data word is **rotated** on the bus so that each data byte lands in the byte lane selected by **data_be_o**. Formula (FACT, RTL): **wdata_offset = addr[1:0] - reg_offset[1:0]**; **01 -> {wdata[23:0], wdata[31:24]}**, **10 -> {wdata[15:0], wdata[31:16]}**, **11 -> {wdata[7:0], wdata[31:8]}**. **2. Why?** The bus is word-oriented: you cannot put "byte 0 of the store" into "byte lane 1" by addressing; you rotate the whole word and let the byte enables pick the lanes. **3. Required behaviour:** the memory applies only the lanes selected by **data_be_o**; the resulting bytes in memory equal the register content placed at the correct addresses. **4. Stimulus:** store value corner pool (**0x00000000, 0xFFFFFFFF, 0x80000000, 0x7FFFFFFF, 0xAAAAAAAA, 0x55555555, 0x01020304, 0xDEADBEEF**) x all offsets. **5. DUT behaviour:** the rotation **always comb**. **6. Observe:** the bytes committed in the memory model (end-of-test compare and in-test store->load read-back). **7. PASS:** **sb_store_mem_match**. **8. FAIL:** bytes at the wrong addresses or with the wrong values. **9. RTL:** **cv32e40p_load_store_unit.sv** (**wdata_offset, data_wdata**).

8.4.7 riscv_lsu_06 - load data alignment and sign/zero extension

1. What is it? **LB/LH sign-extend** (fill the upper bits with the sign bit), **LBU/LHU zero-extend**, **LW** returns the full word; a misaligned load is reassembled from the two responses. **2. Why?** RISC-V spec: the loaded value is written into a 32-bit register, so a smaller item must be extended; which extension is used is encoded in instruction bit 14. **3. Required behaviour:** e.g. **LB of 0x80 -> 0xFFFFF80**; **LBU of 0x80 -> 0x00000080**; **LH of 0x8000 -> 0xFFFF8000**; **LHU -> 0x00008000**; misaligned **LW** at offset 1 -> **{byte@addr+3, byte@addr+2, byte@addr+1, byte@addr+0}** reassembled as described in 8.4.5. **4. Stimulus:** loads where the target byte/halfword MSB toggles 0/1 at every offset. **5. DUT behaviour:** **data_sign_extension_o = {1'b0, ~instr_rdata_i[14]}** (decoder) ->

data_sign_ext q -> rdata b_ext/rdata_h_ext (cv32e40p_load_store_unit.sv). 6. **Observe:** the value of **rd** at retirement. 7. **PASS:** **sb_load_result_match** for every retired load. 8. **FAIL:** wrong extension (sign instead of zero or vice-versa), wrong byte selected, wrong misaligned reassembly. 9. **RTL:** **cv32e40p_decoder.sv (OPCODE_LOAD)**, **cv32e40p_load_store_unit.sv (rdata_w_ext, rdata_h_ext, rdata_b_ext, rdata_q).** 12. **Flow:** **riscv_lsu_sequence(sign pool) -> program -> core -> data_req_o -> memory model -> data_rdata_i -> LSU extension -> WB -> rvfi_monitor -> scoreboard vs ref_model -> PASS/FAIL.**

8.4.8 risc_lsu_07 - memory consistency (store->load read-back)

1. **What is it?** A load must observe the value of the most recent **previously issued** store to the same byte address. 2. **Why?** With a single hart and in-order responses this must hold; the plan checks it explicitly even though the core has no store buffer, because it is the cheapest way to catch a byte-lane bug. 3. **Required behaviour:** the memory model commits a store at its **rvalid** and never reorders. 4. **Stimulus:** **riscv_mem_b2b_sequence:** store->load same address (same and different sizes/offsets), load->store, store->store, load->load, plus addresses one word apart. 5. **DUT behaviour:** issues the accesses in program order; waits for each response. 6. **Observe:** every load value, plus the scoreboard's **shadow memory** (its own copy updated from observed stores). 7. **PASS:** **sb_mem_consistency** - every load value equals the shadow memory's value at that byte address. 8. **FAIL:** a stale value, a partially applied store, or a reordering.

8.4.9 risc_lsu_08 - flushed instructions must not touch the data bus

1. **What is it?** A load/store that is younger (in program order) than a taken branch, jump or trap must be killed **before** it issues its request. No spurious **data_req_o** may appear. 2. **Why?** It is the data-path twin of **risc_if_05/06** and **risc_haz_04**; a leaked memory access is a real architectural bug (it corrupts memory even though the instruction never "happened"). 3. **Required behaviour:** the data-bus transaction stream equals **exactly** the set of loads/stores that retire. 4. **Stimulus:** **riscv_ctrl_mem_mix_sequence:** branch/jump/trap immediately followed (in program order) by loads/stores, including while LSU wait states are active. 5. **DUT behaviour:** on a taken branch the controller sets **is_decoding_o = 0** and flushes IF+ID, so the younger load/store in ID never reaches EX (halting it before **data_req_ex_i**). 6. **Observe:** **data_req_o** transactions vs the retired memory instructions. 7. **PASS:** **sb_lsu_trans_stream_match** (no extra, no missing transactions). 8. **FAIL:** an extra **data_req_o** from a flushed instruction, or a missing one. 9. **RTL:** **cv32e40p_controller.sv (DECODE -> branch_taken_ex_i -> is_decoding_o = 0, FLUSH_EX/FLUSH_WB)**, **cv32e40p_id_stage.sv (data_req_ex_o).**

8.4.10 risc_lsu_09 - there is no error response

1. **What is it?** The OBI data interface has no **err** signal, so the TB memory always completes transactions successfully; no error stimulus is generated or expected. 2. **Why?** Databook **load_store_unit.rst.txt:** the interface "does not implement ... **err** ..."; PMP is not supported. **RTL:** **cv32e40p_core.sv** ties **data_err_i(1'b0)** and the LSU contains an assertion **a_no_error** requiring both error inputs to be 0. 3. **Required behaviour:** never drive an error; never see an error path taken. 4-6. **Stimulus/observe:** none (constraint on the environment). 7. **PASS:** no error is ever injected; the core never takes a load/store-access-fault trap. 8. **FAIL:** the environment modelling an error, or a trap with cause 5/7 appearing.

8.5 Family **risc_dec_***, **risc_alu_***, **risc_m_*** - Decode, ALU, Multiply/Divide

8.5.1 risc_dec_00 - all RV32I base instructions execute correctly

1. **What is it?** The 33 listed instructions (LUI, AUIPC, JAL, JALR, the 6 branches, the 5 loads, the 3 stores, the 6 I-type ALU ops, the 3 shifts-immediate, the 10 R-type ALU ops) must produce the architecturally correct result. **FENCE/FENCE.I/ECALL/EBREAK** are part of RV32I but are **omitted from the stimulus** (guideline G6). 2. **Why?** This is the "I" of "only vanilla I & M" (G1, G10). 3. **Required behaviour:** for every retired instruction: correct **rd**, correct memory effect, correct next PC. 4. **Stimulus:** per instruction class, random registers (including **x0**), corner immediates; **riscv_alu_sequence**, **riscv_upper_sequence**, **riscv_lsu_sequence**, **riscv_branch_sequence**, **riscv_shift_sequence**. 5. **DUT behaviour:** decode in **cv32e40p_decoder.sv**; execute in **cv32e40p_alu.sv / cv32e40p_load_store_unit.sv**; write back from EX (**regfile_alu_we_fw_o**) or WB (**regfile_we_wb_o**). 6. **Observe:** the retire stream (PC, instruction, **rd**, value, memory access). 7. **PASS:** **sb_rv32i_result_match** for every retired instruction; **rv32i_instr_cg** - every mnemonic retired at least once (distinguishing ADD/SUB and SRL/SRA via **funct7**). 8. **FAIL:** any result, memory effect or next-PC mismatch. 9. **RTL:** **cv32e40p_decoder.sv + cv32e40p_alu.sv + cv32e40p_id_stage.sv**. 12. **Flow:** **riscv_alu_sequence -> riscv_program -> instr_mem_model -> Core -> rvfi_monitor (actual) + ref_model (expected) -> uvm_scoreboard -> PASS/FAIL + rv32i_instr_cg.**

8.5.2 risc_dec_01 - all 8 RV32M instructions

Same template as 8.5.1, with **sb_rv32m_result_match** (result and timing class) and **rv32m_instr_cg x m_operands_cg**. **RTL:** **cv32e40p_decoder.sv** (6'b00_0001, **funct3**) rows), **cv32e40p_mult.sv**, **cv32e40p_alu_div.sv**.

8.5.3 risc_dec_02 - illegal-instruction detection (categories a/b/c)

1. **What is it?** Any **undefined encoding inside the RV32I/M opcode space** must raise an illegal-instruction exception. The three categories: * **(a)** an undefined **major opcode** (bits **[6:0]**); * **(b)** a legal major opcode with an undefined **funct3/funct7** combination; * **(c)** the **reserved shift encodings:** **OP-IMM/OP** with **funct3 = 001/101** and **imm[11:5]** (**funct7**) other than **0000000** or **0100000**. 2. **Why?** Databook **exceptions_interrupts.rst.txt:** exception cause 2 = illegal instruction; "The illegal instruction exception ... cannot be disabled and is always active"; and the core raises it for "any instruction that is left undefined" (unless configured as a custom instruction, which is not our case). Guideline G6 excludes ECALL/EBREAK - **not** illegal opcodes. [!] Guideline Team 2 Q10 asked exactly this and was **not answered**; we treat it as in scope because it is decoder behaviour of vanilla I&M and the ORPLAN includes it. (Section 13.) 3. **Required behaviour (from the RTL trace in 5.8):** 1. No register write (**deassert_we_o** when **illegal_insn_i**). 2. No memory request from the illegal instruction. 3. **mepc** <- the PC of the illegal instruction; **mcause**

<- 2. 4. Fetch redirects to {**mtvec_addr_i[31:8], 8'h00**} (the **mtvec base**, because this is an exception, and exceptions are not vectored). 5. Execution resumes at the handler (in our tests the handler is simply part of the generated program). **4. Stimulus: riscv illegal sequence** - random filler bits for each category, injected at random positions; **riscv illegal context** mode - back-to-back illegals, illegals during IF wait-state windows, illegals immediately after taken branches/jumps. Encodings of **disabled** extensions (FPU/atomic/CORE-V custom/CSR opcodes) are **never generated at all** (out of scope). **5. DUT behaviour: cv32e40p_decoder.sv** asserts **illegal_insn_o**; **cv32e40p_controller.sv** walks **DECODE -> FLUSH EX -> FLUSH WB** and sets **pc_mux_o = PC_EXCEPTION**, **exc_pc_mux_o = EXC_PC_EXCEPTION**; **cv32e40p_if_stage.sv** computes **exc_pc = {trap_base_addr, 8'h0}**. **6. Observe:** the retire stream (the illegal instruction must appear as a trap, not as a normal retire), the next fetch address, **rd** unchanged, no data-bus transaction. **7. PASS: sb illegal_trap + exc_trap_base_prop**; **illegal_category_cg** (a/b/c) and **illegal_context_cg** hit. **8. FAIL:** the illegal instruction retiring normally; a register written; a memory access; no redirect; a redirect to the wrong address (e.g. **mtvec base + 4*k**, which is the **interrupt** vector formula, or a base that is not the programmed **mtvec_addr_i**). **9. RTL: cv32e40p_decoder.sv** (all **illegal_insn_o** sites), **cv32e40p_controller.sv** (lines ~537, ~903, ~1022), **cv32e40p_if_stage.sv** (**EXC_PC_EXCEPTION**), **cv32e40p_cs_registers.sv** (**mtvec_n = csr_mtvec_init_i ? mtvec_addr_i[31:8] : mtvec_q**). **10. Guideline:** G6 (ECALL/EBREAK excluded, illegal included), G10. **11. Databook: exceptions_interrupts.rst.txt**. **12. Flow: riscv illegal sequence -> program** (with a defined handler region) -> core -> decoder asserts **illegal_insn_o** -> controller -> next fetch = **mtvec base** -> **if_passive_agent.output_monitor** (address stream) + **lsu_passive_agent** (no transaction) + **rvfi_monitor** (trap retire) -> scoreboard (**sb_illegal_trap**) + SVA (**exc_trap_base_prop**) -> PASS/FAIL.

8.5.4 risc_dec_03 / risc_rf_00 - x0 semantics

1. What is it? **x0** is hardwired to zero: reads return 0, writes are discarded - even when a load targets **x0**. Instructions that use **x0** as an operand stay **legal**. **2. Why?** Databook **register_file.rst.txt**: "Register **x0** is statically bound to 0 and can only be read, it does not contain any sequential logic." **3. Required behaviour: x0 == 0** after every retired instruction; **ADD x0,rs1,rs2** writes nothing; **LW x0, ...** performs the load but writes nothing; **JAL x0, off** jumps but does not link. **4. Stimulus:** bias register selection to force **rd = x0** and **rs1/rs2 = x0** across all classes. **5. DUT behaviour: cv32e40p_register_file_ff.sv** (**mem[0] <= 32'b0**); the forwarding logic excludes address 0 (**reg_d * is_reg* = ... && (regfile_addr_ra_id != '0)**). **6. Observe: x0** in the retire stream / register-file state. **7. PASS: sb_x0_zero; x0 usage_cg** (**x0** as **rd**, **rs1**, **rs2** per class). **8. FAIL: x0 != 0** at any point; **JAL x0** creating a link; an undefined encoding becoming legal because it uses **x0**.

8.5.5 risc_alu_00 - integer ops in 1 cycle + EX->ID forwarding

1. What is it? All integer computational instructions produce their result in **1 cycle**, and a back-to-back dependent instruction pays **0 cycles** thanks to forwarding. **2. Why?** Databook **pipeline.rst.txt**: "Integer Computational | 1" and "forwarding path ... allows to have 0-cycle penalty between those instructions and immediately following one when using result". **3. Required behaviour:** result correct; dependent consumer at distance 1 reads the producer's value with no stall; **SLT/SLTU** use signed/unsigned compare; **SLTIU compares rs1 unsigned against the sign-extended immediate** (a classic trap - the immediate is sign-extended first, then treated as unsigned). **4. Stimulus: riscv_hazard_sequence (alu chain mode)** - producer/consumer overlap on **rs1** only, **rs2** only, both, with **x0** mixed in. **5. DUT behaviour: cv32e40p_controller.sv** forwarding muxes (**SEL_FW_EX**), **cv32e40p_alu.sv** (**ALU_SLTS**, **ALU_SLTU**). **6. Observe:** results and cycle counts. **7. PASS: sb_rv32i_result_match** on dense dependent chains; **alu_operands_cg** (corner sets incl. signed/unsigned extremes). **8. FAIL:** a consumer reading a stale value (forwarding bug), wrong signed/unsigned comparison, or a result that takes 2 cycles.

8.5.6 risc_alu_01 - shifts

1. What is it? A **barrel shifter** (a combinational shifter that can shift by any amount in one cycle). The shift amount is **operand b[4:0]** (register form) or **imm[4:0]** (immediate form) - i.e. only the low 5 bits count. **SRA/SRAI** replicate the sign bit; **SLL/SRL** shift in zeros; **shamt = 0** leaves the operand unchanged. **2. Why?** RISC-V spec: for RV32 the shift amount is taken mod 32. **3. Required behaviour:** as above for **shamt** in {0,1,15,16,30,31} plus random, on operands 1, MSB-set, **0x80000000, 0xFFFFFFFF, 0xAAAAAAAA**. **4. Stimulus: riscv_shift_sequence**. **5. DUT behaviour: cv32e40p_alu.sv** - **shift_amt_int** is used as **shift_right result = shift_op a 32 >> shift_amt_int[4:0]**; (the [4:0] slice is the "mod 32" rule) and **shift_arithmetic = (operator_i == ALU_SRA)**. **6. Observe: rd** values. **7. PASS: sb_shift_result_match; shift_amount_cg** (**shamt x op x MSB pattern**). **8. FAIL:** **SRA** shifting in zeros, **SRL** replicating the sign, or **shamt** using more than 5 bits (e.g. **rs2 = 32** shifting by 32 instead of 0).

8.5.7 risc_alu_02 - LUI / AUIPC

1. What is it? **LUI** writes **imm[31:12] << 12** (the low 12 bits are zero). **AUIPC** writes **PC + (imm << 12)** - **position dependent**, i.e. the result depends on where the instruction sits. **2. Why?** These are the two ways RISC-V builds 32-bit constants and PC-relative addresses. **3. Required behaviour:** as above; **AUIPC** must be checked at **different PC values** because the result depends on the PC. **4. Stimulus: riscv_upper_sequence** - immediate corners (0, 1, **0x800000**, **0xFFFFF**, walking patterns), **AUIPC** placed at varied PCs including right after taken branches/jumps. **5. DUT behaviour: cv32e40p_decoder.sv** (**OPCODE_LUI: OP_A_IMM/IMMA_ZERO + OP_B_IMM/IMMB_U**; **OPCODE_AUIPC: OP_A_CURRPC + IMMB_U**) and **cv32e40p_id_stage.sv** (**imm_u_type = {instr[31:12], 12'b0}**). **6. Observe: rd** values and the PC at which each **AUIPC** retired. **7. PASS: sb_upper_result_match; upper_imm_cg**. **8. FAIL:** using the wrong PC (e.g. **PC+4**), or a wrong immediate field.

8.5.8 risc_m_00 - MUL is single cycle

1. What is it? **MUL** returns the **lower 32 bits** of the 32x32 product in 1 cycle. **2. Why?** Databook: "CV32E40P uses a single-cycle 32-bit x 32-bit multiplier with a 32-bit result." **3. Required behaviour:** result = lower 32 bits; 1 cycle; **EX** not stalled. **4. Stimulus: riscv_mul_sequence** with corner operands (0, 1, -1, **INT_MIN**, **INT_MAX**, random 16x16 patterns stressing partial products). **5. DUT behaviour: cv32e40p_mult.sv** **int_result = op_c + op_b_msu + op_a_msu * op_b** (with **op_c = 0** and **int_is_msu = 0** for **MUL**). **6. Observe: rd**, cycle count. **7. PASS: sb_mul_result_match; m_operands_cg**. **8. FAIL:** a wrong lower half, or a result that takes more than 1 cycle.

8.5.9 risc_m_01 - MULH / MULHSU / MULHU take 5 cycles

1. What is it? These return the **upper 32 bits** of the product: **MULH** (signed x signed), **MULHSU** (signed x unsigned), **MULHU** (unsigned x unsigned). They take **5 cycles** and stall EX. **2. Why?** Databook: "5 (mulh, mulhsu, mulhu)". RTL:

the FSM **IDLE_MULT** -> **STEP0** -> **STEP1** -> **STEP2** -> **FINISH** computes the 64-bit product as three 16x16 partial products across the steps (one 32-bit multiplier is re-used). **3. Required behaviour:** correct upper half for each sign mode; EX stalled meanwhile; **no other write-back during the stall window**. **4. Stimulus:** **riscv_mul_sequence** (**mulh mode**) - sign-cornor matrix: (**INT_MIN**, -1), (**INT_MAX**, **INT_MAX**), (**0x80000000**, 1), (**0xFFFFFFFF**, **0xFFFFFFFF**), zero operands; plus a dependent consumer immediately after. **5. DUT behaviour:** **cv32e40p_mult.sv** FSM (**multicycle o** stalls EX), **cv32e40p_id_stage.sv** (**mult_multicycle i** keeps the operands). **6. Observe:** **rd**, the cycle count, and the number of register writes (must be exactly one). **7. PASS:** **sb_mulh_result_match** + **sb_no_spurious_wb** (via **rvfi_monitor**: one write per retire); **mulh_corners_cg**, **multicycle_hazard_cg**. **8. FAIL:** wrong sign mode, wrong upper half, extra/missing write-back, or a latency != 5.

8.5.10 risc_m_02 - DIV/DIVU/REM/REMU: iterative, 3..35 cycles

1. What is it? The divider is **iterative with early termination**: the latency depends on the number of **leading zeros of the divisor**. Minimum **3** cycles (divisor **0x80000000**: no leading zeros), maximum **35** (divisor = 0). EX stalls meanwhile. **2. Why?** Databook cycle table (section 4.3). RTL: **cv32e40p_alu_div.sv** loads **Cnt = div_shift**, where **div_shift** is the leading-zero count derived from **cv32e40p_ff_one** (section 5.7). **3. Required behaviour:** correct quotient/remainder; a measured retire latency inside 3..35 at zero wait states; EX stalled meanwhile. **4. Stimulus:** **riscv_div_sequence** sweeping the divisor's leading-zero count over {0, 1, 8, 16, 31, 32 (= divisor 0)} plus random operands. **5. DUT behaviour:** **cv32e40p_alu.sv** (**div_shift**, **ff_one i**) + **cv32e40p_alu_div.sv** (**IDLE** -> **DIVIDE** -> **FINISH**); **alu_ready** (= **div_ready**) participates in **ex_ready o**. **6. Observe:** the result and the issue->retire latency. **7. PASS:** **sb_div_result_match** + **sb_div_latency_range**; **div_latency_cg** bins {3, 4-10, 11-20, 21-34, 35} per op. **8. FAIL:** a wrong result or a latency outside 3..35 (a latency bug is a real bug: it breaks the pipeline's stall logic and any software timing assumptions).

8.5.11 risc_m_03 - division corner cases

1. What is it? The RISC-V-defined special results: * divide by zero: **DIV** and **DIVU** -> quotient = -1 (**0xFFFFFFFF**); **REM** and **REMU** -> remainder = dividend; * signed overflow **DIV** (-2³¹)/(-1) -> quotient = -2³¹ (**0x80000000**); **REM** (-2³¹)/(-1) -> 0; * the sign of **REM** follows the **dividend**; **REMU** is unsigned. **2. Why?** These are mandated by the RISC-V spec ([standard/riscv-spec-20191213.pdf](#)) and are the classic places where a hand-written divider is wrong. **3. Required behaviour:** as tabulated. **4. Stimulus:** **riscv_div_corner_sequence** - a directed matrix {dividend in 0, 1, -1, **INT_MIN**, **INT_MAX**, random} x {divisor in 0, 1, -1, **INT_MIN**, **INT_MAX**} for all four ops. **5. DUT behaviour:** **cv32e40p_alu_div.sv** (**OpBisZero SI**, **ResInv SP**, **PmSel S**, **RemSel SP** - the corner handling is entirely in this file's sign/invert logic). **6. Observe:** **rd** at retirement. **7. PASS:** **sb_div_corner_results**; **div_corners_cg** (all corner cells including div0 and overflow). **8. FAIL:** e.g. -2³¹ / -1 returning **0x7FFFFFFF**, or **REM x, 0** returning 0 instead of the dividend.

8.5.12 risc_m_04 - a multi-cycle instruction older than a taken branch still completes

1. What is it? If a **MULH/DIV/REM** is **older** than a taken branch, it completes and writes back normally even though the flush of the younger path happens in the same window. Only the **younger** stages (IF/ID) are flushed. **2. Why?** Guideline Team 4 Q4 asked exactly this and got **no answer**; the Databook says "Multi-cycle instructions will stall this stage until they are complete" and a flush caused by a younger branch must not kill an older instruction that is already in EX. The RTL confirms it: the multicycle FSM keeps running while **id_ready o** (and hence IF/ID) is stalled, and the branch flush only clears IF/ID. **3. Required behaviour:** 1. The multicycle instruction finishes and writes its result. 2. The taken branch redirects fetch to its target. 3. Instructions between them (younger than the branch) never retire. **4. Stimulus:** **riscv_ctrl_mix_sequence** - **MULH/DIV** followed at distance 1-2 by a taken branch, with and without a RAW dependency between them. **5. DUT behaviour:** **cv32e40p_mult.sv** FSM is clocked by the (gated) clock and is **not** reset by the branch flush; **cv32e40p_controller.sv** handles the branch in **DECODE** while **mult_multicycle_i** keeps ID stalled. **6. Observe:** the multicycle result, the branch target PC stream, and the retire stream. **7. PASS:** **sb_multicycle_plus_branch**; **multicycle_branch_cg** (mulh/div x following taken branch x dependency distance). **8. FAIL:** the multicycle result lost, a wrong branch target, or a younger instruction retiring.

8.6 Family risc_haz_*, risc_rf_* - Hazards and register file

8.6.1 risc_haz_00 - RAW forwarding, 0-cycle penalty at distances 1/2/3

1. What is it? A **RAW** (Read-After-Write) hazard: an instruction reads a register that a previous instruction is about to write. **Forwarding** (bypassing) copies the result from EX back to the ID operand muxes, so the consumer needs **no stall**. **2. Why?** Databook: **pipeline.rst.txt**: "0-cycle penalty between those instructions and immediately following one when using result". **3. Required behaviour:** the consumer sees the correct producer value at dependency distance 1, 2 and 3, in operand slot **rs1**, **rs2** or both, for producers of type ALU, MUL and DIV. **4. Stimulus:** **riscv_hazard_sequence** - dense RAW chains of length >= 4, distances 1/2/3. **5. DUT behaviour:** **cv32e40p_controller.sv** (**SEL_FW_EX** / **SEL_FW_WB** with **reg_d * is reg * detection**), **cv32e40p_id_stage.sv** (operand muxes). **6. Observe:** every consumer result. **7. PASS:** **sb_rv32i_result_match** on the dense chains; **raw_distance_cg** (producer type x distance x slot). **8. FAIL:** a consumer reading the old register value (missing forwarding) or the wrong producer's value.

8.6.2 risc_haz_01 - load-use hazard = exactly 1 stall cycle

1. What is it? When the instruction **immediately** after a load consumes the loaded register, the data is not yet available (it only arrives at the end of WB) -> the pipeline stalls **exactly one cycle**. Consumers at distance >= 2 are **not** stalled. **2. Why?** Databook: "CV32E40P experiences a 1-cycle penalty on the following hazards: Load data hazard in case the instruction immediately following a load uses the result of that load". **3. Required behaviour:** the consumer gets the correct data; exactly one bubble; no spurious write during the stall. **4. Stimulus:** **riscv_hazard_sequence** (**load use mode**) - load -> consumer at distance 1 and 2, across all load types and consumer classes (ALU, branch, JALR, store data). **5. DUT behaviour:** **cv32e40p_controller.sv**: **load_stall o = 1**; **deassert we o = 1**; when (**data_req_ex i && regfile_we_ex i || !wb_ready i && regfile_we_wb i**) && (**reg_d_ex is reg_a i || ...**). **6. Observe:** the consumer result and the retire spacing (1 extra cycle at zero wait). **7. PASS:** **sb_load_use_correct** + **sb_load_use_1cycle**; **load_use_cg** (load type x consumer class x distance). **8. FAIL:** the consumer reading a stale value (too few stalls) or the pipeline stalling 2+ cycles (too many stalls - a performance bug, and it would show in the

latency measurement).

8.6.3 risc_haz_02 - JALR hazard: 1 stall cycle and target (rs1+imm) & ~1

1. What is it? A JALR whose base register **rs1** is produced by the immediately preceding instruction is stalled **1 cycle (jr_stall)** before the target is computed; the target is **(rs1 + imm) & ~1** (bit 0 cleared). **2. Why?** Databook: "Jump register (jalr) data hazard in case that a jalr depends on the result of an immediately preceding instruction". RISC-V spec: the target's LSB is cleared so that the ISA stays compatible with compressed (2-byte) instructions. **3. Required behaviour:** target = **(rs1 + imm) & ~1**; one stall cycle when the producer is the previous instruction; link = **PC + 4**. **4. Stimulus:** **riscv_branch_sequence (jalr hazard mode)** - ALU->JALR and LW->JALR at distances 1/2/3, with **odd** immediates to exercise the **&~1** clearing. **5. DUT behaviour:** **cv32e40p_controller.sv (jr_stall o** condition on **BRANCH JALR)**, **cv32e40p_id_stage.sv (JT JALR: jump_target = regfile_data_ra_id + imm T type)**, **cv32e40p_if_stage.sv (.branch_addr i({branch_addr n[31:1], 1'b0}))**. **6. Observe:** the retired PC stream (the target) and the cycle count. **7. PASS:** **sb_jalr_target_match**; **jr_hazard_cg** (producer type x distance). **8. FAIL:** an odd target, a target computed from a stale **rs1**, or a wrong/missing stall.

8.6.4 risc_haz_03 - WAW: the younger instruction wins

1. What is it? A WAW (Write-After-Write) collision: a load finishing in WB and a younger ALU/MUL/DIV in EX write the same **rd** in the same cycle. The register file resolves it with **port-B (EX) priority**, so the **architecturally younger** instruction wins. **2. Why?** Guideline Team 4 Q6 asked this and got no answer; the RTL answers it (section 5.5): **if (we_b_dec[i]) mem[i] <= wdata_b i; else if (we_a_dec[i]) mem[i] <= wdata_a i**; with port B = EX. **3. Required behaviour:** after such a collision the final **rd** equals the **younger** result. **4. Stimulus:** **riscv_waw_sequence - LW rd** followed at distance 1..3 by an ALU/MUL/DIV writing the same **rd** (including a misaligned-load producer), then read **rd** back. **5. DUT behaviour:** **cv32e40p_register_file_ff.sv** (port priority), **cv32e40p_id_stage.sv** (port wiring). **6. Observe:** the final **rd** value (targeted read-back + golden final-state compare). **7. PASS:** **sb_waw_younger_wins**; **waw_cg** (collision window positions). **8. FAIL:** the final value being the **load's** value (older instruction winning) - a genuine architectural bug, because program order must be respected.

8.6.5 risc_haz_04 - control-flush cleanliness

1. What is it? Instructions younger than a taken branch/jump/trap never produce **architectural side effects**: no register write, no memory request. **2. Why?** Companion to **risc_if_06** and **risc_lsu_08**; it is the general statement of the rule. **3-8. Stimulus:** **riscv_ctrl_mix_sequence / riscv_ctrl_mem_mix_sequence**; checks **sb_flush_clean** (retire stream) + **sb_lsu_trans_stream_match** (bus log). **PASS:** nothing from the flushed path appears in either view. **FAIL:** a register write or a bus transaction from a flushed instruction. **9. RTL:** **cv32e40p_controller.sv (is_decoding_o = 0** on flush, **halt_id_o**, **clear_instr_valid**).

8.6.6 risc_rf_01 - 31 registers, 3 read ports, 2 write ports

1. What is it? **x1..x31** exist and reset to 0 (an **implementation** property of the FF register file, not an ISA requirement - the plan is careful about this); reads from EX and WB. **2. Why?** Databook **register file.rst.txt** + **pipeline.rst.txt**; guideline G5 names the register file as a major block worth a scoreboard. **3. Required behaviour:** every register usable as **rd**, **rs1**, **rs2** in every instruction class; two simultaneous writers are resolved (see 8.6.4). **4. Stimulus:** **riscv_alu_sequence (reg sweep)** - sweep all 32 registers. **5. DUT behaviour:** **cv32e40p_register_file_ff.sv**, **cv32e40p_id_stage.sv**. **6. Observe:** register values in the retire stream and at the end. **7. PASS:** golden compare; **reg_access_cg** (every **x1..x31** as **rd/rs1/rs2** across classes); **sb_rf_reset_zero_opt** - initial reads of unwritten registers return 0 (checked **once at test start**, documented as implementation-specific). **8. FAIL:** any register not writable/readable, or a mismatch.

8.7 Family risc_rst_* - reset in interesting places

All three rows share the same idea, so they are described together with their differences.

1. What is it? Re-asserting reset in the middle of an activity: **risc_rst_00** during an outstanding instruction fetch; **risc_rst_01** during outstanding data transactions (including mid-way through a misaligned pair); **risc_rst_02** during a multi-cycle DIV/REM/MULH or during a branch flush. **2. Why?** A reset must be clean at any time. If it is not, the second "boot" after the reset starts from a corrupted state - a very common class of CPU bug (stale FIFOs, outstanding counters that never return to 0, a divider FSM stuck in **DIVIDE**). **3. Required behaviour:** core and TB slaves both drop the transaction; after re-enabling, the core re-boots from **boot_addr_i** with a clean pipeline; no stale **rvalid** is consumed; no write-back from a killed instruction appears; the data memory image is not corrupted (already-completed stores stay, a half-done misaligned pair never completes). **4. Stimulus:** **sys_ctrl_reset_inject_sequence** synchronized to IF wait-state windows, LSU windows (misaligned pair targeted), DIV iterations, and immediately after a taken branch. **5. DUT behaviour:** **rst_ni** is asynchronous; all pipeline registers in **cv32e40p_if_stage.sv**, **cv32e40p_id_stage.sv**, the LSU **cnt_q**, and the divider FSM have **if (rst_n == 1'b0)** resets. **6. Observe:** the post-reset boot (first fetch == **boot_addr_i**), the retire stream, the final state. **7. PASS:** **sb_reset_resync** (+ **sb_store_mem_match**, **sb_no_spurious_wb**); **reset_context_cg** bins {idle, IF outstanding, LSU outstanding, multicycle DIV, branch flush}. **8. FAIL:** the core not restarting, restarting at the wrong address, a stale transaction completing after reset, a counter that never returns to 0 (visible as a permanently stalled bus), or a write-back from the killed instruction.

8.8 The remaining three sheets, in tables

8.8.1 Generation_ - which stimulus each requirement needs (summary)

ID	Stimulus (Generation cell)	Object
risc_sys_02	reset with random hold and random de-assert phase; mid-test re-assertion	sys_ctrl_init_sequence, sys_ctrl_reset_inject_sequence

ID	Stimulus (Generation cell)	Object
risc_sys_03	randomise the delay before fetch_enable_i (0,1,...,N)	sys_ctrl_init_sequence (fe_delay)
risc_if_00/01/02/03	gnt delayed 0..K; gnt held high with no request; rvalid latency 1..L; mixed zero-wait/random windows	obi_memory_sequence (if channel, gnt_always_high mode)
risc_if_05/06	jumps fwd/back, link x0/x1/x5, JALR with odd immediates; all 6 branches x taken/not-taken x operand relations	riscv_branch_sequence (+ jump mode)
risc_if_08	only 32-bit RV32I/M encodings, word-aligned targets, ends at the sentinel	riscv_program
risc_lsu_00/01	LSU gnt delay 0..K; slow-response windows with a store followed by a load	obi_memory_sequence (lsu channel, stall mode)
risc_lsu_03	lw/sw, lh/sh, lb/lbu/sb at all 4 offsets	riscv_lsu_sequence (offset sweep)
risc_lsu_04	misaligned lw/sw at offsets 1,2,3; lh/sh at offset 3 and 1,2; back-to-back pairs	riscv_lsu_misaligned_sequence
risc_lsu_05	store value corner pool x all offsets	riscv_lsu_sequence (data pool)
risc_lsu_06	loads with MSB 0/1 at every offset	riscv_lsu_sequence (sign pool)
risc_lsu_07	st->ld, ld->st, st->st, ld->ld, same/overlapping addresses	riscv_mem_b2b_sequence
risc_lsu_08	branch/jump/trap immediately followed by loads/stores, incl. during LSU wait states	riscv_ctrl_mem_mix_sequence
risc_dec_00/01	streams per class with random registers/immediates incl. corners	riscv_alu_sequence, riscv_upper_sequence, riscv_mul_sequence, riscv_div_sequence
risc_dec_02	illegal items of each category (a)-(c) with randomised filler bits, injected at random positions	riscv_illegal_sequence
risc_dec_03	bias register selection to force rd = x0, rs1/rs2 = x0	riscv_alu_sequence, riscv_corner_case_sequence (x0 mode)
risc_alu_00/01/02	dependent ALU chains; shifts with shamt corners; LUI/AUIPC at varied PCs	riscv_hazard_sequence (alu chain), riscv_shift_sequence, riscv_upper_sequence
risc_m_00/01/02/03	MUL corners; MULH sign matrix; divisor leading-zero sweep {0,1,8,16,31,32}; directed corner matrix	riscv_mul_sequence, riscv_div_sequence, riscv_div_corner_sequence
risc_m_04	MULH/DIV followed at distance 1-2 by a taken branch	riscv_ctrl_mix_sequence
risc_haz_00/01/02	RAW chains distance 1/2/3; load->consumer pairs; ALU->JALR and LW->JALR	riscv_hazard_sequence (+ modes)
risc_haz_03	LW rd then ALU/MUL/DIV to the same rd at distance 1..3 (+ misaligned variant)	riscv_waw_sequence
risc_rf_01	sweep all 32 registers in every class	riscv_alu_sequence (reg sweep)
risc_rst_00/01/02	reset injected during IF/LSU/div/branch-flush windows	sys_ctrl_reset_inject_sequence + background sequences
risc_env_01	seed sweeping: short/medium/long delay distributions, simultaneous stalls on both buses	obi_memory_sequence

8.8.2 Checking_ - how each requirement is judged (summary)

ID	Check	Property name	Where it lives (plan's own comment)
risc_sys_02	*_req_o == 0 during reset	sys_reset_no_req_prop	SVA in if_passive_agent.output_monitor / lsu_passive_agent.output_monitor

ID	Check	Property name	Where it lives (plan's own comment)
risc_sys_03	no fetch before fetch_enable_i	sys_no_fetch_before_fe_prop	SVA in if_passive_agent.output_mon itor
risc_if_00	request persistence + address stability	if_obj_req_persistence_prop, if_obj_addr_stable_prop	SVA in if_passive_agent.output_mon itor
risc_if_01	#(req >) == #(rvalid)	if_obj_trans_match_prop	if_passive_agent counters, compared at end of test
risc_if_02	one rvalid per grant, in order, rdata stable	if_obj_slave_protocol_prop	assertions inside if_agent (instr_mem_model) - an environment self-check
risc_if_03	fetch stream consistent with retire	sb_fetch_stream_consistent	uvm_scoreboard
risc_if_05	PC flow + link register	sb_pc_flow_match, sb_jump_link_match	uvm_scoreboard (rvfi_monitor + ref_model)
risc_if_06	no side effects from the fall-through path; PC	sb_pc_flow_match, sb_flush_clean	uvm_scoreboard
risc_if_09	continues at the target no trap on legal streams	sb_no_unexpected_trap	uvm_scoreboard
risc_lsu_00	persistence + stability of addr/we/be/wdata	lsu_obj_req_persistence_pro p, lsu_obj_stable_prop	SVA in lsu_passive_agent.output_m onitor
risc_lsu_01	slave protocol + never >2 outstanding	lsu_obj_slave_protocol_prop, lsu_max_outstanding_prop	env assertions in lsu_agent + lsu_passive_agent counter
risc_lsu_03	legal be for (type, address, phase)	lsu_be_legal_prop	SVA/LUT in lsu_passive_agent.output_m onitor
risc_lsu_04	two beats, right addresses/be, no trap, right value	lsu_misaligned_order_prop, sb_load_result_match, sb_no_exc_on_misaligned	SVA + uvm_scoreboard
risc_lsu_05	stored bytes == expected	sb_store_mem_match	uvm_scoreboard vs golden memory model
risc_lsu_06	every retired load vs golden	sb_load_result_match	uvm_scoreboard
risc_lsu_07	store->load read-back	sb_mem_consistency	uvm_scoreboard shadow memory
risc_lsu_08	bus transactions == retired memory ops	sb_lsu_trans_stream_match	uvm_scoreboard (lsu_passive_agent log vs ref_model)
risc_dec_00/01	every retired I/M instruction vs golden	sb_rv32i_result_match, sb_rv32m_result_match	uvm_scoreboard
risc_dec_02	illegal retires to trap, no side effects, fetch -> mtvec base	sb_illegal_trap, exc_trap_base_prop	uvm_scoreboard + SVA
risc_dec_03	x0 destination -> x0 unchanged	sb_x0_zero, sb_pc_flow_match	uvm_scoreboard
risc_alu_00/01/02	results incl. forwarding chains / shamt corners / AUIPC at diverse PCs	sb_rv32i_result_match, sb_shift_result_match, sb_upper_result_match	uvm_scoreboard
risc_m_00/01	MUL / MULH results, single write per retire	sb_mul_result_match, sb_mulh_result_match, sb_no_spurious_wb	uvm_scoreboard (+ rvfi_monitor)
risc_m_02/03	div results and latency 3..35; corner values	sb_div_result_match, sb_div_latency_range, sb_div_corner_results	uvm_scoreboard (latency measured issue->retire at zero wait)
risc_m_04	multicycle result written back + branch path retires	sb_multicycle_plus_branch	uvm_scoreboard
risc_haz_00/01/02/03	consumer results, 1-cycle spacing, JALR target, WAW winner	sb_rv32i_result_match, sb_load_use_correct, sb_load_use_1cycle, sb_jalr_target_match, sb_waw_younger_wins	uvm_scoreboard
risc_haz_04	no side effects from flushed path	sb_flush_clean, sb_lsu_trans_stream_match	uvm_scoreboard
risc_rf_00/01	x0 == 0 after every retire; unwritten registers read 0 at start	sb_x0_zero, sb_rf_reset_zero_opt	uvm_scoreboard / optional hierarchical SVA on RF mem[0]

ID	Check	Property name	Where it lives (plan's own comment)
risc_rst_00/01/02	post-reset re-boot, resync, no corruption, no spurious write-back	sb_reset_resync, sb_store_mem_match, sb_no_spurious_wb	uvm_scoreboard
risc_env_01	slave self-guards	env_slave_protocol_prop	assertions inside if_agent / lsu_agent drivers
risc_env_02	drain + final state + no timeout	sb_final_state_match, env_tearardown_check	uvm_scoreboard

8.8.3 Coverage_ - covergroups (summary)

Group of bins	Covergroup name(s)
Reset contexts (idle / IF outstanding / LSU outstanding / DIV / branch flush)	reset_context_cg
fetch_enable delay {0,1,2..8,>8}	fetch_enable_delay_cg
IF bus: gnt wait states, gnt-before-req, outstanding depth, rvalid latency, back-to-back	if_obi_wait_states_cg, if_gnt_before_req_cg, if_obi_outstanding_cg, if_obi_rvalid_latency_cg, if_obi_back2back_cg
Jumps: direction x link x sign x consecutive	jump_cg
Branches: branch x taken/not-taken x direction x operand relation x target-wait cross	branch_taken_cg, branch_offset_cg, branch_operands_cg, branch_target_wait_cross_cg
LSU bus: gnt wait states, outstanding depth, rvalid latency, stall mid-pair	lsu_obi_wait_states_cg, lsu_outstanding_cg, lsu_obi_rvalid_latency_cg, lsu_stall_mid_pair_cg
LSU data: be patterns, misaligned cases x wait cross, wdata corners	lsu_be_pattern_cg, lsu_misaligned_cg, misaligned_wait_cross_cg, lsu_wdata_corners_cg
Loads: sign extension	load_sign_ext_cg
Memory: back-to-back patterns	mem_b2b_cg
Control x memory mixes	ctrl_mem_mix_cg
Instruction coverage: every RV32I / RV32M mnemonic	rv32i_instr_cg, rv32m_instr_cg
Illegal categories and contexts	illegal_category_cg, illegal_context_cg
Operand corners: ALU, shift amounts, upper immediates, MUL/MULH	alu_operands_cg, shift_amount_cg, upper_imm_cg, m_operands_cg, mulh_corners_cg
Multi-cycle: div latency bins {3,4-10,11-20,21-34,35}, div corners, multicycle hazard, multicycle+branch	div_latency_cg, div_corners_cg, multicycle_hazard_cg, multicycle_branch_cg
Hazards: RAW distance, load-use, JALR, WAW	raw_distance_cg, load_use_cg, jr_hazard_cg, waw_cg
Registers: every x1..x31 as rd/rs1/rs2; x0 usage	reg_access_cg, x0_usage_cg
Dual-bus pressure	dual_bus_stress_cg

8.8.4 test list - the 27 tests

Test	Sequence	Purpose (in one line)
risc_reset_test	sys_ctrl_init_sequence	reset/boot flow, output inactivity, corner boot_addr_i/mtvec_addr_i, a second reset mid-idle
risc_fetch_enable_stress_test	sys_ctrl_init_sequence (fe_delay)	sweep fetch-enable delays incl. >8 cycles
risc_alu_r_type_test	riscv_alu_sequence	the 10 R-type ops, ~2000 instr x 10 seeds
risc_alu_i_type_test	riscv_alu_sequence	I-type ALU ops with corner immediates
risc_shift_test	riscv_shift_sequence	shamt corners x operand MSB patterns
risc_upper_test	riscv_upper_sequence	LUI/AUIPC at diverse PCs
risc_load_store_aligned_test	riscv_lsu_sequence	all aligned loads/stores, zero-wait
risc_load_sign_test	riscv_lsu_sequence (sign pool)	sign vs zero extension at every offset
risc_misaligned_test	riscv_lsu_misaligned_sequence	the two-beat split, order, be patterns, merged data
risc_mem_b2b_test	riscv_mem_b2b_sequence	shadow-memory consistency
risc_branch_test	riscv_branch_sequence	all 6 conditions, taken/not-taken, extremes
risc_jump_test	riscv_branch_sequence (jump)	JAL/JALR, link registers, &~1 rule
risc_jalr_hazard_test	riscv_branch_sequence (jalr hazard)	ALU/LW -> JALR at distance 1/2/3
risc_mul_test	riscv_mul_sequence	MUL/MULH/MULHSU/MULHU sign matrix
risc_div_test	riscv_div_sequence	divisor leading-zero sweep (latency 3..35)

Test	Sequence	Purpose (in one line)
risc_div_corner_test	riscv_div_corner_sequence	div-by-zero, overflow, +/-1
risc_illegal_instr_test	riscv_illegal_sequence	illegal categories (a)-(c), trap redirect
risc_illegal_context_test	riscv_illegal_sequence (context)	illegal back-to-back, during wait states, after control transfer
risc_hazard_raw_test	riscv_hazard_sequence	dense RAW chains
risc_load_use_test	riscv_hazard_sequence (load_use)	load->consumer distance 1 and 2
risc_waw_test	riscv_waw_sequence	LW then ALU/MUL/DIV to the same rd
risc_multicycle_branch_test	riscv_ctrl_mix_sequence	MULH/DIV + taken branch
risc_obi_waitstate_test	obi_memory_sequence	randomised gnt/rvalid on both buses
risc_obi_stall_stress_test	obi_memory_sequence (stall)	saturate 2-outstanding on both buses
risc_gnt_always_high_test	obi_memory_sequence	gnt held high, incl. with no request
risc_reset_inject_test	sys_ctrl_reset_inject_sequence	reset during IF/LSU/misaligned/div/branch-flush
risc_random_mix_test	riscv_random_mix_sequence	the main regression: random RV32IM (~2% illegals) + random OBI delays, ~5000 instr x 30 seeds

9. Requirement -> Verification Mapping

Legend for **Source**: **DB** = Databook chapter, **GL** = guideline (section 6), **ORP** = ORPLAN. **Status**: [OK] covered by the architecture - [!] covered with a caveat (see 9.2) - [?] needs clarification.

#	Requirement (short)	Source	RTL location	Verification item (ORPLAN ID)	TB component	Check	Status
1	DUT = DB cv32e40p_to p, default params	DB integration; GL G5; ORP risc_sys_00	cv32e40p_to p.sv	risc_sys_00	top_tb	elaboration + all later checks	[OK]
2	All ports connected/observed/tied, no X	DB integration/sl eep/exc; GL G1; ORP risc_sys_01	port list of cv32e40p_to p.sv	risc_sys_01	top_tb + sys_ctrl_agent + new oos_ports	\$isunknown() SVA on all inputs	[OK]
3	rst_ni async, outputs inactive, clock gated	DB sleep; ORP risc_sys_02	cv32e40p_slep_unit.sv, cv32e40p_controller.sv (RESET)	risc_sys_02	sys_ctrl_agent + IF/LSU passive monitors	sys_reset_no_req_prop	[OK]
4	fetch_enable_i gating, sticky, static inputs stable	DB integration; ORP risc_sys_03	cv32e40p_slep_unit.sv (fetch_enable_q), cv32e40p_if_stage.sv (PC_BOOT)	risc_sys_03	sys_ctrl_agent + IF passive monitor	sys_no_fetch_before_fe_prop + first PC == boot_addr_i	[OK]
5	IF request persistence / address stability	DB intro+IF; GL G4; ORP risc_if_00	cv32e40p_obi_interface.sv (FSM, TRANS_STABLE=0)	risc_if_00	if_passive_agent.output_monitor	if_obi_req_persistence_prop, if_obi_addr_stable_prop	[OK]
6	No rvalid->req path; gnt before request legal	DB intro (history); ORP risc_if_01	cv32e40p_core.sv (PULP_OBI=0), cv32e40p_pre_fetch_controller.sv	risc_if_01	if_agent + if_passive_agent counters	if_obi_trans_match_prop	[OK]

#	Requirement (short)	Source	RTL location	Verification item (ORPLAN ID)	TB component	Check	Status
7	IF response phase: 1 rvalid per grant, in order, <=2 outstanding	DB IF; ORP risc_if_02	cv32e40p_prefetch_controller.sv (cnt_q), cv32e40p_prefetch_buffer.sv (FIFO_DEPTH=2)	risc_if_02	if_agent (instr_mem_model)	if_obi_slave_protocol_prop	[OK]
8	Back-to-back fetches	DB IF/protocol; ORP risc_if_03	cv32e40p_obi_interface.sv (trans_ready_o)	risc_if_03	if_passive_agent + scoreboard	sb_fetch_stream_consistent	[OK]
9	Speculative prefetch up to 2 words, no side effects	DB IF; ORP risc_if_04	cv32e40p_prefetch_buffer.sv	risc_if_04	new instr_mem_model	memory serves any address	[OK]
10	Flush on taken jump (2 cycles), link = PC+4, &~1	DB pipeline; ORP risc_if_05	cv32e40p_controller.sv (PC_JUMP), cv32e40p_id_stage.sv, cv32e40p_if_stage.sv cv32e40p_ex_stage.sv (branch_decision_o), controller DECODE	risc_if_05	rvfi_monitor + ref_model + scoreboard	sb_pc_flow_match, sb_jump_link_match	[OK]
11	Flush on taken branch (3 cycles), not-taken 1	DB pipeline; ORP risc_if_06	cv32e40p_ex_stage.sv (branch_decision_o), controller DECODE	risc_if_06	scoreboard	sb_pc_flow_match, sb_flush_clean	[OK]
12	32-bit word-aligned bounded programs only	GL Team2 Q1; ORP risc_if_08	n/a (stimulus)	risc_if_08	new riscv_program	build-time constraint	[OK]
13	No instruction-address-misaligned trap exists	DB IF ("Misaligned Accesses"); ORP risc_if_09	cv32e40p_prefetch_buffer.sv (p_instr_addr_word_aligned)	risc_if_09	scoreboard	sb_no_unexpected_trap	[OK]
14	LSU request handshake + stability	DB LSU; GL G4; ORP risc_lsu_00	cv32e40p_load_store_unit.sv (trans_*), obi_interface (TRANS_STABLE=1)	risc_lsu_00	lsu_passive_agent.output_monitor	lsu_obi_req_persistence_prop, lsu_obi_stable_prop	[OK]
15	LSU response phase, <=2 outstanding, back-pressure	DB LSU; ORP risc_lsu_01	cv32e40p_load_store_unit.sv (DEPTH=2, lsu_ready_ex_o)	risc_lsu_01	lsu_agent + lsu_passive_agent	lsu_obi_slave_protocol_prop, lsu_max_outstanding_prop	[OK]
16	Cycle cost 1 (aligned) / 2 (split)	DB pipeline; ORP risc_lsu_02	cv32e40p_load_store_unit.sv (data_misaligned_o)	risc_lsu_02	lsu_passive_agent + scoreboard	transaction-count per retire	[OK]
17	Byte-enable legality table	DB LSU; ORP risc_lsu_03	cv32e40p_load_store_unit.sv (BE always_comb)	risc_lsu_03	lsu_passive_agent.output_monitor	lsu_be_legal_prop	[OK]

#	Requirement (short)	Source	RTL location	Verification item (ORPLAN ID)	TB component	Check	Status
18	Misaligned split into 2 transactions, no exception	DB LSU; GL G6; ORP risc_lsu_04	cv32e40p_load_store_unit.sv + cv32e40p_id_stage.sv (operand_b<=4)	risc_lsu_04	lsu_passive_agent + scoreboard	lsu_misaligned_order_prop, sb_no_exc_on_misaligned	[OK]
19	Store data rotation	ORP risc_lsu_05	cv32e40p_load_store_unit.sv (wdata_offset)	risc_lsu_05	data_mem_model + scoreboard	sb_store_memory_match	[OK]
20	Load alignment sign extension / reassembly	ORP risc_lsu_06	cv32e40p_load_store_unit.sv (rdata*_ext, rdata_q), decoder bit 14	risc_lsu_06	scoreboard	sb_load_result_match	[OK]
21	Memory consistency (store->load) Flushed instructions	ORP risc_lsu_07	n/a (ordering property)	risc_lsu_07	scoreboard shadow memory	sb_mem_consistency	[OK]
22	issue no bus request	ORP risc_lsu_08	cv32e40p_controller.sv (is_decoding_o=0 on flush)	risc_lsu_08	lsu_passive_agent + scoreboard	sb_lsu_transaction_match	[OK]
23	No error response exists / generated	DB LSU (no err); ORP risc_lsu_09	re.sv (.data_err_i(1'b0)), a_no_error	risc_lsu_09	lsu_agent	constraint (no error stimulus)	[OK]
24	All RV32I instructions correct	GL G1/G10; ORP risc_dec_00	cv32e40p_decoder.sv, cv32e40p_alu.sv, LSU	risc_dec_00	ref_model + rvfi_monitor + scoreboard	sb_rv32i_result_match	[OK]
25	All RV32M instructions correct	GL G1/G10; ORP risc_dec_01	cv32e40p_mult.sv, cv32e40p_alu_div.sv	risc_dec_01	same	sb_rv32m_result_match	[OK]
26	Illegal-instruction detection (a)(b)(c)	DB exceptions; GL G6; ORP risc_dec_02	cv32e40p_decoder.sv, controller FLUSH_EX/WB, cv32e40p_if_stage.sv	risc_dec_02	scoreboard + SVA	sb_illegal_trap, exc_trap_base_prop	[!] (guideline Q10 unanswered) [?]
27	x0 operands stay legal	GL G10; DB regfile; ORP risc_dec_03	cv32e40p_decoder (no x0 special-casing except forwarding)	risc_dec_03	scoreboard	sb_x0_zero, sb_pc_flow_match	[OK]
28	Integer ops 1 cycle + forwarding	DB pipeline; ORP risc_alu_00	cv32e40p_controller.sv (SEL_FW_EX), cv32e40p_alu.sv	risc_alu_00	scoreboard	sb_rv32i_result_match	[OK]
29	Barrel shifter, shamt = operand_b[4:0], SRA sign	ORP risc_alu_01	cv32e40p_alu.sv (shift_amt_int[4:0])	risc_alu_01	scoreboard	sb_shift_result_match	[OK]

#	Requirement (short)	Source	RTL location	Verification item (ORPLAN ID)	TB component	Check	Status
30	LUI / AUIPC (PC-dependent)	ORP risc_alu_02	cv32e40p_decoder.sv (IMMB_U, OP_A_CURRPC)	risc_alu_02	scoreboard	sb_upper_result_match	[OK]
31	MUL = 1 cycle	DB pipeline; ORP risc_m_00	cv32e40p_mult.sv (int_result)	risc_m_00	scoreboard	sb_mul_result_match	[OK]
32	MULH/MULHSU/MULHU = 5 cycles + sign modes	DB pipeline; ORP risc_m_01	cv32e40p_mult.sv (FSM STEP0->FINISH)	risc_m_01	scoreboard + rvfi_monitor	sb_mulh_result_match, sb_no_spurious_wb	[OK]
33	DIV/DIVU/REM/REMU 3..35 cycles	DB pipeline; ORP risc_m_02	cv32e40p_alu_div.sv, cv32e40p_alu.sv (div_shift), cv32e40p_ff_one.sv	risc_m_02	scoreboard	sb_div_result_match, sb_div_latency_range	[OK]
34	Division corner results (div0, overflow)	RISC-V spec; ORP risc_m_03	cv32e40p_alu_div.sv (OpBIsZero_Sl, inversion logic)	risc_m_03	scoreboard	sb_div_corner_results	[OK]
35	Multi-cycle op older than a branch completes	ORP risc_m_04 (+ unanswered GL Team4 Q4)	cv32e40p_mult.sv FSM not reset by flush	risc_m_04	scoreboard	sb_multicycle_plus_branch	[!] (behaviour derived from RTL, not from an answered guideline) [?]
36	RAW forwarding 0-cycle penalty	DB pipeline; ORP risc_haz_00	cv32e40p_controller.sv forwarding muxes	risc_haz_00	scoreboard (dense chains)	sb_rv32i_result_match	[OK]
37	Load-use = exactly 1 stall	DB pipeline; ORP risc_haz_01	cv32e40p_controller.sv (load_stall_o)	risc_haz_01	scoreboard	sb_load_use_correct, sb_load_use_1cycle	[OK]
38	JALR hazard + (rs1+imm)&~1	DB pipeline; ORP risc_haz_02	controller (jr_stall_o), cv32e40p_if_stage.sv (bit 0 cleared)	risc_haz_02	scoreboard	sb_jalr_target_match	[OK]
39	WAW: younger (EX, port B) wins	ORP risc_haz_03 (+ unanswered GL Team4 Q6)	cv32e40p_register_file_ff.sv (we_b checked first)	risc_haz_03	scoreboard	sb_waw_younger_wins	[!] (priority taken from RTL) [?]
40	No side effects from flushed path	ORP risc_haz_04	controller (is_decoding_o, halt_id_o, clear_instr_valid)	risc_haz_04	scoreboard	sb_flush_clean, sb_lsu_transaction_match	[OK]
41	x0 hardwired to 0	DB regfile; ORP risc_rf_00	cv32e40p_register_file_ff.sv (mem[0]), forwarding excludes 0	risc_rf_00	rvfi_monitor + scoreboard	sb_x0_zero	[OK]
42	31 registers, 3R/2W, reset 0	DB regfile; GL G5; ORP risc_rf_01	cv32e40p_register_file_ff.sv, cv32e40p_id_stage.sv	risc_rf_01	scoreboard	golden compare + sb_rf_reset_zero_opt	[OK]

#	Requirement (short)	Source	RTL location	Verification item (ORPLAN ID)	TB component	Check	Status
43	Reset during an outstanding fetch	ORP risc_rst_00	cv32e40p_if_stage.sv, cv32e40p_pre_fetch_controller.sv resets	risc_rst_00	sys_ctrl_agent + if_agent + scoreboard	sb_reset_resync	[OK]
44	Reset during outstanding data (incl. mid misaligned pair)	ORP risc_rst_01	cv32e40p_load_store_unit.sv (cnt_q reset)	risc_rst_01	sys_ctrl_agent + lsu_agent + scoreboard	sb_reset_resync, sb_store_memory_match	[OK]
45	Reset during multicycle op / branch flush	ORP risc_rst_02	cv32e40p_alu_div.sv, cv32e40p_mult.sv resets	risc_rst_02	sys_ctrl_agent + scoreboard	sb_reset_resync, sb_no_spurious_wb	[OK]
46	TB owns instruction + data memories, deterministic pre-load OBI gnt/rvalid delays + 4 modes per bus	GL G3; ORP risc_env_00	n/a (TB)	risc_env_00	new_instr_mem_model / data_mem_model	sb_final_state_match	[OK]
47	Teardown: drain, complete retire stream, final state, watchdog	GL G4; ORP risc_env_01	n/a (TB)	risc_env_01	if_agent / lsu_agent drivers	env_slave_protocol_prop	[OK]
48		ORP risc_env_02	n/a (TB)	risc_env_02	scoreboard + top_tb + new watchdog	sb_final_state_match, env_teardown_check	[OK]
49	Out-of-scope features stay inactive / ungenerated	GL G1/G6/G8/G9/G10, Team2 Q1; ORP risc_scope_00	cv32e40p_aligner.sv, cv32e40p_compressed_decoder.sv, cv32e40p_cs_registers.sv, cv32e40p_int_controller.sv, ...	risc_scope_00	riscv_program constraints + tie-offs	build-time tie-off SVA	[OK]

9.2 Notes on the three [!] rows

- **Row 26 (illegal instruction).** In scope by reasoning, not by an explicit answer: the guideline excludes **ECALL/EBREAK/hints**, not undecodable encodings; the Databook makes illegal-instruction detection always active; and the ORPLAN includes it. Ask the supervisor to confirm (Team 2 Q10).
- **Row 35 (multicycle + branch flush).** The behaviour required by ORPLAN was derived from the RTL (the multiplier/divider FSMs are not reset by a flush), because the guideline question (Team 4 Q4) was not answered. If the architect intended the opposite, both the ORPLAN row and **risc_multicycle_branch_test** must change.
- **Row 39 (WAW priority).** "Port B (EX) wins" is a **fact of the RTL**, not of the Databook (the Databook does not document write-port priority - the guideline Team 4 Q6 confirms the plan authors could not find it either). The check encodes the RTL behaviour; if the architect says the older instruction should win, the RTL has a bug.

10. Testbench Architecture (existing diagram, explained)

File: **architecture/uvm_architecture_pro_updated.drawio** (draw.io XML, one diagram named "Professional UVM Architecture"; 102 **mxCell** elements = 100 user cells + the two default root cells).

10.1 The visual language (what colours and shapes mean)

Visual	Meaning	Examples
Grey swimlane #f5f5f5	the test / top_tb layer (static part of the TB)	test
Purple swimlane #e1d5e7	the UVM environment (uvm_env)	env
Blue swimlane #dae8fc	an agent (a container grouping sequencer + driver + monitor)	rst_agt, if_agt, lsu_agt, if_pas, lsu_pas, rvfi_pas
White rounded box	a UVM component inside an agent or the env	Sequencer, Driver, Input Monitor, Output Monitor, env_cfg
Green rounded box #d5e8d4	a subscriber (receives transactions, computes something)	Reference Model (Subscriber), Coverage (Subscriber)
Yellow rounded box #fff2cc	a checker	uvm_scoreboard, Assertions (SVA ...), Watchdog Timer
Red rounded box #f8cecc	the DUT	Design Under Test (CV32E40P)
Orange dashed box #ffe6cc	a virtual interface (the pin bundle)	IF/LSU/System Control/RVFI Virtual Interface
(red) red circle	an export / imp - the receiving end of an analysis connection	ref_exp_*, sb_imp_*, cov_exp*
(green) green diamond	an analysis port - the publishing end	*_ap
(orange) orange square	a driver's sequencer port	*_drv_port
Blue dashed arrow	an analysis (TLM) connection: monitor -> subscriber	e_ifp_sb, e_rvfip_sb, ...
Black solid arrow	a structural connection: sequencer->driver, component->interface, interface->DUT	e_if_agt_sqr_drv, e_ifdrv_intf, e_intfif_dut
Black dashed arrow	an auxiliary/bind relationship	e_sva_dut, e_oos_dut

10.2 The layers, top to bottom

Layer 1 - Test (top_tb) (the grey swimlane, absolute rect ~ 20,30 -> 1530,1090): * **env_cfg (Configuration)** - the knobs. * **Virtual Sequences** (ellipse = start/end shape) - cross-agent scenarios. * **riscv_program (Program Builder)** (added) - builds the instruction stream. * **Sequence Library (riscv_*_sequence, obi_memory_sequence)** (added) - where the 19 named sequences live. * **env** - the whole UVM environment (purple swimlane).

Layer 2 - the environment (purple, ~ 40,180 -> 1500,1070): * **Virtual Sequencer (v_sqr)** - holds handles to the three sub-sequencers; the virtual sequences start the sequences through it (**e_vsqr_rst, e_vsqr_lsu, e_vsqr_if**). * **System Control Agent (Active)** (top-left): Sequencer -> Driver -> **System Control Interface**; its Input Monitor publishes reset/enable events to the **reference model** (so the model can resync) and (added) to **coverage (reset_context_cg, fetch_enable_delay_cg)**. * **IF Agent (Active)** (middle-left) and **LSU Agent (Active)** (middle): Sequencer -> Driver -> their virtual interfaces; their Input Monitors publish the **request** stream to the **reference model** (this is what the model executes). * **IF Agent (Passive)** and **LSU Agent (Passive)** (below them): Output Monitors publish the **response** stream to the **scoreboard** and **coverage**, and host the interface SVA (**if_obi_***, **lsu_be_legal_prop, lsu_misaligned_order_prop, ...**). * **Reference Model (Subscriber)** (centre): consumes IF + LSU request streams (+ reset events) and publishes the **expected** architectural state to the scoreboard. * **uvm_scoreboard** (right of centre): receives expected (ref model), observed (IF/LSU passive monitors + RVFI monitor) and (added) the final data-memory image; owns the shadow memory. * **Coverage (Subscriber)**: receives IF and LSU bus transactions, and (added) the retire stream and the reset context. * **RVFI Monitor (Passive)**: the retirement view; publishes to the scoreboard, (added) to coverage and (added) to the watchdog. * **instr_mem_model (OBI Slave) / data_mem_model (OBI Slave)** (added): bottom-left, under their respective bus columns; feed the active drivers with the content they respond with. * **Watchdog Timer (global timeout)** (added): bottom-centre, under the RVFI monitor.

Layer 3 - the static/DUT layer (below the swimlane): * **IF Virtual Interface, LSU Virtual Interface, System Control Interface, RVFI Internal Interface** (orange). * **Design Under Test (CV32E40P)** (red), fed by all four interfaces. * **Assertions (SVA) bind cv32e40p_top / agent output monitors** (yellow, bound to the DUT). * **Out-of-Scope Ports (tied off / no X-prop)** (added) - the tie-off bundle (**irq_i, debug_req_i, pulp_clock_en_i, scan_cg_en_i, dm_*_addr_i**) feeding the DUT.

10.3 Why the agents are split "Active" and "Passive"

The same physical interface is wrapped twice: * the **active** agent drives the slave side (it answers the core's requests) and its "Input Monitor" watches the **request** direction (what the core asked for) - that is what the reference model needs; * the **passive** agent only watches the **response** direction (what the TB answered and when) - that is what the protocol checkers, the scoreboard and the coverage need. This split is the diagram's way of keeping "what the core asked" separate from "what the memory answered", which is exactly what makes the wait-state checks possible.

10.4 What the architecture does not show (by design)

- No per-module agents/scoreboards - guideline G5 forbids that (top-level only, scoreboard on the datapath).
- No CSR/interrupt/debug/FPU agents - out of scope (**risc_scope_00**); they appear only as tie-offs.
- No third-party reference model - guideline G7.

10.5 A note about the RVFI interface

The Databook's verification chapter says the official environment used **RVFI/RVVI** to observe retirement. **This**

repository's RTL contains no RVFI module (verified: no **rvfi** string anywhere in **rtl/**). Therefore the **RVFI Monitor (Passive)** in the diagram must be implemented by us - either (a) by hierarchical (cross-module) references into the core from a **bind** file instantiated in **top_tb**, or (b) by adding a small, non-synthesizable RVFI wrapper module in the TB area (the core signals needed are: retire_valid, PC, instruction word, **rd**, **rd** value, memory address/data). Option (b) is cleaner and matches the Databook's own description of **cv32e40p_rvfi_trace** ("It is a behavioral, non-synthesizable, module instantiated in the example testbench").

11. Architecture Changes (every modification to the draw.io diagram)

Design rules I followed

1. Nothing was redesigned; no existing block was moved, resized, recoloured or renamed (except the two label changes listed in 11.6-11.7, which are additions of missing information, not restyling).
2. Every new block was placed in previously **empty** space, and every new route was verified programmatically to be orthogonal and not to cross any existing block.
3. The existing visual language was reused: same palette, same shapes, same port symbols (red circle = import, green diamond = analysis port), same two arrow styles (blue dashed = analysis/TLM, solid = structural).
4. Only components that ORPLAN names **and** that were missing were added.

Total: **6 new blocks, 9 new connections, 2 label/documentation edits**. Cells: 82 -> 100. Edges: 28 -> 37.

11.1 Added block - **riscv_program (Program Builder)**

Item	Detail
Where	Test layer, top row, at the right of Virtual Sequences (empty area)
What it is	The generator that assembles the instruction stream and produces the 32-bit words loaded into the instruction memory ORPLAN names it three times: risc_if_08 ("generation constraint (riscv_program)"), risc_env_00 ("if_agent (instr_mem_model), riscv_program") and the Generation sheet ("Enforced inside the program builder: only 32-bit encodings ..."). Without it the diagram had no home for the most important stimulus-generation constraint of the project.
Why added	
Receives	Constraints/knobs from env_cfg (program length, instruction mix, illegal ratio, sentinel address)
Produces	The program image (32-bit words) -> instr_mem_model; the same program is what the reference model will execute
Connects	e_prog_imem: riscv_program -> instr_mem_model (pre-load, before the core is released from reset)

11.2 Added block - **Sequence Library (riscv*_sequence, obi_memory_sequence)**

Item	Detail
Where	Test layer, top row, right of the program builder (empty area)
What it is	The home of the 19 sequences named by ORPLAN (riscv_alu_sequence, riscv_lsu_misaligned_sequence, obi_memory_sequence, sys_ctrl_reset_inject_sequence, ...) The test list sheet maps 27 tests -> 19 sequences, and the Generation sheet names the sequence for every requirement. The generic "Virtual Sequences" ellipse did not show where those objects live.
Why added	
Receives	Nothing
Produces	Transactions started on the sequencers (through the virtual sequencer)

Item	Detail
Connects	e_seqlib_vsqr: Sequence Library -> Virtual Sequencer (v_sqr) - the sequences are started on the sub-sequencers that v_sqr points to (drawn in the same blue-dashed style as the existing vsqr -> agent sequencer arrows)

11.3 Added blocks - **instr_mem_model (OBI Slave)** and **data_mem_model (OBI Slave)**

Item	Detail
Where	Inside env, in the free bottom strip, directly under their own bus columns (instr_mem_model under IF Agent (Active)/(Passive); data_mem_model under LSU Agent (Active)/(Passive))
What they are	Testbench memory models that behave as OBI slaves: they hold the program / the data image and answer the core's requests; the timing of the answers is controlled by the agents' drivers ORPLAN names them in risc_env_00 ("The TB includes its own instruction memory and data memory models working as OBI slaves ...", components "if_agent (instr_mem_model), lsu_agent (data_mem_model)", in risc_if_04, risc_lsu_05, risc_lsu_07 and in risc_env_02 (final dmem == golden). Guideline G3 explicitly puts the memories inside the TB.
Why added	instr_mem_model: the program from riscv_program. data_mem_model: the deterministic data pool from env_cfg
Receive	instr_mem_model: instruction words -> the IF driver. data_mem_model: read data -> the LSU driver; the final memory image -> the scoreboard
Produce	e_prog_imem (in), e_imem_ifdrv / e_dmem_lsudrv (out, to the drivers that answer on the bus), e_dmem_sb (out, end-of-test image compare)
Connect	The bottom strip of the env swimlane was empty, and this position keeps the existing column structure (IF column, LSU column, RVFI column) intact - no existing block had to move
Why there	Blue (#dae8fc / #6c8ebf) - the same colour family as the agents, because ORPLAN places them inside if_agent and lsu_agent
Style	

11.4 Added block - **Watchdog Timer (global timeout)**

Item	Detail
Where	Inside env, in the free bottom strip, directly under the RVFI monitor
What it is	A timer that fails the test if nothing retires for N cycles ORPLAN risc_env_02: "a global watchdog timeout catches any hung core", with check env_teardown_check ("no test timeout") and component top_tb. Without it, a hung core (e.g. waiting for a response the TB never sends) would only be caught by the simulator's global time limit, which is a terrible debugging experience.
Why added	The retire stream from the RVFI monitor (the only place where "progress" is observable)
Receives	A uvm_error / test termination
Produces	e_rvfi_wdog: rvfi_pas_ap -> wdog (short vertical run)
Connects	

Item	Detail
Style	Yellow (#fff2cc / #d6b656, dashed) - same family as the scoreboard and the SVA block, because it is a checker

11.5 Added block - Out-of-Scope Ports (tied off / no X-prop)

Item	Detail
Where	Bottom row, to the left of the DUT (the only free slot in that row)
What it is	The bundle of DUT inputs that are not part of the verification goal but must be driven to a harmless value: irq_i, debug_req_i, pulp_clock_en_i, scan_cg_en_i, dm_halt_addr_i, dm_exception_addr_i
Why added	ORPLAN risc_sys_01 ("Port list ... must be connected and observed while ignoring or tying out of scope ports and no X propagation") and risc_scope_00 ("kept inactive or never generated"). Guideline G1 demands exactly this "minimal work ... to ensure the normal operation" (e.g. driving debug_req_i = 0 so the controller works).
Receives	Static values from env_cfg / top_tb
Produces	Tied signals into the DUT
Connects	e_oos_dut: oos_ports -> dut (dashed, same style as the existing sva -> dut bind arrow)

11.6 Added connections (summary)

#	New edge	Source -> Target	What travels	Why required (ORPLAN)
1	e_prog_imem	riscv_program instr_mem_model	-> the program image (pre-load, before fetch_enable_i)	risc_env_00 "The instruction memory is pre-loaded with a generated or directed program before the core is enabled"
2	e_seqlib_vsqr	Sequence Library v_sqr	-> sequence requests	start test list ("sequence run") + every Generation_row
3	e_imem_ifdrv	instr_mem_model if_agt_drv	-> instruction words + read behaviour	risc_if_04 "the instruction memory model must serve any address without side effects"
4	e_dmem_lsudrv	data_mem_model lsu_agt_drv	-> read data + write behaviour (byte lanes)	risc_lsu_05 "the memory applies data_be_o lanes only"; risc_lsu_07 "the TB memory model commits a store at its rvalid"
5	e_dmem_sb	data_mem_model sb_imp_dmem	-> the final memory image	risc_env_02 sb_final_state_match: "final dmem == golden"

#	New edge	Source -> Target	What travels	Why required (ORPLAN)
6	e_rvfi_cov	rvfi_pas_ap cov_exp3	-> the retire stream	rv32i_instr_cg, rv32m_instr_cg, reg_access_cg, x0_usage_cg, raw_distance_cg, load_use_cg, div_latency_cg, waw_cg - all of these describe retired instructions and are impossible to cover from the bus alone reset_context_cg ("idle / during IF outstanding / LSU outstanding / multicycle DIV / branch flush") and fetch_enable_delay_ cg - only the system-control monitor sees these
7	e_rst_cov	rst_agt_ap cov_exp4	-> reset/enable events and their context	risc_env_02 watchdog
8	e_rvfi_wdog	rvfi_pas_ap -> wdog	retirement progress	risc_sys_01, risc_scope_00
9	e_oos_dut	oos_ports -> dut	tied-off input values	

New port symbols added to existing blocks (needed to terminate connections 5, 6 and 7): **sb_imp_dmem** (left edge of the scoreboard), **cov_exp3** (bottom edge of the coverage block, aligned directly above the RVFI analysis port) and **cov_exp4** (top edge of the coverage block). All three use the existing red-circle import style; the two existing coverage imports were left untouched.

11.7 Label edits on existing blocks (no visual change)

Block	Before	After	Why
test (outer swimlane)	grey (empty title)	Test (top_tb)	ORPLAN risc_sys_00 says "top_tb instantiates cv32e40p_top.sv" and risc_env_02 lists top_tb as a component. An unnamed container made the diagram harder to read for a newcomer. The ORPLAN Checking sheet locates almost every property inside an agent: "SVA inside if_passive_agent.output_monitor / lsu_passive_agent.output_monitor", "SVA/LUT check in lsu_passive_agent.output_monitor", "Assertions inside if_agent / lsu_agent drivers". A module bound only to the top cannot see transaction context (e.g. which access type produced a data_be_o), so the label now states both homes.
sva	Assertions (SVA Module) bind cv32e40p_top	Assertions (SVA) cv32e40p_top / output_monitors	bind agent

11.8 What was deliberately **not** changed

Candidate change	Why it was rejected
Adding CSR / interrupt / debug / FPU agents	Out of scope (risc_scope_00, guidelines G1/G9/G10). The tie-off block covers what is needed.
One agent per module (aligner, decoder, ALU, mult, ...)	Guideline G5: "Focus is top level verification ... not one scoreboard per module".
Splitting the scoreboard into several scoreboards (ALU scoreboard, LSU scoreboard, ...)	Guideline G5 allows "one scoreboard per major group of blocks"; the current single scoreboard with a reference model already covers the datapath, and ORPLAN's property names (sb_*) all prefix to one scoreboard.
Moving cov or intf_rvfi to fix the two pre-existing overlaps	Instruction: preserve the existing layout. They are documented in section 13 instead.
Re-colouring, re-shaping or renaming any block	Instruction: preserve style.
Adding an "instruction-generation" arrow into the reference model	Not required by ORPLAN: the reference model is fed by the IF input monitor (the fetch-address/word stream), which the existing diagram already shows.
Generating a new .jpg export	Only draw.io can produce a faithful export. See section 14.

12. End-to-End Verification Flow (adapted to this repository)

Below is what actually happens, in order, in one simulation of **risc_random_mix_test** (the main regression test). Every step names the component that performs it.

12.1 Build phase

- 1. top_tb** (static module) instantiates **cv32e40p_top** (the DUT), the four interfaces (**IF**, **LSU**, **System Control**, **RVFI Internal**), generates **clk_i**, and publishes the virtual interfaces into the UVM configuration database.
- 2. Test (risc_random_mix_test)** creates and randomises **env_cfg** (zero-wait vs random vs long-stall mode per bus, program length, illegal ratio ~2%, seed).
- 3. Test** builds the **env**: the three active agents, three passive agents, **ref_model**, **uvm_scoreboard**, **cov**, the two memory models, the watchdog.
- 4. sys_ctrl_agent** ties the out-of-scope ports (**oos_ports**) - **irq_i = 0**, **debug_req_i = 0**, **pulp_clock_en_i = 0**, **scan_cg_en_i = 0**, **dm * addr_i = fixed** - and programs **boot_addr_i**, **mtvec_addr_i**, **hart_id_i**.
- 5. riscv_program** builds the program under the constraints of **risc_if_08** (32-bit only, word-aligned targets, bounded, ends at the sentinel) and **pre-loads instr_mem_model**; **data_mem_model** is pre-loaded with a deterministic data pool.
- 6. Reset is applied (sys_ctrl_init_sequence)**: **rst_ni** low, **fetch_enable_i** low, random hold time.
- 7. Reset released**; after a random **fe_delay**, **fetch_enable_i** is raised -> the sleep unit opens the clock gate -> the controller leaves **RESET** -> the first fetch goes to **boot_addr_i**.

12.2 Run phase (per cycle / per transaction)

```

Sequences (riscv_*_sequence, obi_memory_sequence)
|   create transactions (instructions to place; bus-response timing to apply)
v
Virtual Sequencer (v_sqr) -> agent Sequencers
v
Drivers (sys_ctrl / if_agt / lsu_agt)
|   convert transactions into pin activity on the virtual interfaces
v
Virtual Interfaces -> DUT (cv32e40p_top)
|   IF: instr_req_o/addr_o <--> instr_gnt_i/rvalid_i/rdata_i (from instr_mem_model)
|   LSU: data_req_o/addr_o/we_o/be_o/wdata_o <--> data_gnt_i/rvalid_i/rdata_i (from
data_mem_model)
|   SYS: rst_ni / fetch_enable_i / boot_addr_i / mtvec_addr_i (+ tie-offs)
|   v
|   the core fetches -> decodes -> executes -> retires
Monitors
+- if_agt / lsu_agt   Input Monitors   -> what the core ASKED for -> Reference Model
+- if_pas / lsu_pas  Output Monitors  -> what the memory ANSWERED -> Scoreboard + Coverage (+SVA)
+- sys_ctrl          Input Monitor    -> reset/enable context -> Reference Model + Coverage
+- rvfi              Internal Monitor -> what actually RETIRED -> Scoreboard + Coverage + Watchdog
v
Reference Model (ref_model)
|   executes the same instruction stream on a simple, obviously-correct model
|   v
|   expected PC / registers / memory
Scoreboard (uvm_scoreboard)
|   compares expected vs observed, keeps a shadow memory for store->load consistency

```

v
 Coverage (cov) samples the covergroups; Watchdog checks that retirement keeps progressing
 v
 PASS / FAIL

12.3 What each check looks like at the moment of comparison

Check family	Expected comes from	Observed comes from	Compared by
Instruction results, PC flow, register file	ref_model	rvfi_monitor (retire stream)	scoreboard (sb_rv32i_result_match, sb_pc_flow_match, ...)
Loads / stores values	ref_model memory image (byte-accurate)	rvfi_monitor data_mem_model	scoreboard + (sb_load_result_match, sb_store_mem_match, sb_mem_consistency)
Bus protocol rules	the OBI rules (Databook)	if_pas / lsu_pas output monitors	SVA inside the monitors (if_obi_*, lsu_be_legal_prop, lsu_misaligned_order_prop, ...)
TB's own behaviour	the OBI slave rules	the drivers / memory models	SVA inside the drivers (if_obi_slave_protocol_prop, env_slave_protocol_prop)
Timing/latency	Databook cycle table	rvfi_monitor timestamps	scoreboard (sb_div_latency_range, sb_load_use_1cycle)
Liveness	"something must retire every N cycles"	rvfi_monitor	watchdog
End of test	ref_model final state	rvfi_monitor data_mem_model	scoreboard + (sb_final_state_match, env_teardown_check)

12.4 End-of-test phase

1. All outstanding responses are delivered; both buses are checked quiet.
2. The retire stream is checked to be complete (the program reached the sentinel).
3. Final GPRs and the final data-memory image are compared against the golden model.
4. Coverage is reported (all in-scope bins).
5. **uvm_report: PASS** if zero **UVM_ERRORS**; **FAIL** otherwise.

12.5 A concrete example: one misaligned store (putting it all together)

```

riscv_lsu_misaligned_sequence -> "sw x5, 1(x6)"
v
riscv_program -> instr_mem_model (pre-load, before fetch_enable_i)
v
if_agt.driver answers the fetch at PC -> core decodes SW -> EX computes 0x1001
v
cv32e40p_load_store_unit: data_misaligned_o = 1 -> controller asserts misaligned_stall
v
beat 1: data_req_o=1, data_addr_o=0x1001, data_be_o=1110, data_wdata_o rotated by 1
lsu_agt.driver (using data_mem_model) grants after k cycles, returns rvalid after m cycles
data_mem_model writes bytes 1,2,3 of the word
v
cv32e40p_id_stage forces operand_b = 4, prepost_useincr = 1, alu_we = 0
v
beat 2: data_addr_o=0x1004, data_be_o=0001 -> data_mem_model writes byte 0
v
lsu_passive_agent.output_monitor records: 2 transactions, addresses, be, order, timing
SVA lsu_misaligned_order_prop + lsu_be_legal_prop fire checks continuously
v
rvfi_monitor records: 1 retirement (PC, SW, no rd write, memory bytes)
v
ref_model: memory[0x1001..0x1004] = the 4 bytes of x5 (expected)
v
scoreboard: sb_store_mem_match (memory bytes) + sb_lsu_trans_stream_match (transaction count)
+ sb_no_exc_on_misaligned (no trap)
v
coverage: lsu_misaligned_cg[word@1] and misaligned_wait_cross_cg[wait on beat k] are hit
v
PASS (or FAIL with a precise message naming the field that differed)

```

13. Open Questions / Ambiguities

13.1 Questions that only the supervisor/architect can answer

#	Question	Why it matters	My recommendation (based on the repository)
Q1	Illegal-instruction testing: is it in scope? (guideline Team 2 Q10 was left unanswered)	It decides whether <code>riscv_illegal_sequence</code> , <code>risc_dec_02</code> , <code>illegal_category_cg</code> stay	Keep it: it is decoder behaviour of vanilla I&M, the Databook makes it always active, and ORPLAN includes it. Only ECALL/EBREAK/hints are excluded by the guidelines.
Q2	Behaviour of a multi-cycle instruction when a flush arrives (Team 4 Q4)	Defines <code>risc_m_04</code> and <code>risc_multicycle_branch_test</code>	RTL behaviour (older op completes, younger path flushed) is what ORPLAN states - keep it, but get it confirmed.
Q3	Register-file write-port priority (Team 4 Q6)	Defines <code>risc_haz_03</code> (<code>sb_waw_younger_wins</code>)	The RTL gives port B (EX, younger) priority; the Databook is silent. Verify the RTL behaviour and flag any change as an RTL bug, not a TB bug.
Q4	Exact values of <code>boot_addr_i</code> and <code>mtvec_addr_i</code> (ORPLAN's own note on <code>risc_sys_01</code>)	Determines where the program and the trap handler live	Use word-aligned, distinct, non-overlapping values, e.g. <code>boot = 0x0000_0080</code> , <code>mtvec = 0x0000_0100</code> , and keep them constant per test. Note: the Databook says <code>boot_addr_i</code> must be half-word aligned while <code>risc_if_08</code> only guarantees word alignment; the RTL silently clears bits [1:0].
Q5	How should the retire (RVFI) information be obtained?	The RTL ships no RVFI module	Add a small non-synthesizable RVFI wrapper in the TB (the Databook documents <code>cv32e40p_rvfi_trace</code> as exactly such a module) rather than relying on fragile hierarchical paths.

13.2 Ambiguities inside the documents themselves

#	Observation	Impact	Suggested action
Q6	The Databook's <code>register_file.rst.txt</code> says register-file writes happen in WB, while <code>pipeline.rst.txt</code> and the RTL show two write ports (EX and WB)	Could mislead a newcomer writing the scoreboard	Trust <code>pipeline.rst.txt</code> + RTL. (Flagged in section 4.3.)
Q7	Databook <code>integration.rst.txt</code> : <code>boot_addr_i</code> "Must be half-word aligned"; ORPLAN <code>risc_if_08</code> guarantees only word-aligned targets	A half-word-aligned boot address has bit 1 silently cleared by the RTL	Generate word-aligned boot addresses; treat half-word alignment as untested.
Q8	ORPLAN skips ID <code>risc_if_07</code> and has no <code>risc_rf_00</code> row in <code>Generation_/Coverage_</code>	Harmless inconsistency	Add the missing rows or renumber, at the plan owner's discretion.

#	Observation	Impact	Suggested action
Q9	risc_env_00 requires a deterministic data-memory pre-load but does not say how it is shared with the golden model	Scoreboard correctness depends on it	Initialise data_mem_model and the reference model's memory from the same seed/data pool.
Q10	Two pre-existing overlaps in the diagram: cov overlaps rvfi_pas (~110x50 px) and intf_rvfi overlaps intf_lsu (~140x40 px)	Cosmetic only; the XML is still valid	Left untouched (instruction: preserve layout). Fix by nudging cov and intf_rvfi when convenient.
Q11	architecture/uvm_architecture_pro_updated.jpg is an export of the old diagram and is now out of date	A viewer may look at the stale picture	Re-export from the .drawio file (draw.io: File -> Export as -> JPEG). I cannot run draw.io in this environment; docs/architecture_preview.png is a programmatic preview in the meantime.
Q12	parameters/parameters.png could not be inspected (I have no image-viewing capability in this session)	It may contain configuration decisions	Please confirm it matches the default parameters in rtl/cv32e40p_top.sv (section 3.3).
Q13	risc_if_01 ("instr_req_o must not combinatorially depend on instr_rvalid_i") is checked in ORPLAN by counter balance (#(req&gnt) == #(rvalid)), not by a direct combinational-path check	A true combinational dependency could in principle pass the counter check	The RTL guarantees it structurally (PULP_OBI = 0), so this is acceptable; note it in the plan.
Q14	Databook says the divider latency depends on "operand b"; the decoder/ALU actually swap the operands so that the ALU's operand_a is rs2 (the divisor) and the divider's OpB is the (pre-shifted) divisor	Naming confusion when writing the latency predictor	Latency depends on the divisor (rs2) - as the Databook's examples (0x80000000, 0) confirm.

14. Appendix

14.1 Final quality checklist (as requested)

Check	Result
Databook was studied	[OK] chapters intro, integration, pipeline, instruction_fetch, load_store_unit, register_file, exceptions_interrupts, sleep, debug, verification (+ skimmed cs_registers, instruction_set_extensions)
RTL was studied	[OK] cv32e40p_top/core/if_stage/prefetch_*/obi_interface/id_stage/decoder/ex_stage/alu/alu_div/mult/load_store_unit/register_file_ff/controller/sleep_unit/cs_registers
Verification guidelines were studied	[OK] all six Team Q&A sheets; 13 rules quoted; 5 unanswered questions identified
ORPLAN was studied completely	[OK] all 5 sheets, all 49 requirement IDs, 38 Generation rows, 41 Checking rows, 37 Coverage rows, 27 tests
Scope came from the guidelines	[OK] section 6 (MUST VERIFY / VERIFY / OUT OF SCOPE / NEEDS CLARIFICATION), each row traced to a quoted answer
ORPLAN items connected to Databook requirements	[OK] sections 8 (each item) and 9 (mapping table, column "Source")
ORPLAN items connected to RTL	[OK] sections 8 (each item) and 9 (column "RTL location")

Check	Result
Every important item has an explained flow	[OK] sections 8 and 12
Existing architecture understood before modification	[OK] section 10 (layers, visual language, per-block role, coordinates)
Architecture changes based on ORPLAN	[OK] section 11 (each change names its ORPLAN ID)
Existing diagram style preserved	[OK] same palette/shapes/arrow styles; no block moved/resized/recoloured
Existing colours preserved	[OK] new blocks reuse #dae8fc (agent family), #fff2cc (checker family), #f5f5f5 (top layer)
Existing layout preserved	[OK] nothing moved
New components placed in empty space	[OK] all 6 verified programmatically to be overlap-free
No unnecessary redesign	[OK] section 11.8 lists what was deliberately not changed
Facts / inferences / unknowns separated	[OK] labelled throughout (FACT / INFERENCE / UNKNOWN)
Understandable by a junior student	[OK] section 2 teaches the fundamentals first; every term defined at first use

14.2 Files delivered / modified

File	Status	What it is
docs/VERIFICATION_STUDY.md	new	This document (the primary deliverable)
architecture/uvm_architecture_pro_updated.drawio	modified	The updated testbench architecture (6 new blocks, 9 new connections, 2 label edits)
docs/architecture_preview.png	new	A programmatic preview rendering of the updated .drawio (for quick viewing; the .drawio remains the source of truth)
architecture/uvm_architecture_pro_updated.jpg	unchanged, now stale	Old export - re-export from the .drawio (see Q11)

14.3 Quick reference: the 20 words a newcomer must know

DUT - testbench - transaction - sequence - sequencer - driver - monitor - agent - analysis port - reference model - scoreboard - covergroup - SVA - interface - OBI request/grant/response - retire - stall - flush - hazard - golden model.

End of document.